Machine-checked certificate

Theorems 1 and 2 of G. Adesso, *Optimality of Gaussian entanglement of formation*

arXiv:2608.01909v3

Lean 4 (v4.35.0-rc4) + Mathlib (commit `49fd9de`) · github.com/gerardo-adesso/gaussian-eof @ `3514b57`

9 October 2026

Summary

All statements below are proved in Lean 4 against Mathlib, with no `sorry`, `admit` or additional axioms; the kernel reports that each depends only on `propext`, `Classical.choice` and `Quot.sound` (Sec. 12). Statements are printed verbatim from the source; the full source is in the Appendix.

Theorem 1 (Part I): the singular-value defect inequality (Theorem S1) in the Schmidt-basis matrix model; the geometric isotonic compression (Theorem S2), including the pool-adjacent-violators construction and Schur concavity; the thermal closure (S:thermalF, S:support, S:Fmoments, S:UpsilonF); the assembly $E(\psi)+\lambda_{q,t}[D_t(\psi)-d_t(q)]\geq\mathcal{E}(q)$ for finite and infinite Schmidt rank; the endpoint $q=t$ and Eq. (S:fixedE) for $0\leq q\leq t$.

Theorem 2 (Part II): Lemma S5 in full (positivity, kernel, and $M(x)\geq\gamma K(x)$ with the explicit constant $\gamma=\kappa\sinh w-w\coth w-|\alpha-\beta|/2$); the derivation of Eqs. (S:defects) and (S:Q12) from the Euler equation; the identity (S:mainid); rigidity; the scalar reduction from the minimum principle to Theorem 2.

Not formalized (inputs)

Theorem 1.

- The meta log-Sobolev theorem of Beigi and Rahimi-Keshari, used only for diagonal states of finite rank (`BRKUpsilon`), together with the diagonal-state moments $\mathrm{Tr}(\rho_z a^\dagger a)=\bar{n}$, $\mathrm{Tr}(\sqrt{\rho_z}a\sqrt{\rho_z}a^\dagger)=T_z$.
- Fock-space facts: $E(\psi)=H(c^2)$, $D_t=G_t+1-t^2$, the matrix model $X=U^\dagger aU$, $Y=V^\dagger aV$ in Schmidt bases indexed by $\mathbb{N}$ (available for every finite-rank state), the index identities from $[a,a^\dagger]=1$, and, for general infinite-rank states, the convergence $G_t(C^{(L)})\to G_t(C)$ of normalized Schmidt truncations (then `theorem1_limit` applies).

Theorem 2.

- Operator facts on Fock space: the canonical commutation relations (giving $[T,O]$, the sum rules (S:wccr) and the matrix model with finitely many relevant directions); the TMSV moments and $\lambda\hat{D}_t=T+\lambda$.
- The second-variation formula $Q(Y)=\mathrm{Re}\langle Y,(T-\mu-\mathsf{F})Y\rangle$ (Daleckii–Krein), the Euler equation (S:euler) and the stability inequality (S:stability).
- The vacuum/gap statement (S:gap): the joint vacuum of the canonical annihilators f_j,e_j is $\bigotimes_j\Psi_{q_j}$.
- The infinite-dimensional analysis: infinite Schmidt rank, Lemma S6, regularization, existence of minimizers.

These require operator theory on infinite-dimensional Hilbert spaces (unbounded ladder operators, trace-class operators, von Neumann entropy) not yet available in Mathlib.

Part I: Theorem 1

1 Definitions

pos — Thm1Cert/Defect.lean

```
/-- Positive part. -/
def pos (x : ℝ) : ℝ := max x 0
```

dd — Thm1Cert/Defect.lean

```
/-- The positive drops `d_n = (t c_n − c_{n+1})₊`. -/
def dd (n : ℕ) : ℝ := pos (t * c n - c (n + 1))
```

dlt — Thm1Cert/Defect.lean

```
/-- Truncated positive drops: `δ_k = d_k` for `k ≤ N`, `0` otherwise. -/
def dlt (N k : ℕ) : ℝ := if k ≤ N then dd t c k else 0
```

alpha — Thm1Cert/Defect.lean

```
/-- `α_i = ∑_{k ≥ i} t c_k δ_k`. -/
def alpha (N i : ℕ) : ℝ := ∑ k ∈ Ico i (N + 1), t * c k * dlt t c N k
```

beta — Thm1Cert/Defect.lean

```
/-- `β_i = −∑_{k ≥ i} c_{k+1} δ_k`. -/
def beta (N i : ℕ) : ℝ := -∑ k ∈ Ico i (N + 1), c (k + 1) * dlt t c N k
```

wt — Thm1Cert/Compression.lean

```
/-- Weights `w_n = t^{2n}`. -/
def wt (n : ℕ) : ℝ := (t ^ n) ^ 2
```

mass — Thm1Cert/Compression.lean

```
/-- Total weighted mass `∑_{n ≤ M} w_n ℓ_n`. -/
def mass (ℓ : ℕ → ℝ) : ℝ := ∑ n ∈ range (M + 1), wt t n * ℓ n
```

pre — Thm1Cert/Compression.lean

```
/-- Prefix mass `∑_{n ≤ K} w_n ℓ_n`. -/
def pre (ℓ : ℕ → ℝ) (K : ℕ) : ℝ := ∑ n ∈ range (K + 1), wt t n * ℓ n
```

en — Thm1Cert/Compression.lean

```
/-- Drop energy (S:senergy): `∑_{n ≤ M} (n+1) t^{2n+2} ((√ℓ_n − √ℓ_{n+1})₊)²`. -/
def en (ℓ : ℕ → ℝ) : ℝ :=
  ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * (t ^ (n + 1)) ^ 2 * pos (√(ℓ n) - √(ℓ (n + 1))) ^ 2
```

Hs — Thm1Cert/Closure.lean

```
/-- Entropy `H(z²) = ∑ −z_n² ln z_n²` of a square-root sequence. -/
def Hs (z : ℕ → ℝ) : ℝ := ∑' n, negMulLog (z n ^ 2)
```

Ft — Thm1Cert/Closure.lean

```
/-- `𝓕_t(z) = ∑ (n+1)(t z_n − z_{n+1})²` (Eq. S:Fdef). -/
def Ft (t : ℝ) (z : ℕ → ℝ) : ℝ := ∑' n : ℕ, ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2
```

zs Thm1Cert/Closure.lean

```
/-- Thermal square-root sequence `z^{(s)}_n = √(1−s²) sⁿ` (Eq. S:thermalz). -/
def zs (s : ℝ) (n : ℕ) : ℝ := √(1 - s ^ 2) * s ^ n
```

phi Thm1Cert/Closure.lean

```
/-- `φ(s) = 𝓔(s) + λ (t−s)²/(1−s²)`. -/
def phi (L t s : ℝ) : ℝ := EE s + L * ((t - s) ^ 2 / (1 - s ^ 2))
```

nbar Thm1Cert/Upsilon.lean

```
/-- Mean photon number `n̄ = ∑ n z_n²`. -/
def nbar (z : ℕ → ℝ) : ℝ := ∑' n : ℕ, (n : ℝ) * z n ^ 2
```

Tz Thm1Cert/Upsilon.lean

```
/-- `T_z = ∑ (n+1) z_n z_{n+1} = Tr(√ρ_z a √ρ_z a†)` (Eq. S:Tz). -/
def Tz (z : ℕ → ℝ) : ℝ := ∑' n : ℕ, ((n : ℝ) + 1) * z n * z (n + 1)
```

Ups Thm1Cert/Upsilon.lean

```
/-- `Y(ρ_z)` for `ν₀ = λt`, `ν₁ = 0`, `ω = λ(1−t)²` (Eqs. S:Upsilon, S:parameters). -/
def Ups (t L : ℝ) (z : ℕ → ℝ) : ℝ :=
  2 * (L * t) * ((nbar z + 1) - Tz z) + L * (1 - t) ^ 2 * nbar z + Hs z
```

BRKUpsilon Thm1Cert/Upsilon.lean

```
/-- **Input (Beigi–Rahimi-Keshari)**, restricted to diagonal states of finite rank:
`Y(ρ_z) ≥ inf_{0 ≤ s < 1} Y(τ_s)`. -/
def BRKUpsilon (t L : ℝ) : Prop :=
  ∀ z : ℕ → ℝ, (∀ n, 0 ≤ z n) → (∃ M, ∀ n, M < n → z n = 0) → HasSum (fun n => z n ^ 2) 1 →
    ∀ c0 : ℝ, (∀ s, 0 ≤ s → s < 1 → c0 ≤ Ups t L (zs s)) → c0 ≤ Ups t L z
```

BRK Thm1Cert/Assembly.lean

```
/-- **Input (Beigi–Rahimi-Keshari, diagonal states of finite rank).** For every finitely
supported `z ≥ 0` with `∑ z_n² = 1`, `H(z²) + λ 𝓕_t(z)` is at least the infimum over thermal
states. (Y(ρ_z) = H(z²) + λ𝓕_t(z) + const for the parameters (S:parameters).) -/
def BRK (t L : ℝ) : Prop :=
  ∀ z : ℕ → ℝ, (∀ n, 0 ≤ z n) → (∃ M, ∀ n, M < n → z n = 0) → HasSum (fun n => z n ^ 2) 1 →
    ∀ c0 : ℝ, (∀ s, 0 ≤ s → s < 1 → c0 ≤ Hs (zs s) + L * Ft t (zs s)) → c0 ≤ Hs z + L * Ft t z
```

2 Theorem S1: singular-value defect inequality

drop_lemma Thm1Cert/Defect.lean

```
/-- `D_ij = ∑_{k=i}^{j-1} (t c_k − c_{k+1})₊ ≤ (t c_i − c_j)₊` for `i ≤ j`. -/
theorem drop_lemma {t : ℝ} {c : ℕ → ℝ} (ht0 : 0 ≤ t) (ht1 : t ≤ 1) (hc : Antitone c)
    (hc0 : ∀ n, 0 ≤ c n) {i j : ℕ} (hij : i ≤ j) :
    ∑ k ∈ Ico i j, dd t c k ≤ pos (t * c i - c j)
```

pointwise Thm1Cert/Defect.lean

```
/-- **Eq. (S:pointwise)**: `|t c_i x − c_j y|² ≥ (α_i − α_j)|x|² + (β_i − β_j)|y|²`. -/
theorem pointwise (ht0 : 0 ≤ t) (ht1 : t ≤ 1) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n)
    (N i j : ℕ) (x y : ℂ) :
    (alpha t c N i - alpha t c N j) * ‖x‖ ^ 2 + (beta t c N i - beta t c N j) * ‖y‖ ^ 2
      ≤ ‖((t * c i : ℝ) : ℂ) * x - (c j : ℂ) * y‖ ^ 2
```

defect_inequality Thm1Cert/Defect.lean

```
/-- **Theorem S1** (singular-value defect inequality), matrix model.
`hG`: `G = ∑_{i,j} |t c_i X_ij − c_j Y_ij|²`; `hX`, `hY`: the index identities (S:index),
required only for indices with `c_i > 0`. Conclusion: `∑_{n ≤ N} (n+1) d_n² ≤ G` for every `N`,
hence `∑_n (n+1) d_n²` converges and is `≤ G`. -/
theorem defect_inequality (ht0 : 0 < t) (ht1 : t ≤ 1) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n)
    (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) (N : ℕ) :
    ∑ k ∈ range (N + 1), ((k : ℝ) + 1) * dd t c k ^ 2 ≤ G
```

defect_inequality_tsum Thm1Cert/Defect.lean

```
/-- Consequence: the defect series converges and `∑_n (n+1) d_n² ≤ G_t(C)`. -/
theorem defect_inequality_tsum (ht0 : 0 < t) (ht1 : t ≤ 1) (hc : Antitone c)
    (hc0 : ∀ n, 0 ≤ c n) (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    Summable (fun n : ℕ => ((n : ℝ) + 1) * dd t c n ^ 2) ∧
    ∑' n : ℕ, ((n : ℝ) + 1) * dd t c n ^ 2 ≤ G
```

3 Theorem S2: geometric isotonic compression

pool_step Thm1Cert/Compression.lean

```
/-- **Pooling step.** If two adjacent levels violate monotonicity, pooling the two maximal
constant runs around them preserves the invariants and strictly reduces the number of breaks. -/
theorem pool_step (ht : 0 < t) {r ℓ : ℕ → ℝ} (hI : Inv M t r ℓ) {n : ℕ} (hn : n < M)
    (hv : ℓ n < ℓ (n + 1)) : ∃ ℓ', Inv M t r ℓ' ∧ brk M ℓ' < brk M ℓ
```

pava_exists Thm1Cert/Compression.lean

```
/-- **Pool-adjacent-violators, existence form.** There are levels satisfying the invariants
that are nonincreasing on `[0, M]`. -/
theorem pava_exists (ht : 0 < t) {r : ℕ → ℝ} (hI : Inv M t r r) :
    ∃ ℓ, Inv M t r ℓ ∧ ∀ n, n < M → ℓ (n + 1) ≤ ℓ n
```

entropy_le_of_prefix Thm1Cert/Compression.lean

```
/-- If `p` is positive and nonincreasing on `[0, M]` and every prefix sum of `q ≥ 0` dominates
that of `p`, with equal totals, then `H(q) ≤ H(p)`. -/
```

```
theorem entropy_le_of_prefix (p q : ℕ → ℝ) (hp : ∀ n, n ≤ M → 0 < p n)
    (hanti : ∀ n, n < M → p (n + 1) ≤ p n) (hq : ∀ n, 0 ≤ q n)
    (hpre : ∀ K, K ≤ M → ∑ n ∈ range (K + 1), p n ≤ ∑ n ∈ range (K + 1), q n)
    (htot : ∑ n ∈ range (M + 1), p n = ∑ n ∈ range (M + 1), q n) :
    ∑ n ∈ range (M + 1), negMulLog (q n) ≤ ∑ n ∈ range (M + 1), negMulLog (p n)
```

compression Thm1Cert/Compression.lean

```
/-- **Theorem S2** (geometric isotonic compression). -/
theorem compression (ht : 0 < t) (c : ℕ → ℝ) (hcpos : ∀ n, n ≤ M → 0 < c n)
    (hanti : ∀ n, n < M → c (n + 1) ≤ c n) (hsupp : ∀ n, M < n → c n = 0)
    (hnorm : ∑ n ∈ range (M + 1), c n ^ 2 = 1) :
    ∃ z : ℕ → ℝ, (∀ n, 0 ≤ z n) ∧ (∀ n, M < n → z n = 0) ∧
      ∑ n ∈ range (M + 1), z n ^ 2 = 1 ∧ (∀ n, z (n + 1) ≤ t * z n) ∧
      ∑ n ∈ range (M + 1), negMulLog (z n ^ 2) ≤ ∑ n ∈ range (M + 1), negMulLog (c n ^ 2) ∧
      ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2
        ≤ ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * pos (t * c n - c (n + 1)) ^ 2
```

4 Thermal closure

H_thermal Thm1Cert/Closure.lean

```
/-- **Eq. (S:thermalF), entropy part**: `H((z^{(s)})²) = 𝓔(s)`. -/
theorem H_thermal {s : ℝ} (hs0 : 0 ≤ s) (hs1 : s < 1) : Hs (zs s) = EE s
```

F_thermal Thm1Cert/Closure.lean

```
/-- **Eq. (S:thermalF), energy part**: `𝓕_t(z^{(s)}) = (t−s)²/(1−s²)`. -/
theorem F_thermal (t : ℝ) {s : ℝ} (hs0 : 0 ≤ s) (hs1 : s < 1) :
    Ft t (zs s) = (t - s) ^ 2 / (1 - s ^ 2)
```

support Thm1Cert/Closure.lean

```
/-- **Eq. (S:support)**: for `0 < q < t ≤ 1` and `λ = λ_{q,t}`,
`𝓔(s) + λ(t−s)²/(1−s²) ≥ 𝓔(q) + λ(t−q)²/(1−q²)` for all `0 ≤ s < 1`. -/
theorem support {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) {s : ℝ} (hs0 : 0 ≤ s)
    (hs1 : s < 1) : phi (lam q t) t q ≤ phi (lam q t) t s
```

Fmoments Thm1Cert/Upsilon.lean

```
/-- **Eq. (S:Fmoments)** in `HasSum` form. -/
lemma Fmoments (t : ℝ) (z : ℕ → ℝ) {N T : ℝ} (h1 : HasSum (fun n => z n ^ 2) 1)
    (hN : HasSum (fun n : ℕ => (n : ℝ) * z n ^ 2) N)
    (hT : HasSum (fun n : ℕ => ((n : ℝ) + 1) * z n * z (n + 1)) T) :
    HasSum (fun n : ℕ => ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2)
      (t ^ 2 * (N + 1) + N - 2 * t * T)
```

Ups_eq Thm1Cert/Upsilon.lean

```
/-- **Eq. (S:UpsilonF)**: `Υ(ρ_z) = H(z²) + λ𝓕_t(z) + λ(2t − t²)`. -/
lemma Ups_eq (t L : ℝ) (z : ℕ → ℝ) (h1 : HasSum (fun n => z n ^ 2) 1)
    (hN : Summable (fun n : ℕ => (n : ℝ) * z n ^ 2))
    (hT : Summable (fun n : ℕ => ((n : ℝ) + 1) * z n * z (n + 1))) :
    Ups t L z = Hs z + L * Ft t z + L * (2 * t - t ^ 2)
```

brk_of_upsilon Thm1Cert/Upsilon.lean

```
/-- The Beigi–Rahimi-Keshari input in Υ form implies the sequence form `BRK`. -/
theorem brk_of_upsilon {t L : ℝ} (hU : BRKUpsilon t L) : BRK t L
```

5 Theorem 1 and corollaries

thm1_spectrum — Thm1Cert/Assembly.lean

```
/-- **Theorem 1, spectral form.** -/
theorem thm1_spectrum (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) (hB : BRK t (lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (hD : Summable (fun n : ℕ => ((n : ℝ) + 1) * dd t c n ^ 2)) :
    phi (lam q t) t q
      ≤ ∑' n, negMulLog (c n ^ 2) + lam q t * ∑' n : ℕ, ((n : ℝ) + 1) * dd t c n ^ 2
```

theorem1 — Thm1Cert/Assembly.lean

```
/-- **Theorem 1** (affine extremality), matrix model. With `E(ψ) = H(c²)` (Eq. S:entropy) and
`D_t(ψ) = G + 1 − t²` (Eq. S:Gdef), where `G = ∑ |t c_i X_ij − c_j Y_ij|²` (Eq. S:Gmatrix):
`E(ψ) + λ_{q,t} [D_t(ψ) − d_t(q)] ≥ 𝓔(q)` for all `0 < q < t ≤ 1`. -/
theorem theorem1 (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) (hB : BRK t (lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    EE q ≤ ∑' n, negMulLog (c n ^ 2) + lam q t * ((G + 1 - t ^ 2) - dt t q)
```

theorem1_upsilon — Thm1Cert/Upsilon.lean

```
/-- **Theorem 1** with the Beigi–Rahimi-Keshari input in its Y form. -/
theorem theorem1_upsilon {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1)
    (hU : BRKUpsilon t (Thm2Cert.lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    Thm2Cert.EE q ≤ ∑' n, negMulLog (c n ^ 2)
      + Thm2Cert.lam q t * ((G + 1 - t ^ 2) - Thm2Cert.dt t q)
```

theorem1_limit — Thm1Cert/Corollaries.lean

```
/-- **Limit step.** If the affine bound holds along a sequence of states with `E_L → E` and
`D_L → D`, it holds in the limit. -/
theorem theorem1_limit {q t : ℝ} (E D : ℝ) (EL DL : ℕ → ℝ)
    (h : ∀ L, EE q ≤ EL L + lam q t * (DL L - dt t q))
    (hE : Tendsto EL atTop (𝓝 E)) (hD : Tendsto DL atTop (𝓝 D)) :
    EE q ≤ E + lam q t * (D - dt t q)
```

entropy_truncation — Thm1Cert/Corollaries.lean

```
/-- Entropy of the normalized Schmidt truncations converges: with `P_M = ∑_{n ≤ M} c_n²`,
`H((c^{(M)})²) = S_M/P_M + ln P_M → H(c²)`. -/
```

```
theorem entropy_truncation (c : ℕ → ℝ) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2))) :
    Tendsto (fun M : ℕ => (∑ n ∈ range (M + 1), negMulLog (c n ^ 2)) / (∑ n ∈ range (M + 1), c n ^ 2)
      + log (∑ n ∈ range (M + 1), c n ^ 2)) atTop (𝓝 (∑' n, negMulLog (c n ^ 2)))
```

theorem1_endpoint — Thm1Cert/Assembly.lean

```
/-- **Eq. (S:endpoint)**: for `t < 1`, if the affine bound holds for all `q < t` and
`D_t(ψ) = d_t(t) = 1 − t²`, then `E(ψ) ≥ 𝓔(t)`. -/
theorem theorem1_endpoint (ht0 : 0 < t) (ht1 : t < 1) {E : ℝ}
    (h : ∀ q, 0 < q → q < t → EE q ≤ E + lam q t * (dt t t - dt t q)) : EE t ≤ E
```

theorem1_endpoint_state — Thm1Cert/Upsilon.lean

```
/-- **Endpoint `q = t < 1`** (Eq. S:endpoint): a state with `D_t(ψ) = d_t(t) = 1 − t²`
(i.e. `G = 0`) has `E(ψ) ≥ 𝓔(t)`. -/
theorem theorem1_endpoint_state {t : ℝ} (ht0 : 0 < t) (ht1 : t < 1)
    (hU : ∀ q, 0 < q → q < t → BRKUpsilon t (Thm2Cert.lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (X Y : ℕ → ℕ → ℂ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) 0)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    Thm2Cert.EE t ≤ ∑' n, negMulLog (c n ^ 2)
```

fixed_entanglement — Thm1Cert/Assembly.lean

```
/-- **Eq. (S:fixedE)**: `E(ψ) ≤ 𝓔(q)` implies `D_t(ψ) ≥ d_t(q)`. -/
theorem fixed_entanglement (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) {E D : ℝ}
    (h1 : EE q ≤ E + lam q t * (D - dt t q)) (hE : E ≤ EE q) : dt t q ≤ D
```

fixed_entanglement_endpoint — Thm1Cert/Corollaries.lean

```
/-- **Eq. (S:fixedE) at `q = t`**: `D_t(ψ) = G + 1 − t² ≥ 1 − t² = d_t(t)` since `G ≥ 0`. -/
theorem fixed_entanglement_endpoint {t G : ℝ} (ht0 : 0 < t) (ht1 : t < 1) (hG : 0 ≤ G) :
    dt t t ≤ G + 1 - t ^ 2
```

separability_criterion — Thm1Cert/Corollaries.lean

```
/-- **Separability criterion (`q = 0` in Eq. S:fixedE)**: if the affine bound holds for all
`0 < q < t` and `E(ψ) ≤ 0` (a product state), then `D_t(ψ) ≥ 1 = d_t(0)`. -/
theorem separability_criterion {t E D : ℝ} (ht0 : 0 < t) (ht : t ≤ 1)
    (h : ∀ q, 0 < q → q < t → EE q ≤ E + lam q t * (D - dt t q)) (hE : E ≤ 0) : 1 ≤ D
```

Part II: Theorem 2

6 Definitions

f — Thm2Cert/MatrixLemma.lean

```
/-- `f x = x / sinh x`, extended continuously by `f 0 = 1`. -/
def f (x : ℝ) : ℝ := if x = 0 then 1 else x / sinh x
```

b — Thm2Cert/MatrixLemma.lean

```
/-- `b x = x coth x`, written as `f x * cosh x` (so that `b 0 = 1`). -/
def b (x : ℝ) : ℝ := f x * cosh x
```

Mdiag — Thm2Cert/MatrixLemma.lean

```
/-- Diagonal entry of `M(x)` for a general `q` (`a = α` or `a = β`). -/
def Mdiag (κ a q x : ℝ) : ℝ := q * (κ - q * a) * exp x + (a - q * κ) * exp (-x) + (1 + q ^ 2) * f x
```

Moff — Thm2Cert/MatrixLemma.lean

```
/-- Off-diagonal entry of `M(x)` for a general `q`. -/
def Moff (κ q x : ℝ) : ℝ := -κ * (1 - q ^ 2) - 2 * q * b x
```

Hyp — Thm2Cert/MatrixLemma.lean

```
/-- Hypotheses of Lemma S5: `α, β, κ > 0`, `αβ ≥ κ²`, and `w > 0` solves the calibration
`κ cosh w − w = (α+β)/2` (then `q = e^{−w}`). -/
structure Hyp (κ α β w : ℝ) : Prop where
  hκ : 0 < κ
  hα : 0 < α
  hβ : 0 < β
  hw : 0 < w
  hκαβ : κ ^ 2 ≤ α * β
  hcal : κ * cosh w - w = (α + β) / 2
```

QM — Thm2Cert/MatrixLemma.lean

```
/-- The quadratic form of `M(x)`, `q = e^{−w}`. -/
def QM (κ α β w x u v : ℝ) : ℝ :=
  Mdiag κ α (exp (-w)) x * u ^ 2 + 2 * Moff κ (exp (-w)) x * u * v + Mdiag κ β (exp (-w)) x * v ^ 2
```

Kdiag — Thm2Cert/Comparison.lean

```
/-- Diagonal entry of `K(x)`: `e^{−x} + q² eˣ` (the off-diagonal entry is `−2q`). -/
def Kdiag (q x : ℝ) : ℝ := exp (-x) + q ^ 2 * exp x
```

QK — Thm2Cert/Comparison.lean

```
/-- The quadratic form of `K(x)`. -/
def QK (w x u v : ℝ) : ℝ :=
  Kdiag (exp (-w)) x * u ^ 2 + 2 * (-2 * exp (-w)) * u * v + Kdiag (exp (-w)) x * v ^ 2
```

gam — Thm2Cert/Comparison.lean

```
/-- `γ = κ sinh w − w coth w − |α − β|/2`. -/
def gam (κ α β w : ℝ) : ℝ := κ * sinh w - w * cosh w / sinh w - |α - β| / 2
```

Lker — Thm2Cert/Identity.lean

```
/-- The logarithmic mean kernel `L(p,p') = (ln p − ln p')/(p − p')`, `= 1/p` on the diagonal. -/
def Lker (p p' : ℝ) : ℝ := if p = p' then 1 / p else (log p - log p') / (p - p')
```

Fdk — Thm2Cert/Derivation.lean

```
/-- `F(p, p') = (p ln p − p' ln p')/(p − p')`, `= 1 + ln p` on the diagonal. -/
def Fdk (p p' : ℝ) : ℝ := if p = p' then 1 + log p else (p * log p - p' * log p') / (p - p')
```

kk — Thm2Cert/Identity.lean

```
/-- `k = Re⟨a b⟩ = ∑ c_l c_r Re(A_lr B_lr)`. -/
def kk (c : ι → ℝ) (A B : ι → ι → ℂ) : ℝ := ∑ l, ∑ r, c l * c r * (A l r * B l r).re
```

sfA — Thm2Cert/Identity.lean

```
/-- `A = ∑ p_l p_r L_lr |A_lr|²` (and `B` with `B`). -/
def sfA (c : ι → ℝ) (A : ι → ι → ℂ) : ℝ :=
  ∑ l, ∑ r, c l ^ 2 * c r ^ 2 * Lker (c l ^ 2) (c r ^ 2) * Complex.normSq (A l r)
```

sfC — Thm2Cert/Identity.lean

```
/-- `C = ∑ √(p_l p_r) (p_l + p_r) L_lr Re(A_lr B_lr)`. -/
def sfC (c : ι → ℝ) (A B : ι → ι → ℂ) : ℝ :=
  ∑ l, ∑ r, √(c l ^ 2 * c r ^ 2) * (c l ^ 2 + c r ^ 2) * Lker (c l ^ 2) (c r ^ 2) * (A l r * B l r).re
```

Q1 — Thm2Cert/Identity.lean

```
/-- Eq. (S:Q12), first line: `Q(O₁ψ)`. -/
def Q1 (α β κ q nA nB k sA sB sC : ℝ) : ℝ :=
  (q * κ - α) * nA + (q ^ 2 * β - q * κ) * (nB + 1) + (κ * (1 - q ^ 2) + q * (α - β)) * k
    - sA - q ^ 2 * sB + q * sC
```

Q2 — Thm2Cert/Identity.lean

```
/-- Eq. (S:Q12), second line: `Q(O₂ψ)`. -/
def Q2 (α β κ q nA nB k sA sB sC : ℝ) : ℝ :=
  (q ^ 2 * α - q * κ) * (nA + 1) + (q * κ - β) * nB + (κ * (1 - q ^ 2) - q * (α - β)) * k
    - q ^ 2 * sA - sB + q * sC
```

O1v — Thm2Cert/Derivation.lean

```
/-- Coefficient matrix of `O₁ψ = (a − q b†)ψ`. -/
def O1v (q : ℝ) (c : ι → ℝ) (A B : ι → ι → ℂ) (l m : ι) : ℂ :=
  A l m * (c m : ℂ) - (q : ℂ) * (c l : ℂ) * conj (B l m)
```

O2v — Thm2Cert/Derivation.lean

```
/-- Coefficient matrix of `O₂ψ = (q a† − b)ψ`. -/
def O2v (q : ℝ) (c : ι → ℝ) (A B : ι → ι → ℂ) (l m : ι) : ℂ :=
  (q : ℂ) * (c m : ℂ) * conj (A m l) - (c l : ℂ) * B m l
```

kterm — Thm2Cert/Comparison.lean

```
/-- Summand of `‖O₁ψ‖² + ‖O₂ψ‖² = ∑ c_l c_r z† K(x_lr) z` (Eq. S:defects) in the
`p`-variables: `c_l c_r K₁₁ = p_r + q² p_l`, `c_l c_r K₁₂ = −2q c_l c_r`. -/
def kterm (q cl cr : ℝ) (A B : ℂ) : ℝ :=
  (cr ^ 2 + q ^ 2 * cl ^ 2) * (Complex.normSq A + Complex.normSq B)
    - 4 * q * (cl * cr) * (A * B).re
```

term Thm2Cert/Identity.lean

```
/-- The summand of Eq. (S:mainid), for Schmidt coefficients `cl, cr ≥ 0`. -/
def term (κ α β q cl cr : ℝ) (A B : ℂ) : ℝ :=
  wdiag κ α q (cl ^ 2) (cr ^ 2) (cl ^ 2 * cr ^ 2 * Lker (cl ^ 2) (cr ^ 2)) * Complex.normSq A
  + wdiag κ β q (cl ^ 2) (cr ^ 2) (cl ^ 2 * cr ^ 2 * Lker (cl ^ 2) (cr ^ 2)) * Complex.normSq B
  + 2 * woff κ q (cl * cr) (√(cl ^ 2 * cr ^ 2) * (cl ^ 2 + cr ^ 2) * Lker (cl ^ 2) (cr ^ 2))
      * (A * B).re
```

lam Thm2Cert/Reduction.lean

```
/-- `λ_{q,t} = −2 q ln q / [(t − q)(1 − t q)]`. -/
def lam (q t : ℝ) : ℝ := -2 * q * log q / ((t - q) * (1 - t * q))
```

dt Thm2Cert/Reduction.lean

```
/-- `d_t(q) = (1 − t q)² / (1 − q²)`. -/
def dt (t q : ℝ) : ℝ := (1 - t * q) ^ 2 / (1 - q ^ 2)
```

gfun Thm2Cert/Reduction.lean

```
/-- `g(y) = (y+1) ln(y+1) − y ln y`. -/
def gfun (y : ℝ) : ℝ := (y + 1) * log (y + 1) - y * log y
```

EE Thm2Cert/Reduction.lean

```
/-- `ℰ(q) = g(q²/(1−q²))`, the entanglement of the TMSV. -/
def EE (q : ℝ) : ℝ := gfun (q ^ 2 / (1 - q ^ 2))
```

7 Lemma S5: positivity and kernel

hyp_nonvacuous Thm2Cert/MatrixLemma.lean

```
/-- Sanity check: the hypotheses are satisfiable (`w = 1`, `κ = 2/(cosh 1 − 1)`,
`α = β = κ cosh 1 − 1`). -/
theorem hyp_nonvacuous : ∃ κ α β w : ℝ, Hyp κ α β w
```

lemmaS5_psd Thm2Cert/MatrixLemma.lean

```
/-- **Lemma S5 (i)**: `M(x) ≥ 0` for every real `x`. -/
theorem lemmaS5_psd (h : Hyp κ α β w) (x u v : ℝ) : 0 ≤ QM κ α β w x u v
```

lemmaS5_pd Thm2Cert/MatrixLemma.lean

```
/-- **Lemma S5 (ii)**: `M(x)` is positive definite for `x ≠ w`, i.e. `ker M(x) = {0}`. -/
theorem lemmaS5_pd (h : Hyp κ α β w) {x : ℝ} (hx : x ≠ w) {u v : ℝ} (huv : u ≠ 0 ∨ v ≠ 0) :
    0 < QM κ α β w x u v
```

lemmaS5_kernel Thm2Cert/MatrixLemma.lean

```
/-- **Lemma S5 (iii)**: at `x = w`, the quadratic form vanishes exactly on `span{(1,1)}`,
i.e. `ker M(w) = span{(1,1)}`. -/
theorem lemmaS5_kernel (h : Hyp κ α β w) (u v : ℝ) : QM κ α β w w u v = 0 ↔ u = v
```

lemmaS5_null Thm2Cert/MatrixLemma.lean

```
/-- `M(w) (1,1)ᵀ = 0`, entrywise. -/
theorem lemmaS5_null :
    Mdiag κ α (exp (-w)) w + Moff κ (exp (-w)) w = 0 ∧
    Moff κ (exp (-w)) w + Mdiag κ β (exp (-w)) w = 0
```

lemmaS5_complex Thm2Cert/MatrixLemma.lean

```
/-- Complex form used in Eq. (S:mainid): for `z = (A, conj B)`,
`z† M(x) z = M₁₁|A|² + M₂₂|B|² + 2 M₁₂ Re(AB) ≥ 0`. -/
theorem lemmaS5_complex (h : Hyp κ α β w) (x : ℝ) (A B : ℂ) :
    0 ≤ Mdiag κ α (exp (-w)) x * Complex.normSq A + Mdiag κ β (exp (-w)) x * Complex.normSq B
      + 2 * Moff κ (exp (-w)) x * (A * B).re
```

lemmaS5_complex_pd Thm2Cert/MatrixLemma.lean

```
/-- Complex strict form: for `x ≠ w` and `z = (A, conj B) ≠ 0`, `z† M(x) z > 0`. -/
theorem lemmaS5_complex_pd (h : Hyp κ α β w) {x : ℝ} (hx : x ≠ w) {A B : ℂ}
    (hAB : A ≠ 0 ∨ B ≠ 0) :
    0 < Mdiag κ α (exp (-w)) x * Complex.normSq A + Mdiag κ β (exp (-w)) x * Complex.normSq B
      + 2 * Moff κ (exp (-w)) x * (A * B).re
```

lemmaS5_complex_kernel Thm2Cert/MatrixLemma.lean

```
/-- Complex kernel at `x = w`: `z† M(w) z = 0` iff `z = (A, conj B)` lies in `span{(1,1)}`. -/
theorem lemmaS5_complex_kernel (h : Hyp κ α β w) (A B : ℂ) :
    Mdiag κ α (exp (-w)) w * Complex.normSq A + Mdiag κ β (exp (-w)) w * Complex.normSq B
      + 2 * Moff κ (exp (-w)) w * (A * B).re = 0 ↔ A = (starRingEnd ℂ) B
```

8 Lemma S5: comparison with explicit constant

gam_pos Thm2Cert/Comparison.lean

```
theorem gam_pos (h : Hyp κ α β w) : 0 < gam κ α β w
```

lemmaS5_gamma Thm2Cert/Comparison.lean

```
/-- **Lemma S5 (iv)**: `M(x) ≥ γ K(x)` for every real `x`, with the explicit
`γ = κ sinh w − w coth w − |α−β|/2 > 0` (`gam_pos`). -/
theorem lemmaS5_gamma (h : Hyp κ α β w) (x u v : ℝ) :
    gam κ α β w * QK w x u v ≤ QM κ α β w x u v
```

lemmaS5_gamma_complex Thm2Cert/Comparison.lean

```
/-- Complex form of `M(x) ≥ γ K(x)`, for `z = (A, conj B)`. -/
theorem lemmaS5_gamma_complex (h : Hyp κ α β w) (x : ℝ) (A B : ℂ) :
    gam κ α β w * (Kdiag (exp (-w)) x * (Complex.normSq A + Complex.normSq B)
        + 2 * (-2 * exp (-w)) * (A * B).re)
      ≤ Mdiag κ α (exp (-w)) x * Complex.normSq A + Mdiag κ β (exp (-w)) x * Complex.normSq B
        + 2 * Moff κ (exp (-w)) x * (A * B).re
```

9 Eqs. (S:defects), (S:Q12), (S:mainid)

defects Thm2Cert/Derivation.lean

```
/-- **Eq. (S:defects)**: `‖O₁ψ‖² + ‖O₂ψ‖² = ∑_{l,r} c_l c_r z† K(x_lr) z` (in `p`-variables). -/
theorem defects (q : ℝ) (c : ι → ℝ) (A B : ι → ι → ℂ) :
    ∑ l, ∑ m, Complex.normSq (O1v q c A B l m) + ∑ l, ∑ m, Complex.normSq (O2v q c A B l m)
      = ∑ l, ∑ r, kterm q (c l) (c r) (A l r) (B l r)
```

Q1_from_euler Thm2Cert/Derivation.lean

```
/-- **Eq. (S:Q12), first line**, derived from the Euler equation and `[T, O₁]`. -/
theorem Q1_from_euler (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : ι → ι → ℂ)
    (α β κ q nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    (∑ l, ∑ m, conj (O1v q c A B l m)
      * ((A l m * ((c m * log (c m ^ 2) : ℝ) : ℂ)
          - (q : ℂ) * ((c l * log (c l ^ 2) : ℝ) : ℂ) * conj (B l m)
          + ((q * κ - α : ℝ) : ℂ) * A l m * (c m : ℂ)
          + ((κ - q * β : ℝ) : ℂ) * (c l : ℂ) * conj (B l m))
        - (Fdk (c l ^ 2) (c m ^ 2) : ℂ) * O1v q c A B l m)).re
    = Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
```

Q2_from_euler Thm2Cert/Derivation.lean

```
/-- **Eq. (S:Q12), second line**, derived from the Euler equation and `[T, O₂]`. -/
theorem Q2_from_euler (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : ι → ι → ℂ)
    (α β κ q nA nB : ℝ)
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r)) :
    (∑ l, ∑ m, conj (O2v q c A B l m)
      * (((q : ℂ) * ((c m * log (c m ^ 2) : ℝ) : ℂ) * conj (A m l)
          - ((c l * log (c l ^ 2) : ℝ) : ℂ) * B m l
          + ((q * α - κ : ℝ) : ℂ) * (c m : ℂ) * conj (A m l)
          + ((β - q * κ : ℝ) : ℂ) * (c l : ℂ) * B m l)
        - (Fdk (c l ^ 2) (c m ^ 2) : ℂ) * O2v q c A B l m)).re
    = Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
```

mainid Thm2Cert/Identity.lean

```
/-- **Eq. (S:mainid)** (exact bookkeeping identity), for an arbitrary real `q`. -/
theorem mainid (c : ι → ℝ) (hc : ∀ l, 0 < c l) (A B : ι → ι → ℂ) (α β κ q nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
    = -∑ l, ∑ r, c l * c r *
        (Mdiag κ α q (log (c l / c r)) * Complex.normSq (A l r)
          + Mdiag κ β q (log (c l / c r)) * Complex.normSq (B l r)
          + 2 * Moff κ q (log (c l / c r)) * (A l r * B l r).re)
```

mainid_general Thm2Cert/Identity.lean

```
/-- **Eq. (S:mainid), general form**: valid for all real `c_l` (no positivity needed). -/
theorem mainid_general (c : ι → ℝ) (A B : ι → ι → ℂ) (α β κ q nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
    = -∑ l, ∑ r, term κ α β q (c l) (c r) (A l r) (B l r)
```

10 Rigidity

rigidity Thm2Cert/Comparison.lean

```
/-- **Rigidity inequality** for one pair: `Q(O₁ψ) + Q(O₂ψ) ≤ −γ ∑ c_l c_r z† K z`. -/
theorem rigidity (h : Hyp κ α β w) (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l)
```

```
    (A B : ι → ι → ℂ) (nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      ≤ -(gam κ α β w * ∑ l, ∑ r, kterm (exp (-w)) (c l) (c r) (A l r) (B l r))
```

rigidity_multi — Thm2Cert/Comparison.lean

```
/-- **Rigidity, many pairs.** For `m` pairs `j` (all sharing the Schmidt coefficients `c`)
with hypotheses `Hyp κ_j α_j β_j w_j`, and any `0 < γ₀ ≤ min_j γ_j`:
`∑_j [Q(O₁,ⱼψ) + Q(O₂,ⱼψ)] ≤ −γ₀ ∑_j ∑ c_l c_r z†_j K_j z_j`. In particular, if the left side
is `≥ 0` (second-order stability with `⟨O_{i,j}⟩ = 0`), every defect sum vanishes. -/
theorem rigidity_multi {m : ℕ} (κ α β w : Fin m → ℝ) (h : ∀ j, Hyp (κ j) (α j) (β j) (w j))
    (γ0 : ℝ) (hγ0 : 0 < γ0) (hγ : ∀ j, γ0 ≤ gam (κ j) (α j) (β j) (w j))
    (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : Fin m → ι → ι → ℂ) (nA nB : Fin m → ℝ)
    (hA0 : ∀ j, nA j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A j l r))
    (hA1 : ∀ j, nA j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A j l r))
    (hB0 : ∀ j, nB j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B j l r))
    (hB1 : ∀ j, nB j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B j l r)) :
    (∑ j, (Q1 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))
      + Q2 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))))
      ≤ -(γ0 * ∑ j, ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r)) ∧
    (0 ≤ ∑ j, (Q1 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))
      + Q2 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))) →
      ∀ j, ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r) = 0)
```

stationary_rigidity — Thm2Cert/Derivation.lean

```
/-- **Rigidity.** For `m` pairs with the hypotheses of Lemma S5, Schmidt coefficients `c ≥ 0`
and the sum rules, if the second variations (S:Q12) along all test vectors `O₁,ⱼψ, O₂,ⱼψ` −
computed from the Euler equation − sum to a nonnegative number, then every test vector
vanishes: `(a_j − q_j b_j†)ψ = 0` and `(q_j a_j† − b_j)ψ = 0` for all `j`. -/
theorem stationary_rigidity {m : ℕ} (κ α β w : Fin m → ℝ)
    (h : ∀ j, Hyp (κ j) (α j) (β j) (w j))
    (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : Fin m → ι → ι → ℂ) (nA nB : Fin m → ℝ)
    (hA0 : ∀ j, nA j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A j l r))
    (hA1 : ∀ j, nA j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A j l r))
    (hB0 : ∀ j, nB j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B j l r))
    (hB1 : ∀ j, nB j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B j l r))
    (hstab : 0 ≤ ∑ j,
      ((∑ l, ∑ m', conj (O1v (exp (-w j)) c (A j) (B j) l m')
        * ((A j l m' * ((c m' * log (c m' ^ 2) : ℝ) : ℂ)
            - ((exp (-w j) : ℝ) : ℂ) * ((c l * log (c l ^ 2) : ℝ) : ℂ) * conj (B j l m')
            + ((exp (-w j) * κ j - α j : ℝ) : ℂ) * A j l m' * (c m' : ℂ)
            + ((κ j - exp (-w j) * β j : ℝ) : ℂ) * (c l : ℂ) * conj (B j l m'))
          - (Fdk (c l ^ 2) (c m' ^ 2) : ℂ) * O1v (exp (-w j)) c (A j) (B j) l m')).re
      + (∑ l, ∑ m', conj (O2v (exp (-w j)) c (A j) (B j) l m')
        * ((((exp (-w j) : ℝ) : ℂ) * ((c m' * log (c m' ^ 2) : ℝ) : ℂ) * conj (A j m' l)
            - ((c l * log (c l ^ 2) : ℝ) : ℂ) * B j m' l
            + ((exp (-w j) * α j - κ j : ℝ) : ℂ) * (c m' : ℂ) * conj (A j m' l)
            + ((β j - exp (-w j) * κ j : ℝ) : ℂ) * (c l : ℂ) * B j m' l)
          - (Fdk (c l ^ 2) (c m' ^ 2) : ℂ) * O2v (exp (-w j)) c (A j) (B j) l m')).re)) :
    ∀ j l m', O1v (exp (-w j)) c (A j) (B j) l m' = 0 ∧ O2v (exp (-w j)) c (A j) (B j) l m' = 0
```

11 Reduction to Theorem 2

calib_iff — Thm2Cert/Reduction.lean

```
/-- The calibration for `(α, β, κ) = λ(t², 1, t)` holds iff `λ = λ_{q,t}`, `q = e^{−w}`. -/
theorem calib_iff {w t l : ℝ} (hw : 0 < w) (hqt : exp (-w) < t) (ht : t ≤ 1) :
    (l * t) * cosh w - w = (l * t ^ 2 + l * 1) / 2 ↔ l = lam (exp (-w)) t
```

lam_pos Thm2Cert/Reduction.lean

```
lemma lam_pos {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) : 0 < lam q t
```

hyp_of_lam Thm2Cert/Reduction.lean

```
/-- The parameters of Theorem 2 satisfy the hypotheses of Lemma S5: for `0 < q < t ≤ 1`,
`w = −ln q`, `λ = λ_{q,t}`, the triple `(α, β, κ) = λ(t², 1, t)` (and the swapped one
`λ(1, t², t)`) satisfies `Hyp`, with `κ² = αβ`. -/
theorem hyp_of_lam {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) :
    Hyp (lam q t * t) (lam q t * t ^ 2) (lam q t * 1) (-log q) ∧
    Hyp (lam q t * t) (lam q t * 1) (lam q t * t ^ 2) (-log q)
```

tmsv_value Thm2Cert/Reduction.lean

```
/-- Equality in the minimum principle on the two-mode squeezed vacuum (scalar part). -/
theorem tmsv_value {κ α β w : ℝ} (hcal : κ * cosh w - w = (α + β) / 2) (hw : 0 < w) :
    EE (exp (-w)) + (α + β) * (exp (-w) ^ 2 / (1 - exp (-w) ^ 2))
      - κ * (2 * exp (-w) / (1 - exp (-w) ^ 2))
      = -log (1 - exp (-w) ^ 2) - κ * exp (-w)
```

energy_identity Thm2Cert/Reduction.lean

```
/-- `𝓔(q) + λ_{q,t} d_t(q) = −ln(1−q²) − λ_{q,t} t q + λ_{q,t}`. -/
theorem energy_identity {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) :
    EE q + lam q t * dt t q = -log (1 - q ^ 2) - lam q t * t * q + lam q t
```

thm2_of_minprinciple Thm2Cert/Reduction.lean

```
/-- **Theorem 2 from the minimum principle.** For `m` pairs with `0 < q_j < t_j ≤ 1`,
`λ_j = λ_{q_j,t_j}`, `κ_j = λ_j t_j`, and `⟨T_j⟩ = λ_j(⟨D̂^{(j)}⟩ − 1)`, the minimum principle
`S + ∑ ⟨T_j⟩ ≥ ∑ [−ln(1−q_j²) − κ_j q_j]` implies
`S + ∑ λ_j [⟨D̂^{(j)}⟩ − d_{t_j}(q_j)] ≥ ∑ 𝓔(q_j)`. -/
theorem thm2_of_minprinciple {m : ℕ} (q t D : Fin m → ℝ) (Sent : ℝ)
    (hq : ∀ j, 0 < q j) (hqt : ∀ j, q j < t j) (ht : ∀ j, t j ≤ 1)
    (hmin : Sent + ∑ j, lam (q j) (t j) * (D j - 1)
      ≥ ∑ j, (-log (1 - q j ^ 2) - (lam (q j) (t j) * t j) * q j)) :
    Sent + ∑ j, lam (q j) (t j) * (D j - dt (t j) (q j)) ≥ ∑ j, EE (q j)
```

12 Verification record

```
Lean toolchain: leanprover/lean4:v4.35.0-rc4
Mathlib commit: 49fd9dee36e7bbf345fe7981d2de6aeeb591f75b (8 Oct 2026)

== Clean build (lake build) ==
⚠ [2293/2307] Built Thm2Cert.MatrixLemma (11s)
⚠ [2294/2307] Built Thm2Cert.Reduction (4.7s)
⚠ [2295/2307] Built Thm2Cert.Identity (4.8s)
⚠ [2296/2307] Built Thm1Cert.Defect (6.1s)
⚠ [2297/2307] Built Thm1Cert.Compression (6.6s)
⚠ [2298/2307] Built Thm1Cert.Closure (7.7s)
i [2299/2307] Built Thm2Cert.Comparison (20s)
⚠ [2300/2307] Built Thm1Cert.Assembly (5.6s)
i [2301/2307] Built Thm2Cert.Derivation (5.4s)
i [2302/2307] Built Thm2Cert (1.6s)
i [2304/2307] Built Thm1Cert.Upsilon (2.4s)
i [2305/2307] Built Thm1Cert.Corollaries (2.2s)
i [2306/2307] Built Thm1Cert (1.7s)
Build completed successfully (2307 jobs).
real        0m49.849s
```

```
(only remaining warnings: Mathlib's generic 'consider adding module' notices)

== Axiom audit (lake env lean Certificate.lean) ==
'Thm2Cert.lemmaS5_psd' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_pd' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_kernel' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_null' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_complex' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_complex_pd' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_complex_kernel' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.hyp_nonvacuous' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.gam_pos' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_gamma' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_gamma_complex' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.defects' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.Q1_from_euler' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.Q2_from_euler' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.mainid' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.mainid_general' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.rigidity' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.rigidity_multi' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.stationary_rigidity' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.calib_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lam_pos' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.hyp_of_lam' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.tmsv_value' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.energy_identity' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.thm2_of_minprinciple' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.drop_lemma' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.pointwise' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.defect_inequality_tsum' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.pava_exists' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.entropy_le_of_prefix' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.compression' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.H_thermal' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.F_thermal' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.support' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.Fmoments' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.brk_of_upsilon' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.thm1_spectrum' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.theorem1' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.theorem1_upsilon' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.theorem1_limit' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.entropy_truncation' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.theorem1_endpoint_state' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.fixed_entanglement' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.fixed_entanglement_endpoint' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.separability_criterion' depends on axioms: [propext, Classical.choice, Quot.sound]

No sorry / admit / axiom declarations in Thm1Cert/*.lean, Thm2Cert/*.lean (3455 lines).

== SHA-256 ==
661c65779ee328d771311577f005439b775219c2e58092bc6e42a6701420de2a  Thm1Cert.lean
86a3d914234d083870e9033a3f2a6502821ce7614638457b6cf3baf2f4914911  Thm2Cert.lean
d6b7f8c68a8a86d6d10280e2e02621fd3950e03176343add15840101e1e8226a  Certificate.lean
fc5f133ca7b63b949bc8d302ed1c3921dae7659df8e1339c4ccea75db15a3fdf  Thm1Cert/Assembly.lean
665bfd8f35f90cefe67362b7e6ba7d9ccdb7434b165a3b8d5b2970cc0b9e93e7  Thm1Cert/Closure.lean
379302da07b64d2b10525243350e322d3f2b5b8014a61e4d4ae72bb27f1e4f4c  Thm1Cert/Compression.lean
eabc7cd2ab81821133023c7695e61ba54c57229934704dc985d98a8b4132a22a  Thm1Cert/Corollaries.lean
d91b7f411dbd614c300cd0203bda7f10f0c18668bddfeaa495fccd10c63b4a70  Thm1Cert/Defect.lean
99d3f7ccb3b53d9584c08c49d812f127e9ff4599566567ce94f6211117db185f  Thm1Cert/Upsilon.lean
17cddd2c66982fdc705e5593c2f9d10719f5b1559ecbad9684e8cc5d64bde661  Thm2Cert/Comparison.lean
8c08ff7ada7a1b6188643d207fd00c94e4643c5b81f8d0910e2e9c07d71ede7f  Thm2Cert/Derivation.lean
dfdfc8f1d40ba675859a8e2a8ebc9507f366dd91ca778a47c7fc986b4bb9e520  Thm2Cert/Identity.lean
d444f89ef18c12792308cf0e2d40fcb15fca44b22992042c76c5d156453f62a9  Thm2Cert/MatrixLemma.lean
74cb74257d090f4f2c87d2ecaa22a74e24589ce711643345a28d13713e0978ef  Thm2Cert/Reduction.lean
64f165e6350db97d4cea9f0067ba844de19005330fd4ec06c3b87764e56b5538  lakefile.toml
8f89aa44fccdb1a0b6cc768c7c9f54b94cd40e47a08d582f82d1235f13075cf7  lean-toolchain
```

Appendix: full source

Thm1Cert/Defect.lean

```
/-
Certificate for Theorem S1 (singular-value defect inequality) of the Supplemental Material of
G. Adesso, "Optimality of Gaussian entanglement of formation" (arXiv:2608.01909v3):

    G_t(C) ≥ ∑_n (n+1) d_n²,   d_n = (t c_n − c_{n+1})₊,

for decreasing Schmidt coefficients c_n ≥ 0 and 0 < t ≤ 1, in the Schmidt-basis matrix model:
    G_t(C) = ∑_{i,j} |t c_i X_ij − c_j Y_ij|²            (Eq. S:Gmatrix),
with the index identities ∑_j |X_ij|² − ∑_j |X_ji|² = 1 (and the same for Y) for every
index with c_i > 0 (Eq. S:index).

Certified: the positive-drop lemma (S:drop), the pointwise bound (S:pointwise) in both cases
i < j and i > j, the telescoping (S:defectN), and the limit N → ∞, for decreasing spectra
indexed by ℕ (finite or infinite support). Inputs: the matrix model (S:Gmatrix), i.e.
X = U†aU, Y = V†aV in paired Schmidt bases indexed by ℕ, and the index identities (S:index),
which follow from [a, a†] = 1. Paired bases indexed by ℕ exist for every state of finite Schmidt
rank (both kernels are infinite dimensional) and for infinite rank when the two kernels have equal
dimension; the remaining states are reached by the truncation limit (`Corollaries.lean`).
-/
import Mathlib.Analysis.SpecialFunctions.Log.Basic
import Mathlib.Analysis.Complex.Norm
import Mathlib.Topology.Algebra.InfiniteSum.Order
import Mathlib.Topology.Algebra.InfiniteSum.Real
import Thm2Cert.MatrixLemma

open Finset

set_option autoImplicit false

noncomputable section

namespace Thm1Cert

/-- `ite_t` / `ite_f` without deprecation warnings. -/
theorem ite_t {α : Sort*} {p : Prop} [Decidable p] (h : p) {a b : α} :
    (if p then a else b) = a := by simp [h]

theorem ite_f {α : Sort*} {p : Prop} [Decidable p] (h : ¬ p) {a b : α} :
    (if p then a else b) = b := by simp [h]

/-- Positive part. -/
def pos (x : ℝ) : ℝ := max x 0

lemma pos_nonneg (x : ℝ) : 0 ≤ pos x := le_max_right _ _

lemma le_pos (x : ℝ) : x ≤ pos x := le_max_left _ _

lemma pos_mul_self (x : ℝ) : x * pos x = pos x ^ 2 := by
  unfold pos
  rcases le_total x 0 with h | h
  · rw [max_eq_right h]; ring
  · rw [max_eq_left h]; ring

variable (t : ℝ) (c : ℕ → ℝ)

/-- The positive drops `d_n = (t c_n − c_{n+1})₊`. -/
def dd (n : ℕ) : ℝ := pos (t * c n - c (n + 1))

/-! ## Positive-drop lemma (S:drop) -/

/-- `D_ij = ∑_{k=i}^{j-1} (t c_k − c_{k+1})₊ ≤ (t c_i − c_j)₊` for `i ≤ j`. -/
theorem drop_lemma {t : ℝ} {c : ℕ → ℝ} (ht0 : 0 ≤ t) (ht1 : t ≤ 1) (hc : Antitone c)
    (hc0 : ∀ n, 0 ≤ c n) {i j : ℕ} (hij : i ≤ j) :
    ∑ k ∈ Ico i j, dd t c k ≤ pos (t * c i - c j) := by
  induction j, hij using Nat.le_induction with
  | base =>
    simp only [Ico_self, sum_empty]; exact pos_nonneg _
  | succ j hij ih =>
    rw [sum_Ico_succ_top hij]
    have hcj : c j ≤ c i := hc hij
    have hcj1 : c (j + 1) ≤ c j := hc (Nat.le_succ j)
    have hc1 := hc0 (j + 1)
    unfold dd pos at *
    -- (t c_i − c_j)₊ + (t c_j − c_{j+1})₊ ≤ (t c_i − c_{j+1})₊
    have key : max (t * c i - c j) 0 + max (t * c j - c (j + 1)) 0
        ≤ max (t * c i - c (j + 1)) 0 := by
      have h1 : t * c j ≤ t * c i := mul_le_mul_of_nonneg_left hcj ht0
      have h2 : t * c j ≤ c j := by nlinarith [hc0 j]
      simp only [max_def]
      split_ifs <;> linarith
    linarith

/-! ## Pointwise bound (S:pointwise) -/

/-- Real form of the case `i < j`: for `0 ≤ D ≤ (A − B)₊`, `A, B ≥ 0`,
```

```
`(A a − B b)² − A D a² + B D b² ≥ 0`. -/
lemma real_case_lt {A B D : ℝ} (hA : 0 ≤ A) (hB : 0 ≤ B) (hD0 : 0 ≤ D) (hD : D ≤ pos (A - B))
    (a b : ℝ) : 0 ≤ (A * a - B * b) ^ 2 - A * D * a ^ 2 + B * D * b ^ 2 := by
  rcases hD0.eq_or_lt with h0 | hpos
  · subst h0; nlinarith [sq_nonneg (A * a - B * b)]
  · have hAB : D ≤ A - B := by
      unfold pos at hD
      rcases le_total (A - B) 0 with h | h
      · rw [max_eq_right h] at hD; linarith
      · rwa [max_eq_left h] at hD
    have e : (A * a - B * b) ^ 2 - A * D * a ^ 2 + B * D * b ^ 2
        = (A * (A - D)) * a ^ 2 - 2 * (A * B) * a * b + (B * (B + D)) * b ^ 2 := by ring
    rw [e]
    have := Thm2Cert.quad_nonneg (a := A * (A - D)) (c := B * (B + D)) (d := A * B)
      (mul_nonneg hA (by linarith)) (by nlinarith [mul_nonneg (mul_nonneg hA hB) hpos.le])
      (mul_nonneg hB (by linarith)) a b
    linarith

/-- Real form of the case `i > j`: for `0 ≤ D ≤ (t B − C)₊`, `A = t C`, `C ≤ B`, `t ≤ 1`,
`(A a − B b)² + A D a² − B D b² ≥ 0`. -/
lemma real_case_gt {t Cc B D : ℝ} (ht0 : 0 ≤ t) (ht1 : t ≤ 1) (hC : 0 ≤ Cc) (hCB : Cc ≤ B)
    (hD0 : 0 ≤ D) (hD : D ≤ pos (t * B - Cc)) (a b : ℝ) :
    0 ≤ (t * Cc * a - B * b) ^ 2 + t * Cc * D * a ^ 2 - B * D * b ^ 2 := by
  have hB : 0 ≤ B := hC.trans hCB
  rcases hD0.eq_or_lt with h0 | hpos
  · subst h0; nlinarith [sq_nonneg (t * Cc * a - B * b)]
  · have hDB : D ≤ t * B - Cc := by
      unfold pos at hD
      rcases le_total (t * B - Cc) 0 with h | h
      · rw [max_eq_right h] at hD; linarith
      · rwa [max_eq_left h] at hD
    have hA : 0 ≤ t * Cc := mul_nonneg ht0 hC
    have hBD : D ≤ B := by nlinarith
    have hgap : 0 ≤ B - t * Cc - D := by nlinarith
    have e : (t * Cc * a - B * b) ^ 2 + t * Cc * D * a ^ 2 - B * D * b ^ 2
        = (t * Cc * (t * Cc + D)) * a ^ 2 - 2 * (t * Cc * B) * a * b + (B * (B - D)) * b ^ 2 := by
      ring
    rw [e]
    have := Thm2Cert.quad_nonneg (a := t * Cc * (t * Cc + D)) (c := B * (B - D))
      (d := t * Cc * B) (mul_nonneg hA (by linarith))
      (by nlinarith [mul_nonneg (mul_nonneg (mul_nonneg hA hB) hpos.le) hgap])
      (mul_nonneg hB (by linarith)) a b
    linarith

lemma norm_sq_ge (A B : ℝ) (hA : 0 ≤ A) (hB : 0 ≤ B) (x y : ℂ) :
    (A * ‖x‖ - B * ‖y‖) ^ 2 ≤ ‖(A : ℂ) * x - (B : ℂ) * y‖ ^ 2 := by
  have h := abs_norm_sub_norm_le ((A : ℂ) * x) ((B : ℂ) * y)
  rw [norm_mul, norm_mul, Complex.norm_real, Complex.norm_real, Real.norm_of_nonneg hA,
    Real.norm_of_nonneg hB] at h
  have h0 : 0 ≤ ‖(A : ℂ) * x - (B : ℂ) * y‖ := norm_nonneg _
  have := sq_le_sq' (abs_le.mp h).1 (abs_le.mp h).2
  exact this

/-! ## Dual potentials -/

/-- Truncated positive drops: `δ_k = d_k` for `k ≤ N`, `0` otherwise. -/
def dlt (N k : ℕ) : ℝ := if k ≤ N then dd t c k else 0

/-- `α_i = ∑_{k ≥ i} t c_k δ_k`. -/
def alpha (N i : ℕ) : ℝ := ∑ k ∈ Ico i (N + 1), t * c k * dlt t c N k

/-- `β_i = −∑_{k ≥ i} c_{k+1} δ_k`. -/
def beta (N i : ℕ) : ℝ := -∑ k ∈ Ico i (N + 1), c (k + 1) * dlt t c N k

lemma sum_Ico_trunc (N : ℕ) (g : ℕ → ℝ) (hg : ∀ k, N < k → g k = 0) {i j : ℕ} (hij : i ≤ j) :
    ∑ k ∈ Ico i (N + 1), g k - ∑ k ∈ Ico j (N + 1), g k = ∑ k ∈ Ico i j, g k := by
  rcases le_total j (N + 1) with hj | hj
  · rw [← sum_Ico_consecutive g hij hj]; ring
  · have e1 : ∑ k ∈ Ico j (N + 1), g k = 0 := by rw [Ico_eq_empty_of_le hj, sum_empty]
    rw [e1, sub_zero]
    rcases le_total i (N + 1) with hi | hi
    · rw [← sum_Ico_consecutive g hi hj]
      have : ∑ k ∈ Ico (N + 1) j, g k = 0 :=
        sum_eq_zero (fun k hk => hg k (by simp at hk; omega))
      rw [this, add_zero]
    · rw [Ico_eq_empty_of_le hi, sum_empty]
      exact (sum_eq_zero (fun k hk => hg k (by simp at hk; omega))).symm

variable {t c}

lemma dd_nonneg (k : ℕ) : 0 ≤ dd t c k := pos_nonneg _

lemma dlt_nonneg (N k : ℕ) : 0 ≤ dlt t c N k := by
  unfold dlt; split_ifs
  · exact dd_nonneg k
  · exact le_rfl

lemma dlt_le (N k : ℕ) : dlt t c N k ≤ dd t c k := by
  unfold dlt; split_ifs
  · exact le_rfl
  · exact dd_nonneg k
```

```

/-- **Eq. (S:pointwise)**: `|t c_i x − c_j y|² ≥ (α_i − α_j)|x|² + (β_i − β_j)|y|²`. -/
theorem pointwise (ht0 : 0 ≤ t) (ht1 : t ≤ 1) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n)
    (N i j : ℕ) (x y : ℂ) :
    (alpha t c N i - alpha t c N j) * ‖x‖ ^ 2 + (beta t c N i - beta t c N j) * ‖y‖ ^ 2
      ≤ ‖((t * c i : ℝ) : ℂ) * x - (c j : ℂ) * y‖ ^ 2 := by
  have hn := norm_sq_ge (t * c i) (c j) (mul_nonneg ht0 (hc0 i)) (hc0 j) x y
  have hg1 : ∀ k, N < k → t * c k * dlt t c N k = 0 := by
    intro k hk; unfold dlt; simp only [show ¬ k ≤ N by omega, ↓reduceIte]; ring
  have hg2 : ∀ k, N < k → c (k + 1) * dlt t c N k = 0 := by
    intro k hk; unfold dlt; simp only [show ¬ k ≤ N by omega, ↓reduceIte]; ring
  rcases lt_trichotomy i j with hij | hij | hij
  · -- i < j
    have hA := sum_Ico_trunc N _ hg1 hij.le
    have hB := sum_Ico_trunc N _ hg2 hij.le
    set D := ∑ k ∈ Ico i j, dlt t c N k
    have hD0 : 0 ≤ D := sum_nonneg (fun k _ => dlt_nonneg N k)
    have hD : D ≤ pos (t * c i - c j) :=
      (sum_le_sum (fun k _ => dlt_le N k)).trans (drop_lemma ht0 ht1 hc hc0 hij.le)
    have hP : ∑ k ∈ Ico i j, t * c k * dlt t c N k ≤ t * c i * D := by
      rw [mul_sum]
      exact sum_le_sum (fun k hk => by
        simp only [mem_Ico] at hk
        exact mul_le_mul_of_nonneg_right
          (mul_le_mul_of_nonneg_left (hc hk.1) ht0) (dlt_nonneg N k))
    have hQ : c j * D ≤ ∑ k ∈ Ico i j, c (k + 1) * dlt t c N k := by
      rw [mul_sum]
      exact sum_le_sum (fun k hk => by
        simp only [mem_Ico] at hk
        exact mul_le_mul_of_nonneg_right (hc (by omega)) (dlt_nonneg N k))
    have hreal := real_case_lt (mul_nonneg ht0 (hc0 i)) (hc0 j) hD0 hD ‖x‖ ‖y‖
    unfold alpha beta
    rw [hA, show -∑ k ∈ Ico i (N + 1), c (k + 1) * dlt t c N k
        - -∑ k ∈ Ico j (N + 1), c (k + 1) * dlt t c N k
        = -(∑ k ∈ Ico i (N + 1), c (k + 1) * dlt t c N k
          - ∑ k ∈ Ico j (N + 1), c (k + 1) * dlt t c N k) by ring, hB]
    have i1 := mul_le_mul_of_nonneg_right hP (sq_nonneg ‖x‖)
    have i2 := mul_le_mul_of_nonneg_right hQ (sq_nonneg ‖y‖)
    nlinarith
  · subst hij
    unfold alpha beta
    simp only [sub_self, zero_mul, add_zero]
    exact sq_nonneg _
  · -- i > j
    have hA := sum_Ico_trunc N _ hg1 hij.le
    have hB := sum_Ico_trunc N _ hg2 hij.le
    set D := ∑ k ∈ Ico j i, dlt t c N k
    have hD0 : 0 ≤ D := sum_nonneg (fun k _ => dlt_nonneg N k)
    have hD : D ≤ pos (t * c j - c i) :=
      (sum_le_sum (fun k _ => dlt_le N k)).trans (drop_lemma ht0 ht1 hc hc0 hij.le)
    have hP : t * c i * D ≤ ∑ k ∈ Ico j i, t * c k * dlt t c N k := by
      rw [mul_sum]
      exact sum_le_sum (fun k hk => by
        simp only [mem_Ico] at hk
        exact mul_le_mul_of_nonneg_right
          (mul_le_mul_of_nonneg_left (hc hk.2.le) ht0) (dlt_nonneg N k))
    have hQ : ∑ k ∈ Ico j i, c (k + 1) * dlt t c N k ≤ c j * D := by
      rw [mul_sum]
      exact sum_le_sum (fun k hk => by
        simp only [mem_Ico] at hk
        exact mul_le_mul_of_nonneg_right (hc (by omega)) (dlt_nonneg N k))
    have hreal := real_case_gt ht0 ht1 (hc0 i) (hc hij.le) hD0 hD ‖x‖ ‖y‖
    unfold alpha beta
    have eA : ∑ k ∈ Ico i (N + 1), t * c k * dlt t c N k
        - ∑ k ∈ Ico j (N + 1), t * c k * dlt t c N k
        = -∑ k ∈ Ico j i, t * c k * dlt t c N k := by linarith
    have eB : -∑ k ∈ Ico i (N + 1), c (k + 1) * dlt t c N k
        - -∑ k ∈ Ico j (N + 1), c (k + 1) * dlt t c N k
        = ∑ k ∈ Ico j i, c (k + 1) * dlt t c N k := by linarith
    rw [eA, eB]
    have i1 := mul_le_mul_of_nonneg_right hP (sq_nonneg ‖x‖)
    have i2 := mul_le_mul_of_nonneg_right hQ (sq_nonneg ‖y‖)
    nlinarith

/-! ## Telescoping (S:defectN) -/

lemma sum_alpha_beta (N : ℕ) :
    ∑ i ∈ range (N + 1), (alpha t c N i + beta t c N i)
      = ∑ k ∈ range (N + 1), ((k : ℝ) + 1) * dd t c k ^ 2 := by
  have h1 : ∀ i, alpha t c N i + beta t c N i
      = ∑ k ∈ Ico i (N + 1), (t * c k - c (k + 1)) * dlt t c N k := by
    intro i; unfold alpha beta; rw [← sub_eq_add_neg, ← sum_sub_distrib]
    exact sum_congr rfl (fun k _ => by ring)
  simp only [h1]
  rw [sum_comm' (t' := range (N + 1)) (s' := fun k => range (k + 1))
    (h := fun i k => by simp only [mem_range, mem_Ico]; omega)]
  refine sum_congr rfl (fun k hk => ?_)
  simp only [mem_range] at hk
  rw [sum_const, card_range, nsmul_eq_mul]
  unfold dlt; simp only [show k ≤ N by omega, ↓reduceIte]
  unfold dd
```

```
  rw [pos_mul_self]
  push_cast; ring

/-! ## Theorem S1 -/

/-- **Theorem S1** (singular-value defect inequality), matrix model.
`hG`: `G = ∑_{i,j} |t c_i X_ij − c_j Y_ij|²`; `hX`, `hY`: the index identities (S:index),
required only for indices with `c_i > 0`. Conclusion: `∑_{n ≤ N} (n+1) d_n² ≤ G` for every `N`,
hence `∑_n (n+1) d_n²` converges and is `≤ G`. -/
theorem defect_inequality (ht0 : 0 < t) (ht1 : t ≤ 1) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n)
    (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) (N : ℕ) :
    ∑ k ∈ range (N + 1), ((k : ℝ) + 1) * dd t c k ^ 2 ≤ G := by
  rw [← sum_alpha_beta]
  -- potentials vanish where c_i = 0
  have hα0 : ∀ i, c i = 0 → alpha t c N i = 0 := by
    intro i hi; unfold alpha
    exact sum_eq_zero (fun k hk => by
      simp only [mem_Ico] at hk
      have : c k = 0 := le_antisymm (hi ▸ hc hk.1) (hc0 k)
      rw [this]; ring)
  have hβ0 : ∀ i, c i = 0 → beta t c N i = 0 := by
    intro i hi; unfold beta
    rw [neg_eq_zero]
    exact sum_eq_zero (fun k hk => by
      simp only [mem_Ico] at hk
      have : c (k + 1) = 0 := le_antisymm (hi ▸ hc (by omega)) (hc0 _)
      rw [this]; ring)
  have hαN : ∀ i, N < i → alpha t c N i = 0 := by
    intro i hi; unfold alpha; rw [Ico_eq_empty_of_le (by omega), sum_empty]
  have hβN : ∀ i, N < i → beta t c N i = 0 := by
    intro i hi; unfold beta; rw [Ico_eq_empty_of_le (by omega), sum_empty, neg_zero]
  -- lower bound on the K × K box
  set R : ℕ → ℝ := fun K => ∑ i ∈ range (N + 1),
    (alpha t c N i * (∑ j ∈ range K, ‖X i j‖ ^ 2 - ∑ j ∈ range K, ‖X j i‖ ^ 2)
      + beta t c N i * (∑ j ∈ range K, ‖Y i j‖ ^ 2 - ∑ j ∈ range K, ‖Y j i‖ ^ 2)) with hR
  have hbox : ∀ K, N + 1 ≤ K → R K ≤ G := by
    intro K hK
    have h1 : ∑ ij ∈ range K ×ˢ range K, ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
        - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2 ≤ G :=
      sum_le_hasSum _ (fun _ _ => sq_nonneg _) hG
    have h2 : ∑ ij ∈ range K ×ˢ range K,
        ((alpha t c N ij.1 - alpha t c N ij.2) * ‖X ij.1 ij.2‖ ^ 2
          + (beta t c N ij.1 - beta t c N ij.2) * ‖Y ij.1 ij.2‖ ^ 2)
        ≤ ∑ ij ∈ range K ×ˢ range K, ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
          - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2 :=
      sum_le_sum (fun ij _ => pointwise ht0.le ht1 hc hc0 N ij.1 ij.2 _ _)
    have h3 : ∑ ij ∈ range K ×ˢ range K,
        ((alpha t c N ij.1 - alpha t c N ij.2) * ‖X ij.1 ij.2‖ ^ 2
          + (beta t c N ij.1 - beta t c N ij.2) * ‖Y ij.1 ij.2‖ ^ 2) = R K := by
      rw [sum_product]
      have hsplit : ∀ (P : ℕ → ℝ) (Z : ℕ → ℕ → ℂ), (∀ i, N < i → P i = 0) →
          ∑ i ∈ range K, ∑ j ∈ range K, (P i - P j) * ‖Z i j‖ ^ 2
            = ∑ i ∈ range (N + 1),
              P i * (∑ j ∈ range K, ‖Z i j‖ ^ 2 - ∑ j ∈ range K, ‖Z j i‖ ^ 2) := by
        intro P Z hP
        have e : ∑ i ∈ range K, ∑ j ∈ range K, (P i - P j) * ‖Z i j‖ ^ 2
            = ∑ i ∈ range K, P i * (∑ j ∈ range K, ‖Z i j‖ ^ 2 - ∑ j ∈ range K, ‖Z j i‖ ^ 2) := by
          simp only [sub_mul, sum_sub_distrib, mul_sub, mul_sum]
          rw [sum_comm (s := range K) (t := range K)
            (f := fun i j => P j * ‖Z i j‖ ^ 2)]
        rw [e, ← sum_range_add_sum_Ico _ hK]
        have : ∑ i ∈ Ico (N + 1) K,
            P i * (∑ j ∈ range K, ‖Z i j‖ ^ 2 - ∑ j ∈ range K, ‖Z j i‖ ^ 2) = 0 :=
          sum_eq_zero (fun i hi => by simp at hi; rw [hP i (by omega), zero_mul])
        rw [this, add_zero]
      have e2 : ∑ i ∈ range K, ∑ j ∈ range K,
          ((alpha t c N i - alpha t c N j) * ‖X i j‖ ^ 2
            + (beta t c N i - beta t c N j) * ‖Y i j‖ ^ 2)
          = ∑ i ∈ range K, ∑ j ∈ range K, (alpha t c N i - alpha t c N j) * ‖X i j‖ ^ 2
            + ∑ i ∈ range K, ∑ j ∈ range K, (beta t c N i - beta t c N j) * ‖Y i j‖ ^ 2 := by
        rw [← sum_add_distrib]; exact sum_congr rfl (fun i _ => sum_add_distrib)
      rw [e2, hsplit _ X hαN, hsplit _ Y hβN, hR, ← sum_add_distrib]
    linarith
  -- limit K → ∞
  have hlim : Filter.Tendsto R Filter.atTop
      (nhds (∑ i ∈ range (N + 1), (alpha t c N i + beta t c N i))) := by
    rw [hR]
    have : ∑ i ∈ range (N + 1), (alpha t c N i + beta t c N i)
        = ∑ i ∈ range (N + 1), (alpha t c N i * (rX i - cX i)
          + beta t c N i * (rY i - cY i)) := by
      refine sum_congr rfl (fun i _ => ?_)
      rcases (hc0 i).eq_or_lt with h0 | hpos
      · rw [hα0 i h0.symm, hβ0 i h0.symm]; ring
```

```
      · rw [hX i hpos, hY i hpos]; ring
    rw [this]
    refine tendsto_finsetSum _ (fun i _ => ?_)
    rcases (hc0 i).eq_or_lt with h0 | hpos
    · simp only [hα0 i h0.symm, hβ0 i h0.symm, zero_mul, add_zero]
      exact tendsto_const_nhds
    · refine Filter.Tendsto.add (Filter.Tendsto.const_mul _ ?_) (Filter.Tendsto.const_mul _ ?_)
      · exact ((hrX i hpos).tendsto_sum_nat).sub ((hcX i hpos).tendsto_sum_nat)
      · exact ((hrY i hpos).tendsto_sum_nat).sub ((hcY i hpos).tendsto_sum_nat)
  exact le_of_tendsto hlim (Filter.eventually_atTop.mpr ⟨N + 1, hbox⟩)

/-- Consequence: the defect series converges and `∑_n (n+1) d_n² ≤ G_t(C)`. -/
theorem defect_inequality_tsum (ht0 : 0 < t) (ht1 : t ≤ 1) (hc : Antitone c)
    (hc0 : ∀ n, 0 ≤ c n) (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    Summable (fun n : ℕ => ((n : ℝ) + 1) * dd t c n ^ 2) ∧
    ∑' n : ℕ, ((n : ℝ) + 1) * dd t c n ^ 2 ≤ G := by
  have hnn : ∀ n : ℕ, 0 ≤ ((n : ℝ) + 1) * dd t c n ^ 2 := fun n => by positivity
  have hb : ∀ n, ∑ i ∈ range n, ((i : ℝ) + 1) * dd t c i ^ 2 ≤ G := by
    intro n
    rcases n with _ | n
    · simp only [range_zero, sum_empty]
      exact hG.nonneg (fun _ => sq_nonneg _)
    · exact defect_inequality ht0 ht1 hc hc0 X Y G hG rX cX rY cY hrX hcX hrY hcY hX hY n
  exact ⟨summable_of_sum_range_le hnn hb, Real.tsum_le_of_sum_range_le hnn hb⟩

end Thm1Cert
```

Thm1Cert/Compression.lean

```
/-
Certificate for Theorem S2 (geometric isotonic compression) of the Supplemental Material of
G. Adesso, "Optimality of Gaussian entanglement of formation" (arXiv:2608.01909v3).

For c_0 ≥ … ≥ c_M > 0 with ∑ c_n² = 1 (c_{M+1} = 0) and t > 0 there is a normalized
z_0, …, z_M ≥ 0 (z_{M+1} = 0) with
    z_{n+1} ≤ t z_n,     H(z²) ≤ H(c²),
    ∑ (n+1)(t z_n − z_{n+1})² ≤ ∑ (n+1)(t c_n − c_{n+1})₊².

The pool-adjacent-violators construction is formalized as an existence proof by strong induction
on the number of level breaks: pooling two adjacent maximal constant runs that violate
monotonicity preserves the total mass, can only raise every prefix sum, and does not increase
the drop energy (S:senergy); prefix domination then gives H(z²) ≤ H(c²) by a tangent-line /
Abel-summation form of Schur concavity.
-/
import Mathlib.Analysis.SpecialFunctions.Log.NegMulLog
import Mathlib.Analysis.SpecialFunctions.Sqrt
import Thm1Cert.Defect

open Finset Real

set_option autoImplicit false

noncomputable section

namespace Thm1Cert

variable (M : ℕ) (t : ℝ)

/-- Weights `w_n = t^{2n}`. -/
def wt (n : ℕ) : ℝ := (t ^ n) ^ 2

/-- Total weighted mass `∑_{n ≤ M} w_n ℓ_n`. -/
def mass (ℓ : ℕ → ℝ) : ℝ := ∑ n ∈ range (M + 1), wt t n * ℓ n

/-- Prefix mass `∑_{n ≤ K} w_n ℓ_n`. -/
def pre (ℓ : ℕ → ℝ) (K : ℕ) : ℝ := ∑ n ∈ range (K + 1), wt t n * ℓ n

/-- Drop energy (S:senergy): `∑_{n ≤ M} (n+1) t^{2n+2} ((√ℓ_n − √ℓ_{n+1})₊)²`. -/
def en (ℓ : ℕ → ℝ) : ℝ :=
  ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * (t ^ (n + 1)) ^ 2 * pos (√(ℓ n) - √(ℓ (n + 1))) ^ 2

/-- Number of level breaks on `[0, M]`. -/
def brk (ℓ : ℕ → ℝ) : ℕ := ((range M).filter (fun n => ℓ n ≠ ℓ (n + 1))).card

/-- Invariants of the pooling procedure, relative to the initial levels `r`. -/
structure Inv (r ℓ : ℕ → ℝ) : Prop where
  nonneg : ∀ n, 0 ≤ ℓ n
  supp : ∀ n, M < n → ℓ n = 0
  mass_eq : mass M t ℓ = mass M t r
  prefix_ge : ∀ K, K ≤ M → pre t r K ≤ pre t ℓ K
  en_le : en M t ℓ ≤ en M t r
```

```
53
54 variable {M t}
55
56 lemma wt_pos (ht : 0 < t) (n : ℕ) : 0 < wt t n := by unfold wt; positivity
57
58 lemma pos_mul_left {k : ℝ} (hk : 0 ≤ k) (u : ℝ) : pos (k * u) = k * pos u := by
59   unfold pos
60   rcases le_total u 0 with h | h
61   · rw [max_eq_right h, max_eq_right (mul_nonpos_of_nonneg_of_nonpos hk h), mul_zero]
62   · rw [max_eq_left h, max_eq_left (mul_nonneg hk h)]
63
64 /-- **Pooling step.** If two adjacent levels violate monotonicity, pooling the two maximal
65 constant runs around them preserves the invariants and strictly reduces the number of breaks. -/
66 theorem pool_step (ht : 0 < t) {r ℓ : ℕ → ℝ} (hI : Inv M t r ℓ) {n : ℕ} (hn : n < M)
67     (hv : ℓ n < ℓ (n + 1)) : ∃ ℓ', Inv M t r ℓ' ∧ brk M ℓ' < brk M ℓ := by
68   classical
69   -- left run [a, n]
70   have hPa : ∃ a, ∀ k, a ≤ k → k ≤ n → ℓ k = ℓ n :=
71     ⟨n, fun k h1 h2 => by rw [le_antisymm h2 h1]⟩
72   set a := Nat.find hPa with ha_def
73   have ha : ∀ k, a ≤ k → k ≤ n → ℓ k = ℓ n := Nat.find_spec hPa
74   have han : a ≤ n := Nat.find_min' hPa (fun k h1 h2 => by rw [le_antisymm h2 h1])
75   have hamin : 0 < a → ℓ (a - 1) ≠ ℓ n := by
76     intro hpos heq
77     apply Nat.find_min hPa (Nat.sub_lt hpos one_pos)
78     intro k h1 h2
79     rcases Nat.lt_or_ge k a with hk | hk
80     · rw [show k = a - 1 by omega]; exact heq
81     · exact ha k hk h2
82   -- right run [n+1, b]
83   set Pb : ℕ → Prop := fun b => n + 1 ≤ b ∧ ∀ k, n + 1 ≤ k → k ≤ b → ℓ k = ℓ (n + 1) with hPb
84   have hPb1 : Pb (n + 1) := ⟨le_rfl, fun k h1 h2 => by rw [le_antisymm h2 h1]⟩
85   set b := Nat.findGreatest Pb M with hb_def
86   have hb : Pb b := Nat.findGreatest_spec (m := n + 1) (by omega) hPb1
87   have hbM : b ≤ M := Nat.findGreatest_le M
88   have hbmax : b < M → ℓ (b + 1) ≠ ℓ (n + 1) := by
89     intro hlt heq
90     apply Nat.findGreatest_is_greatest (P := Pb) (k := b + 1) (Nat.lt_succ_self b) (show b + 1 ≤ M from hlt)
91     refine ⟨by omega, fun k h1 h2 => ?_⟩
92     rcases Nat.lt_or_ge b k with hk | hk
93     · rw [show k = b + 1 by omega]; exact heq
94     · exact hb.2 k h1 hk
95   have hnb : n + 1 ≤ b := hb.1
96   set x := ℓ n with hx
97   set y := ℓ (n + 1) with hy
98   have hval : ∀ k, a ≤ k → k ≤ b → x ≤ ℓ k ∧ ℓ k ≤ y := by
99     intro k h1 h2
100    rcases Nat.lt_or_ge n k with hk | hk
101    · rw [hb.2 k hk h2]; exact ⟨hv.le, le_rfl⟩
102    · rw [ha k h1 hk]; exact ⟨le_rfl, hv.le⟩
103  -- pooled level
104  set W := ∑ k ∈ Icc a b, wt t k with hW
105  have hWpos : 0 < W := sum_pos (fun k _ => wt_pos ht k) ⟨a, by simp; omega⟩
106  set m := (∑ k ∈ Icc a b, wt t k * ℓ k) / W with hm
107  have hxm : x ≤ m := by
108    rw [hm, le_div_iff₀ hWpos, hW, mul_sum]
109    exact sum_le_sum (fun k hk => by
110      simp only [mem_Icc] at hk
111      rw [mul_comm]; exact mul_le_mul_of_nonneg_left (hval k hk.1 hk.2).1 (wt_pos ht k).le)
112  have hmy : m ≤ y := by
113    rw [hm, div_le_iff₀ hWpos, hW, mul_sum]
114    exact sum_le_sum (fun k hk => by
115      simp only [mem_Icc] at hk
116      rw [mul_comm y]; exact mul_le_mul_of_nonneg_left (hval k hk.1 hk.2).2 (wt_pos ht k).le)
117  set ℓ' : ℕ → ℝ := fun k => if a ≤ k ∧ k ≤ b then m else ℓ k with hℓ'
118  set g : ℕ → ℝ := fun k => wt t k * (ℓ' k - ℓ k) with hg
119  have hgout : ∀ k, ¬ (a ≤ k ∧ k ≤ b) → g k = 0 := by
120    intro k hk; simp only [hg, hℓ', ite_f hk, sub_self, mul_zero]
121  have hgtot : ∑ k ∈ range (M + 1), g k = 0 := by
122    have hsub : Icc a b ⊆ range (M + 1) := by
123      intro k hk; simp only [mem_Icc] at hk; simp only [mem_range]; omega
124    rw [← sum_subset hsub (fun k _ hk => hgout k (by simpa using hk))]
125    have : ∑ k ∈ Icc a b, g k = m * W - ∑ k ∈ Icc a b, wt t k * ℓ k := by
126      rw [hW, mul_sum, ← sum_sub_distrib]
127      refine sum_congr rfl (fun k hk => ?_)
128      simp only [mem_Icc] at hk
129      simp only [hg, hℓ', ite_t hk]; ring
130    rw [this, hm, div_mul_cancel₀ _ hWpos.ne', sub_self]
131  have hgnn : ∀ k, k ≤ n → 0 ≤ g k := by
132    intro k hk
133    by_cases h : a ≤ k ∧ k ≤ b
134    · simp only [hg, hℓ', ite_t h, ha k h.1 hk]
135      exact mul_nonneg (wt_pos ht k).le (by linarith)
136    · rw [hgout k h]
137  have hgnp : ∀ k, n + 1 ≤ k → g k ≤ 0 := by
138    intro k hk
139    by_cases h : a ≤ k ∧ k ≤ b
140    · simp only [hg, hℓ', ite_t h, hb.2 k hk h.2]
141      exact mul_nonpos_of_nonneg_of_nonpos (wt_pos ht k).le (by linarith)
142    · rw [hgout k h]
143  have hpre_diff : ∀ K, pre t ℓ' K - pre t ℓ K = ∑ k ∈ range (K + 1), g k := by
```

```
    intro K; unfold pre; rw [← sum_sub_distrib]
    exact sum_congr rfl (fun k _ => by simp only [hg]; ring)
  refine ⟨ℓ', ⟨?_, ?_, ?_, ?_, ?_⟩, ?_⟩
  · -- nonneg
    intro k
    simp only [hℓ']
    split_ifs
    · exact (hI.nonneg n).trans hxm
    · exact hI.nonneg k
  · -- support
    intro k hk
    simp only [hℓ']
    rw [ite_f (by omega)]
    exact hI.supp k hk
  · -- mass
    have h1 := hpre_diff M
    have h2 := hI.mass_eq
    unfold pre at h1
    unfold mass at h2 ⊢
    linarith
  · -- prefix
    intro K hK
    have hdiff : 0 ≤ ∑ k ∈ range (K + 1), g k := by
      rcases Nat.lt_or_ge n K with hnK | hnK
      · have hsplit := sum_range_add_sum_Ico g (show K + 1 ≤ M + 1 by omega)
        have : ∑ k ∈ Ico (K + 1) (M + 1), g k ≤ 0 :=
          sum_nonpos (fun k hk => by simp at hk; exact hgnp k (by omega))
        linarith
      · exact sum_nonneg (fun k hk => by simp at hk; exact hgnn k (by omega))
    have := hpre_diff K
    linarith [hI.prefix_ge K hK]
  · -- energy
    refine le_trans (sum_le_sum (fun j _ => ?_)) hI.en_le
    apply mul_le_mul_of_nonneg_left _ (by positivity)
    apply pow_le_pow_left₀ (pos_nonneg _)
    by_cases hj : a ≤ j ∧ j ≤ b
    · by_cases hj1 : a ≤ j + 1 ∧ j + 1 ≤ b
      · simp only [hℓ', ite_t hj, ite_t hj1, sub_self]
        unfold pos; rw [max_self]; exact le_max_right _ _
      · have hjb : j = b := by omega
        have hyj : ℓ j = y := by rw [hjb]; exact hb.2 b hnb le_rfl
        simp only [hℓ', ite_t hj, ite_f hj1]
        have : √m ≤ √(ℓ j) := Real.sqrt_le_sqrt (by rw [hyj]; exact hmy)
        exact max_le_max (by linarith) le_rfl
    · by_cases hj1 : a ≤ j + 1 ∧ j + 1 ≤ b
      · have hja : j + 1 = a := by omega
        have hxa : ℓ (j + 1) = x := by rw [hja]; exact ha a le_rfl han
        simp only [hℓ', ite_f hj, ite_t hj1]
        have : √(ℓ (j + 1)) ≤ √m := Real.sqrt_le_sqrt (by rw [hxa]; exact hxm)
        exact max_le_max (by linarith) le_rfl
      · simp only [hℓ', ite_f hj, ite_f hj1]; exact le_rfl
  · -- breaks
    unfold brk
    have hnS : n ∈ (range M).filter (fun j => ℓ j ≠ ℓ (j + 1)) := by
      simp only [mem_filter, mem_range]; exact ⟨hn, hv.ne⟩
    have hsub : (range M).filter (fun j => ℓ' j ≠ ℓ' (j + 1))
        ⊆ ((range M).filter (fun j => ℓ j ≠ ℓ (j + 1))).erase n := by
      intro j hjS
      simp only [mem_filter, mem_range] at hjS
      obtain ⟨hjM, hne⟩ := hjS
      simp only [mem_erase, mem_filter, mem_range]
      by_cases hj : a ≤ j ∧ j ≤ b
      · by_cases hj1 : a ≤ j + 1 ∧ j + 1 ≤ b
        · simp only [hℓ', ite_t hj, ite_t hj1] at hne; exact absurd rfl hne
        · have hjb : j = b := by omega
          refine ⟨by omega, hjM, ?_⟩
          rw [hjb, hb.2 b hnb le_rfl]
          exact fun h => hbmax (by omega) h.symm
      · by_cases hj1 : a ≤ j + 1 ∧ j + 1 ≤ b
        · have hja : j + 1 = a := by omega
          refine ⟨by omega, hjM, ?_⟩
          rw [hja, ha a le_rfl han, show j = a - 1 by omega]
          exact hamin (by omega)
        · simp only [hℓ', ite_f hj, ite_f hj1] at hne
          exact ⟨fun h => hj ⟨by omega, by omega⟩, hjM, hne⟩
    calc _ ≤ (((range M).filter (fun j => ℓ j ≠ ℓ (j + 1))).erase n).card := card_le_card hsub
      _ < _ := card_erase_lt_of_mem hnS

/-- **Pool-adjacent-violators, existence form.** There are levels satisfying the invariants
that are nonincreasing on `[0, M]`. -/
theorem pava_exists (ht : 0 < t) {r : ℕ → ℝ} (hI : Inv M t r r) :
    ∃ ℓ, Inv M t r ℓ ∧ ∀ n, n < M → ℓ (n + 1) ≤ ℓ n := by
  suffices h : ∀ k ℓ, brk M ℓ = k → Inv M t r ℓ →
      ∃ ℓ', Inv M t r ℓ' ∧ ∀ n, n < M → ℓ' (n + 1) ≤ ℓ' n from h _ _ rfl hI
  intro k
  induction k using Nat.strong_induction_on with
  | _ k ih =>
    intro ℓ hk hIℓ
    by_cases hall : ∀ n, n < M → ℓ (n + 1) ≤ ℓ n
    · exact ⟨ℓ, hIℓ, hall⟩
    · push Not at hall
```

```
      obtain ⟨n, hn, hv⟩ := hall
      obtain ⟨ℓ', hI', hlt⟩ := pool_step ht hIℓ hn hv
      exact ih _ (hk ▸ hlt) ℓ' rfl hI'

/-! ## Schur concavity of the entropy under prefix domination -/

lemma tangent (p q : ℝ) (hp : 0 < p) (hq : 0 ≤ q) :
    negMulLog q ≤ negMulLog p + (-log p - 1) * (q - p) := by
  unfold negMulLog
  rcases hq.eq_or_lt with h0 | hqp
  · subst h0; simp; nlinarith
  · have h := Real.log_le_sub_one_of_pos (div_pos hp hqp)
    rw [Real.log_div hp.ne' hqp.ne'] at h
    have : q * (log p - log q) ≤ q * (p / q - 1) := mul_le_mul_of_nonneg_left h hqp.le
    have e : q * (p / q - 1) = p - q := by field_simp
    rw [e] at this
    nlinarith

/-- If `p` is positive and nonincreasing on `[0, M]` and every prefix sum of `q ≥ 0` dominates
that of `p`, with equal totals, then `H(q) ≤ H(p)`. -/
theorem entropy_le_of_prefix (p q : ℕ → ℝ) (hp : ∀ n, n ≤ M → 0 < p n)
    (hanti : ∀ n, n < M → p (n + 1) ≤ p n) (hq : ∀ n, 0 ≤ q n)
    (hpre : ∀ K, K ≤ M → ∑ n ∈ range (K + 1), p n ≤ ∑ n ∈ range (K + 1), q n)
    (htot : ∑ n ∈ range (M + 1), p n = ∑ n ∈ range (M + 1), q n) :
    ∑ n ∈ range (M + 1), negMulLog (q n) ≤ ∑ n ∈ range (M + 1), negMulLog (p n) := by
  set g : ℕ → ℝ := fun n => -log (p n) - 1 with hg
  have hgmono : ∀ n, n < M → g n ≤ g (n + 1) := by
    intro n hn
    simp only [hg]
    have := Real.log_le_log (hp (n + 1) (by omega)) (hanti n hn)
    linarith
  -- Abel summation: ∑_{n ≤ K} g_n Δ_n ≤ g_K S_K
  set S : ℕ → ℝ := fun K => ∑ n ∈ range (K + 1), (q n - p n) with hS
  have hSnn : ∀ K, K ≤ M → 0 ≤ S K := by
    intro K hK; simp only [hS, sum_sub_distrib]; linarith [hpre K hK]
  have habel : ∀ K, K ≤ M → ∑ n ∈ range (K + 1), g n * (q n - p n) ≤ g K * S K := by
    intro K
    induction K with
    | zero => intro _; simp [hS]
    | succ K ih =>
      intro hK
      rw [sum_range_succ, show S (K + 1) = S K + (q (K + 1) - p (K + 1)) by
        simp only [hS]; rw [sum_range_succ]]
      have h1 := ih (by omega)
      have h2 := mul_le_mul_of_nonneg_right (hgmono K (by omega)) (hSnn K (by omega))
      nlinarith
  have hSM : S M = 0 := by simp only [hS, sum_sub_distrib]; linarith
  have hlin : ∑ n ∈ range (M + 1), g n * (q n - p n) ≤ 0 := by
    have := habel M le_rfl; rw [hSM, mul_zero] at this; exact this
  have htan : ∑ n ∈ range (M + 1), negMulLog (q n)
      ≤ ∑ n ∈ range (M + 1), (negMulLog (p n) + g n * (q n - p n)) :=
    sum_le_sum (fun n hn => by
      simp only [mem_range] at hn
      exact tangent (p n) (q n) (hp n (by omega)) (hq n))
  rw [sum_add_distrib] at htan
  linarith

/-! ## Theorem S2 -/

/-- **Theorem S2** (geometric isotonic compression). -/
theorem compression (ht : 0 < t) (c : ℕ → ℝ) (hcpos : ∀ n, n ≤ M → 0 < c n)
    (hanti : ∀ n, n < M → c (n + 1) ≤ c n) (hsupp : ∀ n, M < n → c n = 0)
    (hnorm : ∑ n ∈ range (M + 1), c n ^ 2 = 1) :
    ∃ z : ℕ → ℝ, (∀ n, 0 ≤ z n) ∧ (∀ n, M < n → z n = 0) ∧
      ∑ n ∈ range (M + 1), z n ^ 2 = 1 ∧ (∀ n, z (n + 1) ≤ t * z n) ∧
      ∑ n ∈ range (M + 1), negMulLog (z n ^ 2) ≤ ∑ n ∈ range (M + 1), negMulLog (c n ^ 2) ∧
      ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2
        ≤ ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * pos (t * c n - c (n + 1)) ^ 2 := by
  classical
  set r : ℕ → ℝ := fun n => if n ≤ M then c n ^ 2 / wt t n else 0 with hr
  have hrnn : ∀ n, 0 ≤ r n := by
    intro n; simp only [hr]; split_ifs
    · exact div_nonneg (sq_nonneg _) (wt_pos ht n).le
    · exact le_rfl
  -- c_n = t^n √r_n for all n ≤ M + 1
  have hcr : ∀ n, n ≤ M + 1 → c n = t ^ n * √(r n) := by
    intro n hn
    rcases Nat.lt_or_ge M n with h | h
    · rw [hsupp n h]; simp only [hr, ite_f (by omega : ¬ n ≤ M), Real.sqrt_zero, mul_zero]
    · simp only [hr, ite_t h]
      rw [Real.sqrt_div (sq_nonneg _), Real.sqrt_sq (hcpos n h).le, wt,
        Real.sqrt_sq (by positivity)]
      field_simp
  have hwr : ∀ n, n ≤ M → wt t n * r n = c n ^ 2 := by
    intro n hn; simp only [hr, ite_t hn]; field_simp [(wt_pos ht n).ne']
  have hIr : Inv M t r r := ⟨hrnn, fun n hn => by simp only [hr, show ¬ n ≤ M by omega, ↓reduceIte],
    rfl, fun _ _ => le_rfl, le_rfl⟩
  obtain ⟨ℓ, hI, hmono⟩ := pava_exists ht hIr
  refine ⟨fun n => t ^ n * √(ℓ n), fun n => by positivity, fun n hn => by
    simp only [hI.supp n hn, Real.sqrt_zero, mul_zero], ?_, ?_, ?_, ?_⟩
  · -- normalization
```

```
    have : ∑ n ∈ range (M + 1), (t ^ n * √(ℓ n)) ^ 2 = mass M t ℓ := by
      unfold mass wt
      exact sum_congr rfl (fun n _ => by rw [mul_pow, Real.sq_sqrt (hI.nonneg n)])
    rw [this, hI.mass_eq]
    unfold mass
    rw [← hnorm]
    exact sum_congr rfl (fun n hn => hwr n (by simp at hn; omega))
  · -- ratio
    intro n
    show t ^ (n + 1) * √(ℓ (n + 1)) ≤ t * (t ^ n * √(ℓ n))
    rcases Nat.lt_or_ge n M with hn | hn
    · rw [pow_succ]
      have := Real.sqrt_le_sqrt (hmono n hn)
      have ht' : 0 ≤ t ^ n * t := by positivity
      calc t ^ n * t * √(ℓ (n + 1)) ≤ t ^ n * t * √(ℓ n) := mul_le_mul_of_nonneg_left this ht'
        _ = t * (t ^ n * √(ℓ n)) := by ring
    · simp only [hI.supp (n + 1) (by omega), Real.sqrt_zero, mul_zero]
      positivity
  · -- entropy
    have hq : ∀ n, (t ^ n * √(ℓ n)) ^ 2 = wt t n * ℓ n := by
      intro n; unfold wt; rw [mul_pow, Real.sq_sqrt (hI.nonneg n)]
    simp only [hq]
    apply entropy_le_of_prefix (fun n => c n ^ 2) (fun n => wt t n * ℓ n)
    · intro n hn; exact pow_pos (hcpos n hn) 2
    · intro n hn
      exact pow_le_pow_left₀ (hcpos (n + 1) (by omega)).le (hanti n hn) 2
    · intro n; exact mul_nonneg (wt_pos ht n).le (hI.nonneg n)
    · intro K hK
      have := hI.prefix_ge K hK
      unfold pre at this
      calc ∑ n ∈ range (K + 1), c n ^ 2 = ∑ n ∈ range (K + 1), wt t n * r n :=
            sum_congr rfl (fun n hn => (hwr n (by simp at hn; omega)).symm)
        _ ≤ _ := this
    · have := hI.mass_eq
      unfold mass at this
      rw [this]
      exact sum_congr rfl (fun n hn => (hwr n (by simp at hn; omega)).symm)
  · -- energy
    have hL : ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * (t * (t ^ n * √(ℓ n))
        - t ^ (n + 1) * √(ℓ (n + 1))) ^ 2 = en M t ℓ := by
      unfold en
      refine sum_congr rfl (fun n hn => ?_)
      simp only [mem_range] at hn
      have hd : 0 ≤ √(ℓ n) - √(ℓ (n + 1)) := by
        rcases Nat.lt_or_ge n M with h | h
        · have := Real.sqrt_le_sqrt (hmono n h); linarith
        · rw [hI.supp (n + 1) (by omega), Real.sqrt_zero, sub_zero]; positivity
      rw [show pos (√(ℓ n) - √(ℓ (n + 1))) = √(ℓ n) - √(ℓ (n + 1)) from max_eq_left hd]
      ring
    have hR : ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * pos (t * c n - c (n + 1)) ^ 2 = en M t r := by
      unfold en
      refine sum_congr rfl (fun n hn => ?_)
      simp only [mem_range] at hn
      rw [hcr n (by omega), hcr (n + 1) (by omega)]
      have : t * (t ^ n * √(r n)) - t ^ (n + 1) * √(r (n + 1))
          = t ^ (n + 1) * (√(r n) - √(r (n + 1))) := by ring
      rw [this]
      rw [pos_mul_left (by positivity : (0:ℝ) ≤ t ^ (n + 1))]
      ring
    rw [hL, hR]
    exact hI.en_le

end Thm1Cert
```

Thm1Cert/Closure.lean

```
/-
Certificate for the thermal log-Sobolev closure of the Supplemental Material of
G. Adesso, "Optimality of Gaussian entanglement of formation" (arXiv:2608.01909v3):

  * Eq. (S:thermalF): for z^{(s)}_n = √(1−s²) sⁿ,
        H((z^{(s)})²) = 𝓔(s),     𝓕_t(z^{(s)}) = (t−s)²/(1−s²);
  * Eq. (S:support): for 0 < q < t ≤ 1 and λ = λ_{q,t}, the function
        φ(s) = 𝓔(s) + λ (t−s)²/(1−s²)
    attains its minimum over 0 ≤ s < 1 at s = q (monotonicity of Λ_t, Eq. S:LambdaMon).

The meta log-Sobolev theorem of Beigi and Rahimi-Keshari is NOT formalized; it enters
`Assembly.lean` as an explicit hypothesis (`BRK`).
-/
import Mathlib.Analysis.SpecialFunctions.Log.NegMulLog
import Mathlib.Analysis.SpecificLimits.Normed
import Mathlib.Analysis.Calculus.Deriv.MeanValue
import Thm1Cert.Compression
import Thm2Cert.Reduction

open Finset Real

set_option autoImplicit false

```

```
noncomputable section

namespace Thm1Cert

open Thm2Cert (EE gfun lam dt EE_formula lam_pos)

/-- Entropy `H(z²) = ∑ −z_n² ln z_n²` of a square-root sequence. -/
def Hs (z : ℕ → ℝ) : ℝ := ∑' n, negMulLog (z n ^ 2)

/-- `𝓕_t(z) = ∑ (n+1)(t z_n − z_{n+1})²` (Eq. S:Fdef). -/
def Ft (t : ℝ) (z : ℕ → ℝ) : ℝ := ∑' n : ℕ, ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2

/-- Thermal square-root sequence `z^{(s)}_n = √(1−s²) sⁿ` (Eq. S:thermalz). -/
def zs (s : ℝ) (n : ℕ) : ℝ := √(1 - s ^ 2) * s ^ n

lemma zs_sq {s : ℝ} (hs1 : s < 1) (hs0 : 0 ≤ s) (n : ℕ) :
    zs s n ^ 2 = (1 - s ^ 2) * (s ^ 2) ^ n := by
  unfold zs
  rw [mul_pow, Real.sq_sqrt (by nlinarith), ← pow_mul, ← pow_mul, mul_comm 2 n]

lemma EE_zero : EE 0 = 0 := by
  unfold EE gfun; simp

/-- **Eq. (S:thermalF), entropy part**: `H((z^{(s)})²) = 𝓔(s)`. -/
theorem H_thermal {s : ℝ} (hs0 : 0 ≤ s) (hs1 : s < 1) : Hs (zs s) = EE s := by
  rcases hs0.eq_or_lt with h0 | hpos
  · subst h0
    rw [EE_zero]
    unfold Hs
    have : ∀ n, negMulLog (zs 0 n ^ 2) = 0 := by
      intro n
      rw [zs_sq (by norm_num) le_rfl]
      rcases n with _ | n
      · simp
      · simp
    simp only [this, tsum_zero]
  · have hx0 : 0 ≤ s ^ 2 := sq_nonneg s
    have hx1 : s ^ 2 < 1 := by nlinarith
    have ha : 0 < 1 - s ^ 2 := by linarith
    have hg := hasSum_geometric_of_lt_one hx0 hx1
    have hn := hasSum_coe_mul_geometric_of_norm_lt_one
      (show ‖s ^ 2‖ < 1 by rw [Real.norm_of_nonneg hx0]; exact hx1)
    have hsum := (hg.mul_left (-(1 - s ^ 2) * log (1 - s ^ 2))).add
      (hn.mul_left (-(1 - s ^ 2) * log (s ^ 2)))
    unfold Hs
    have hterm : ∀ n : ℕ, negMulLog (zs s n ^ 2)
        = -(1 - s ^ 2) * log (1 - s ^ 2) * (s ^ 2) ^ n
          + -(1 - s ^ 2) * log (s ^ 2) * ((n : ℝ) * (s ^ 2) ^ n) := by
      intro n
      rw [zs_sq hs1 hs0 n]
      unfold negMulLog
      rw [Real.log_mul ha.ne' (pow_ne_zero _ (by positivity)), Real.log_pow]
      ring
    simp only [hterm]
    rw [hsum.tsum_eq, EE_formula hpos hs1, Real.log_pow]
    field_simp
    push_cast
    ring

/-- **Eq. (S:thermalF), energy part**: `𝓕_t(z^{(s)}) = (t−s)²/(1−s²)`. -/
theorem F_thermal (t : ℝ) {s : ℝ} (hs0 : 0 ≤ s) (hs1 : s < 1) :
    Ft t (zs s) = (t - s) ^ 2 / (1 - s ^ 2) := by
  have hx0 : 0 ≤ s ^ 2 := sq_nonneg s
  have hx1 : s ^ 2 < 1 := by nlinarith
  have ha : 0 < 1 - s ^ 2 := by linarith
  have hg := hasSum_geometric_of_lt_one hx0 hx1
  have hn := hasSum_coe_mul_geometric_of_norm_lt_one
    (show ‖s ^ 2‖ < 1 by rw [Real.norm_of_nonneg hx0]; exact hx1)
  have hsum := (hn.add hg).mul_left ((1 - s ^ 2) * (t - s) ^ 2)
  unfold Ft
  have hterm : ∀ n : ℕ, ((n : ℝ) + 1) * (t * zs s n - zs s (n + 1)) ^ 2
      = (1 - s ^ 2) * (t - s) ^ 2 * ((n : ℝ) * (s ^ 2) ^ n + (s ^ 2) ^ n) := by
    intro n
    have e : t * zs s n - zs s (n + 1) = (t - s) * zs s n := by unfold zs; ring
    rw [e, mul_pow, zs_sq hs1 hs0 n]; ring
  simp only [hterm]
  rw [hsum.tsum_eq]
  field_simp
  ring

/-! ## The supporting-line inequality (S:support) -/

/-- `φ(s) = 𝓔(s) + λ (t−s)²/(1−s²)`. -/
def phi (L t s : ℝ) : ℝ := EE s + L * ((t - s) ^ 2 / (1 - s ^ 2))

/-- A continuous closed form of `φ` on `[0,1)`. -/
def psi (L t s : ℝ) : ℝ :=
  -log (1 - s ^ 2) - 2 * s * (s * log s) / (1 - s ^ 2) + L * ((t - s) ^ 2 / (1 - s ^ 2))

lemma phi_eq_psi (L t : ℝ) {s : ℝ} (hs0 : 0 ≤ s) (hs1 : s < 1) : phi L t s = psi L t s := by
  unfold phi psi
```

```
  rcases hs0.eq_or_lt with h0 | hpos
  · subst h0; rw [EE_zero]; simp
  · rw [EE_formula hpos hs1]; ring

lemma psi_continuousAt (L t : ℝ) {s : ℝ} (hs0 : 0 ≤ s) (hs1 : s < 1) :
    ContinuousAt (psi L t) s := by
  have hd : 1 - s ^ 2 ≠ 0 := by nlinarith
  unfold psi
  have c1 : ContinuousAt (fun s : ℝ => 1 - s ^ 2) s := by fun_prop
  have c2 : ContinuousAt (fun s : ℝ => s * log s) s := continuous_mul_log.continuousAt
  refine ((ContinuousAt.neg (c1.log hd)).sub ?_).add (continuousAt_const.mul ?_)
  · exact ((continuousAt_const.mul continuousAt_id).mul c2).div c1 hd
  · exact (((continuousAt_const.sub continuousAt_id).pow 2)).div c1 hd

/-- `g(s) = −2 s ln s − λ (t−s)(1−ts)`; `φ'(s) = 2 g(s)/(1−s²)²`. -/
def gsign (L t s : ℝ) : ℝ := -2 * s * log s - L * (t - s) * (1 - t * s)

lemma psi_hasDerivAt (L t : ℝ) {s : ℝ} (hs0 : 0 < s) (hs1 : s < 1) :
    HasDerivAt (psi L t) (2 * gsign L t s / (1 - s ^ 2) ^ 2) s := by
  have hd : 1 - s ^ 2 ≠ 0 := by nlinarith
  have d1 : HasDerivAt (fun s : ℝ => 1 - s ^ 2) (-(2 * s)) s := by
    simpa using (hasDerivAt_pow 2 s).const_sub 1
  have d2 : HasDerivAt (fun s : ℝ => s * log s) (log s + 1) s := hasDerivAt_mul_log hs0.ne'
  have d3 := ((hasDerivAt_id' s).const_mul (2:ℝ)).mul d2
  have d4 := ((hasDerivAt_id' s).const_sub t).pow 2
  have h := ((d1.log hd).neg.sub (d3.div d1 hd)).add ((d4.div d1 hd).const_mul L)
  unfold psi
  convert h using 1
  unfold gsign
  simp only [Pi.mul_apply, Pi.pow_apply]
  field_simp
  ring

/-- `k(s) = −s ln s/(1−s)` is strictly increasing on `(0,1)`. -/
lemma k_strictMono : StrictMonoOn (fun s : ℝ => -(s * log s) / (1 - s)) (Set.Ioo 0 1) := by
  apply strictMonoOn_of_deriv_pos (convex_Ioo 0 1)
  · intro s hs
    have : 1 - s ≠ 0 := by linarith [hs.2]
    exact (continuous_mul_log.continuousAt.neg.div (continuousAt_const.sub continuousAt_id)
      this).continuousWithinAt
  · intro s hs
    rw [interior_Ioo] at hs
    have h1 : 1 - s ≠ 0 := by linarith [hs.2]
    have hd := ((hasDerivAt_mul_log hs.1.ne').neg).div ((hasDerivAt_id' s).const_sub 1) h1
    rw [show deriv (fun s : ℝ => -(s * log s) / (1 - s)) s
        = (-(log s + 1) * (1 - s) - (-(s * log s)) * -1) / (1 - s) ^ 2 from hd.deriv]
    have hl := Real.log_lt_sub_one_of_pos hs.1 (by linarith [hs.2] : s ≠ 1)
    have : 0 < (1 - s) ^ 2 := by positivity
    apply div_pos _ this
    nlinarith

/-- `Λ_t(s) = −2 s ln s/((t−s)(1−ts))` is strictly increasing on `(0,t)` (Eq. S:LambdaMon). -/
lemma Lambda_strictMono {t : ℝ} (ht0 : 0 < t) (ht1 : t ≤ 1) :
    StrictMonoOn (fun s : ℝ => -2 * s * log s / ((t - s) * (1 - t * s))) (Set.Ioo 0 t) := by
  intro s1 hs1 s2 hs2 h12
  have hs1' : s1 ∈ Set.Ioo (0:ℝ) 1 := (hs1.1, by linarith [hs1.2])
  have hs2' : s2 ∈ Set.Ioo (0:ℝ) 1 := (hs2.1, by linarith [hs2.2])
  have hk := k_strictMono hs1' hs2' h12
  simp only at hk
  have hk1 : 0 < -(s1 * log s1) / (1 - s1) := by
    apply div_pos _ (by linarith [hs1'.2])
    have := Real.log_neg hs1'.1 hs1'.2
    nlinarith [hs1'.1]
  -- m(s) = (1−s)/((t−s)(1−ts)) is positive and nondecreasing
  have hm1 : 0 < (1 - s1) / ((t - s1) * (1 - t * s1)) := by
    apply div_pos (by linarith [hs1'.2]) (mul_pos (by linarith [hs1.2]) (by nlinarith [hs1'.2]))
  have hm : (1 - s1) / ((t - s1) * (1 - t * s1)) ≤ (1 - s2) / ((t - s2) * (1 - t * s2)) := by
    have a1 : 0 < t - s1 := by linarith [hs1.2]
    have a2 : 0 < t - s2 := by linarith [hs2.2]
    have b1 : 0 < 1 - t * s1 := by nlinarith [hs1'.2]
    have b2 : 0 < 1 - t * s2 := by nlinarith [hs2'.2]
    rw [div_le_div_iff₀ (mul_pos a1 b1) (mul_pos a2 b2)]
    have e1 : (1 - s1) * (t - s2) ≤ (1 - s2) * (t - s1) := by nlinarith
    have e2 : 1 - t * s2 ≤ 1 - t * s1 := by nlinarith
    have := mul_le_mul e1 e2 b2.le (by nlinarith [hs2'.2])
    nlinarith
  have f1 : -2 * s1 * log s1 / ((t - s1) * (1 - t * s1))
      = 2 * (-(s1 * log s1) / (1 - s1)) * ((1 - s1) / ((t - s1) * (1 - t * s1))) := by
    have : 1 - s1 ≠ 0 := by linarith [hs1'.2]
    field_simp
  have f2 : -2 * s2 * log s2 / ((t - s2) * (1 - t * s2))
      = 2 * (-(s2 * log s2) / (1 - s2)) * ((1 - s2) / ((t - s2) * (1 - t * s2))) := by
    have : 1 - s2 ≠ 0 := by linarith [hs2'.2]
    field_simp
  simp only
  rw [f1, f2]
  have := mul_lt_mul hk hm hm1 (by linarith)
  nlinarith

/-- Sign of `g`: negative on `(0,q)`, positive on `(q,1)`. -/
lemma gsign_cases {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) {s : ℝ} (hs0 : 0 < s)
```

```
    (hs1 : s < 1) :
    (s < q → gsign (lam q t) t s < 0) ∧ (q < s → 0 < gsign (lam q t) t s) := by
  have hL := lam_pos hq hqt ht
  have hΛq : -2 * q * log q / ((t - q) * (1 - t * q)) = lam q t := by
    unfold lam; ring_nf
  have hfac : ∀ s, 0 < s → s < t →
      gsign (lam q t) t s = (t - s) * (1 - t * s)
        * (-2 * s * log s / ((t - s) * (1 - t * s)) - lam q t) := by
    intro s h0 h1
    have e1 : t - s ≠ 0 := by linarith
    have e2 : 1 - t * s ≠ 0 := by nlinarith
    have e3 : 1 - s * t ≠ 0 := by rw [mul_comm]; exact e2
    unfold gsign; field_simp
  have ht0 : 0 < t := hq.trans hqt
  constructor
  · intro hsq
    have hst : s < t := hsq.trans hqt
    rw [hfac s hs0 hst]
    have := Lambda_strictMono ht0 ht ⟨hs0, hst⟩ ⟨hq, hqt⟩ hsq
    simp only at this
    rw [hΛq] at this
    exact mul_neg_of_pos_of_neg (mul_pos (by linarith) (by nlinarith)) (by linarith)
  · intro hqs
    rcases lt_or_ge s t with hst | hst
    · rw [hfac s hs0 hst]
      have := Lambda_strictMono ht0 ht ⟨hq, hqt⟩ ⟨hs0, hst⟩ hqs
      simp only at this
      rw [hΛq] at this
      exact mul_pos (mul_pos (by linarith) (by nlinarith)) (by linarith)
    · unfold gsign
      have hl : log s < 0 := Real.log_neg hs0 hs1
      have : 0 ≤ (s - t) * (1 - t * s) := mul_nonneg (by linarith) (by nlinarith)
      nlinarith [mul_pos hs0 (neg_pos.mpr hl)]

/-- **Eq. (S:support)**: for `0 < q < t ≤ 1` and `λ = λ_{q,t}`,
`𝓔(s) + λ(t−s)²/(1−s²) ≥ 𝓔(q) + λ(t−q)²/(1−q²)` for all `0 ≤ s < 1`. -/
theorem support {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) {s : ℝ} (hs0 : 0 ≤ s)
    (hs1 : s < 1) : phi (lam q t) t q ≤ phi (lam q t) t s := by
  have hq1 : q < 1 := lt_of_lt_of_le hqt ht
  rw [phi_eq_psi _ _ hq.le hq1, phi_eq_psi _ _ hs0 hs1]
  have hderiv : ∀ x, 0 < x → x < 1 →
      deriv (psi (lam q t) t) x = 2 * gsign (lam q t) t x / (1 - x ^ 2) ^ 2 :=
    fun x h0 h1 => (psi_hasDerivAt _ _ h0 h1).deriv
  have hdiff : ∀ x, 0 < x → x < 1 → DifferentiableAt ℝ (psi (lam q t) t) x :=
    fun x h0 h1 => (psi_hasDerivAt _ _ h0 h1).differentiableAt
  rcases le_total s q with hsq | hsq
  · -- antitone on [0, q]
    have hanti : AntitoneOn (psi (lam q t) t) (Set.Icc 0 q) := by
      apply antitoneOn_of_deriv_nonpos (convex_Icc 0 q)
      · intro x hx
        exact (psi_continuousAt _ _ hx.1 (lt_of_le_of_lt hx.2 hq1)).continuousWithinAt
      · intro x hx
        rw [interior_Icc] at hx
        exact (hdiff x hx.1 (hx.2.trans hq1)).differentiableWithinAt
      · intro x hx
        rw [interior_Icc] at hx
        rw [hderiv x hx.1 (hx.2.trans hq1)]
        have := (gsign_cases hq hqt ht hx.1 (hx.2.trans hq1)).1 hx.2
        have : 0 < (1 - x ^ 2) ^ 2 := by
          have : 0 < 1 - x ^ 2 := by nlinarith [hx.2.trans hq1, hx.1]
          positivity
        apply div_nonpos_of_nonpos_of_nonneg (by linarith) this.le
    exact hanti ⟨hs0, hsq⟩ ⟨hq.le, le_rfl⟩ hsq
  · -- monotone on [q, 1)
    have hmono : MonotoneOn (psi (lam q t) t) (Set.Ico q 1) := by
      apply monotoneOn_of_deriv_nonneg (convex_Ico q 1)
      · intro x hx
        exact (psi_continuousAt _ _ (hq.le.trans hx.1) hx.2).continuousWithinAt
      · intro x hx
        rw [interior_Ico] at hx
        exact (hdiff x (hq.trans hx.1) hx.2).differentiableWithinAt
      · intro x hx
        rw [interior_Ico] at hx
        rw [hderiv x (hq.trans hx.1) hx.2]
        have := (gsign_cases hq hqt ht (hq.trans hx.1) hx.2).2 hx.1
        apply div_nonneg (by linarith) (by positivity)
    exact hmono ⟨le_rfl, hq1⟩ ⟨hsq, hs1⟩ hsq

end Thm1Cert
```

Thm1Cert/Assembly.lean

```
/-
Assembly of Theorem 1 of G. Adesso, "Optimality of Gaussian entanglement of formation"
(arXiv:2608.01909v3), Supplemental Material Sec. "Completion of the proof":

  * `thm1_spectrum`: for every decreasing Schmidt spectrum c with ∑ c_n² = 1 and finite entropy,
        𝓔(q) + λ_{q,t}(t−q)²/(1−q²) ≤ H(c²) + λ_{q,t} ∑ (n+1) d_n²
    (compression on normalized truncations, the closure, and the limit M → ∞);
```

```
  * `theorem1`: with the defect inequality (Theorem S1),
        E(ψ) + λ_{q,t} [D_t(ψ) − d_t(q)] ≥ 𝓔(q),     D_t = G_t + 1 − t²   (Eq. S:Gdef),
    in the Schmidt-basis matrix model of `Defect.lean`; the identifications E(ψ) = H(c²)
    (Eq. S:entropy), D_t = G_t + 1 − t² and the matrix model itself are inputs;
  * `theorem1_endpoint`: the case q = t < 1 (Eq. S:endpoint);
  * `fixed_entanglement`: E(ψ) ≤ 𝓔(q) ⇒ D_t(ψ) ≥ d_t(q) (Eq. S:fixedE).

Input: `BRK t λ` − the Beigi–Rahimi-Keshari meta log-Sobolev theorem, restricted to diagonal
states of finite rank: H(z²) + λ 𝓕_t(z) ≥ inf_{0≤s<1} [H((z^{(s)})²) + λ 𝓕_t(z^{(s)})].
-/
import Mathlib.Algebra.BigOperators.Field
import Thm1Cert.Closure

open Finset Real Filter Topology

set_option autoImplicit false

noncomputable section

namespace Thm1Cert

open Thm2Cert (EE gfun lam dt EE_formula lam_pos)

/-- **Input (Beigi–Rahimi-Keshari, diagonal states of finite rank).** For every finitely
supported `z ≥ 0` with `∑ z_n² = 1`, `H(z²) + λ 𝓕_t(z)` is at least the infimum over thermal
states. (Y(ρ_z) = H(z²) + λ𝓕_t(z) + const for the parameters (S:parameters).) -/
def BRK (t L : ℝ) : Prop :=
  ∀ z : ℕ → ℝ, (∀ n, 0 ≤ z n) → (∃ M, ∀ n, M < n → z n = 0) → HasSum (fun n => z n ^ 2) 1 →
    ∀ c0 : ℝ, (∀ s, 0 ≤ s → s < 1 → c0 ≤ Hs (zs s) + L * Ft t (zs s)) → c0 ≤ Hs z + L * Ft t z

variable {q t : ℝ}

/-- Closure for a compressed, finitely supported sequence. -/
lemma closure_finite (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) (hB : BRK t (lam q t))
    (M : ℕ) (z : ℕ → ℝ) (hz0 : ∀ n, 0 ≤ z n) (hzs : ∀ n, M < n → z n = 0)
    (hzn : ∑ n ∈ range (M + 1), z n ^ 2 = 1) :
    phi (lam q t) t q ≤ ∑ n ∈ range (M + 1), negMulLog (z n ^ 2)
      + lam q t * ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2 := by
  have hout : ∀ n, n ∉ range (M + 1) → M < n := fun n hn => by simp at hn; omega
  have hH : Hs z = ∑ n ∈ range (M + 1), negMulLog (z n ^ 2) := by
    unfold Hs
    exact tsum_eq_sum (fun n hn => by rw [hzs n (hout n hn)]; simp)
  have hF : Ft t z = ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2 := by
    unfold Ft
    exact tsum_eq_sum (fun n hn => by
      rw [hzs n (hout n hn), hzs (n + 1) (by have := hout n hn; omega)]; ring)
  have hsum : HasSum (fun n => z n ^ 2) 1 := by
    rw [← hzn]
    exact hasSum_sum_of_ne_finset_zero (fun n hn => by rw [hzs n (hout n hn)]; ring)
  have := hB z hz0 ⟨M, hzs⟩ hsum (phi (lam q t) t q) (fun s hs0 hs1 => by
    rw [H_thermal hs0 hs1, F_thermal t hs0 hs1]
    exact support hq hqt ht hs0 hs1)
  rwa [hH, hF] at this

lemma negMulLog_div {x P : ℝ} (hx : 0 < x) (hP : 0 < P) :
    negMulLog (x / P) = negMulLog x / P + x / P * log P := by
  unfold negMulLog
  rw [Real.log_div hx.ne' hP.ne']
  field_simp
  ring

/-- **Truncation bound.** For `c_M > 0` and `P = ∑_{n ≤ M} c_n²`, the normalized truncation gives
`φ(q) ≤ S_M/P + ln P + λ (∑_{n<M}(n+1)d_n² + (M+1)t² c_M²)/P`. -/
lemma trunc_bound (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) (hB : BRK t (lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (_hc0 : ∀ n, 0 ≤ c n) (M : ℕ) (hcM : 0 < c M) :
    phi (lam q t) t q
      ≤ (∑ n ∈ range (M + 1), negMulLog (c n ^ 2)) / (∑ n ∈ range (M + 1), c n ^ 2)
        + log (∑ n ∈ range (M + 1), c n ^ 2)
        + lam q t * ((∑ n ∈ range M, ((n : ℝ) + 1) * dd t c n ^ 2
            + ((M : ℝ) + 1) * (t * c M) ^ 2) / (∑ n ∈ range (M + 1), c n ^ 2)) := by
  classical
  have ht0 : 0 < t := hq.trans hqt
  have hpos : ∀ n, n ≤ M → 0 < c n := fun n hn => lt_of_lt_of_le hcM (hc hn)
  set P := ∑ n ∈ range (M + 1), c n ^ 2 with hP
  have hP0 : 0 < P := sum_pos (fun n hn => pow_pos (hpos n (by simp at hn; omega)) 2)
    ⟨0, by simp⟩
  have hsP : 0 < √P := Real.sqrt_pos.mpr hP0
  set cM : ℕ → ℝ := fun n => if n ≤ M then c n / √P else 0 with hcM_def
  have hcMpos : ∀ n, n ≤ M → 0 < cM n := fun n hn => by
    simp only [hcM_def, ite_t hn]; exact div_pos (hpos n hn) hsP
  have hcManti : ∀ n, n < M → cM (n + 1) ≤ cM n := fun n hn => by
    simp only [hcM_def, ite_t (by omega : n + 1 ≤ M), ite_t hn.le]
    exact div_le_div_of_nonneg_right (hc (Nat.le_succ n)) hsP.le
  have hcMsupp : ∀ n, M < n → cM n = 0 := fun n hn => by
    simp only [hcM_def, show ¬ n ≤ M by omega, ↓reduceIte]
  have hsq : ∀ n, n ≤ M → cM n ^ 2 = c n ^ 2 / P := fun n hn => by
    simp only [hcM_def, ite_t hn]; rw [div_pow, Real.sq_sqrt hP0.le]
  have hcMnorm : ∑ n ∈ range (M + 1), cM n ^ 2 = 1 := by
    rw [sum_congr rfl (fun n hn => hsq n (by simp at hn; omega)), ← sum_div, div_self hP0.ne']
  obtain ⟨z, hz0, hzs, hzn, -, hzH, hzE⟩ :=
    compression ht0 cM hcMpos hcManti hcMsupp hcMnorm
```

```
  have hcl := closure_finite hq hqt ht hB M z hz0 hzs hzn
  have hL := (lam_pos hq hqt ht).le
  -- entropy of the truncation
  have hent : ∑ n ∈ range (M + 1), negMulLog (cM n ^ 2)
      = (∑ n ∈ range (M + 1), negMulLog (c n ^ 2)) / P + log P := by
    rw [sum_congr rfl (fun n hn => by
      rw [hsq n (by simp at hn; omega), negMulLog_div (pow_pos (hpos n (by simp at hn; omega)) 2) hP0])]
    rw [sum_add_distrib, ← sum_div, ← sum_mul, ← sum_div, ← hP, div_self hP0.ne', one_mul]
  -- energy of the truncation
  have hen : ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * pos (t * cM n - cM (n + 1)) ^ 2
      = (∑ n ∈ range M, ((n : ℝ) + 1) * dd t c n ^ 2 + ((M : ℝ) + 1) * (t * c M) ^ 2) / P := by
    rw [sum_range_succ, add_div, sum_div]
    congr 1
    · refine sum_congr rfl (fun n hn => ?_)
      simp only [mem_range] at hn
      simp only [hcM_def, ite_t hn.le, ite_t (by omega : n + 1 ≤ M)]
      rw [show t * (c n / √P) - c (n + 1) / √P = (1 / √P) * (t * c n - c (n + 1)) by
        field_simp]
      rw [pos_mul_left (by positivity), mul_pow, div_pow, one_pow, Real.sq_sqrt hP0.le]
      unfold dd; field_simp
    · simp only [hcM_def, ite_t le_rfl, show ¬ M + 1 ≤ M by omega, ↓reduceIte, sub_zero]
      have h0 : 0 ≤ t * (c M / √P) := by positivity
      rw [show pos (t * (c M / √P)) = t * (c M / √P) from max_eq_left h0, mul_pow, div_pow,
        Real.sq_sqrt hP0.le]
      field_simp
  calc phi (lam q t) t q ≤ _ := hcl
    _ ≤ ∑ n ∈ range (M + 1), negMulLog (cM n ^ 2)
        + lam q t * ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * pos (t * cM n - cM (n + 1)) ^ 2 :=
      add_le_add hzH (mul_le_mul_of_nonneg_left hzE hL)
    _ = _ := by rw [hent, hen]

/-- For decreasing summable `p ≥ 0`, `(M+1) p_M → 0`. -/
lemma tendsto_mul_antitone (p : ℕ → ℝ) (hp0 : ∀ n, 0 ≤ p n) (hanti : Antitone p) {A : ℝ}
    (hs : HasSum p A) : Tendsto (fun M : ℕ => ((M : ℝ) + 1) * p M) atTop (𝓝 0) := by
  set tail : ℕ → ℝ := fun k => A - ∑ n ∈ range k, p n with htail
  have htl : Tendsto tail atTop (𝓝 0) := by
    have := (tendsto_const_nhds (x := A)).sub hs.tendsto_sum_nat
    simpa [htail] using this
  have hdiv : Tendsto (fun M : ℕ => M / 2) atTop atTop :=
    tendsto_atTop_atTop.mpr (fun b => (2 * b, fun a ha => by omega))
  have hup : Tendsto (fun M : ℕ => 2 * tail (M / 2)) atTop (𝓝 0) := by
    simpa using (htl.comp hdiv).const_mul 2
  refine tendsto_of_tendsto_of_tendsto_of_le_of_le tendsto_const_nhds hup
    (fun M => mul_nonneg (by positivity) (hp0 M)) (fun M => ?_)
  simp only
  set k := M / 2
  have hk : k ≤ M + 1 := by omega
  have h1 : ((M : ℝ) + 1 - k) * p M ≤ ∑ n ∈ Ico k (M + 1), p n := by
    have : ∑ n ∈ Ico k (M + 1), p M ≤ ∑ n ∈ Ico k (M + 1), p n :=
      sum_le_sum (fun n hn => hanti (by simp at hn; omega))
    rw [sum_const, Nat.card_Ico, nsmul_eq_mul] at this
    rw [Nat.cast_sub hk] at this
    push_cast at this
    linarith
  have h2 : ∑ n ∈ range (M + 1), p n ≤ A := sum_le_hasSum _ (fun n _ => hp0 n) hs
  have h3 := sum_range_add_sum_Ico p hk
  have h4 : ((M : ℝ) + 1) ≤ 2 * ((M : ℝ) + 1 - k) := by
    have : 2 * k ≤ M := Nat.mul_div_le M 2
    have : (2 : ℝ) * k ≤ M := by exact_mod_cast this
    linarith
  have h5 := mul_le_mul_of_nonneg_right h4 (hp0 M)
  simp only [htail]
  nlinarith

/-- **Theorem 1, spectral form.** -/
theorem thm1_spectrum (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) (hB : BRK t (lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (hD : Summable (fun n : ℕ => ((n : ℝ) + 1) * dd t c n ^ 2)) :
    phi (lam q t) t q
      ≤ ∑' n, negMulLog (c n ^ 2) + lam q t * ∑' n : ℕ, ((n : ℝ) + 1) * dd t c n ^ 2 := by
  classical
  have hL := (lam_pos hq hqt ht).le
  by_cases hfin : ∃ R, c (R + 1) = 0
  · -- finite Schmidt rank
    set R := Nat.find hfin with hR
    have hR1 : c (R + 1) = 0 := Nat.find_spec hfin
    have hzero : ∀ n, R < n → c n = 0 := fun n hn =>
      le_antisymm (hR1 ▸ hc (by omega)) (hc0 n)
    have hcR : 0 < c R := by
      rcases (hc0 R).eq_or_lt with h | h
      · exfalso
        rcases Nat.eq_zero_or_pos R with h0 | h0
        · -- c 0 = 0 forces c ≡ 0, contradicting normalization
          have hall : ∀ n, c n = 0 := fun n =>
            le_antisymm ((hc (Nat.zero_le n)).trans (by rw [← h0]; exact h.symm.le)) (hc0 n)
          have : HasSum (fun n => c n ^ 2) 0 := by simp [hall]
          exact one_ne_zero (hnorm.unique this)
        · exact Nat.find_min hfin (show R - 1 < R by omega)
            (by rw [show R - 1 + 1 = R by omega]; exact h.symm)
      · exact h
```

```
    have hb := trunc_bound hq hqt ht hB c hc hc0 R hcR
    have hout : ∀ n, n ∉ range (R + 1) → R < n := fun n hn => by simp at hn; omega
    have hP : ∑ n ∈ range (R + 1), c n ^ 2 = 1 := by
      rw [← hnorm.tsum_eq]
      exact (tsum_eq_sum (fun n hn => by rw [hzero n (hout n hn)]; ring)).symm
    have hS : ∑' n, negMulLog (c n ^ 2) = ∑ n ∈ range (R + 1), negMulLog (c n ^ 2) :=
      tsum_eq_sum (fun n hn => by rw [hzero n (hout n hn)]; simp)
    have hDs : ∑' n : ℕ, ((n : ℝ) + 1) * dd t c n ^ 2
        = ∑ n ∈ range R, ((n : ℝ) + 1) * dd t c n ^ 2 + ((R : ℝ) + 1) * (t * c R) ^ 2 := by
      rw [tsum_eq_sum (s := range (R + 1)) (fun n hn => by
        unfold dd; rw [hzero n (hout n hn), hzero (n + 1) (by have := hout n hn; omega)]
        simp [pos]), sum_range_succ]
      unfold dd; rw [hR1, sub_zero, show pos (t * c R) = t * c R from
        max_eq_left (mul_nonneg (hq.trans hqt).le (hc0 R))]
    rw [hP, Real.log_one, div_one, div_one, add_zero] at hb
    rw [hS, hDs]; exact hb
  · -- infinite Schmidt rank: pass to the limit M → ∞
    simp only [not_exists] at hfin
    have hcpos : ∀ M, 0 < c M := fun M =>
      lt_of_lt_of_le ((hc0 (M + 1)).lt_of_ne (Ne.symm (hfin M))) (hc (Nat.le_succ M))
    have hP : Tendsto (fun M : ℕ => ∑ n ∈ range (M + 1), c n ^ 2) atTop (𝓝 1) :=
      (hnorm.tendsto_sum_nat).comp (tendsto_add_atTop_nat 1)
    have hS : Tendsto (fun M : ℕ => ∑ n ∈ range (M + 1), negMulLog (c n ^ 2)) atTop
        (𝓝 (∑' n, negMulLog (c n ^ 2))) :=
      (hH.hasSum.tendsto_sum_nat).comp (tendsto_add_atTop_nat 1)
    have hDt : Tendsto (fun M : ℕ => ∑ n ∈ range M, ((n : ℝ) + 1) * dd t c n ^ 2) atTop
        (𝓝 (∑' n : ℕ, ((n : ℝ) + 1) * dd t c n ^ 2)) := hD.hasSum.tendsto_sum_nat
    have hsqanti : Antitone (fun n => c n ^ 2) := fun a b hab =>
      pow_le_pow_left₀ (hc0 b) (hc hab) 2
    have hB0 : Tendsto (fun M : ℕ => ((M : ℝ) + 1) * (t * c M) ^ 2) atTop (𝓝 0) := by
      have := (tendsto_mul_antitone (fun n => c n ^ 2) (fun n => sq_nonneg _) hsqanti hnorm).const_mul
        (t ^ 2)
      simp only [mul_zero] at this
      refine this.congr (fun M => ?_)
      ring
    have hlog : Tendsto (fun M : ℕ => log (∑ n ∈ range (M + 1), c n ^ 2)) atTop (𝓝 0) := by
      rw [← Real.log_one]
      exact (Real.continuousAt_log one_ne_zero).tendsto.comp hP
    have hlim := ((hS.div hP one_ne_zero).add hlog).add
      (((hDt.add hB0).div hP one_ne_zero).const_mul (lam q t))
    simp only [div_one, add_zero] at hlim
    exact ge_of_tendsto hlim (Eventually.of_forall
      (fun M => trunc_bound hq hqt ht hB c hc hc0 M (hcpos M)))

lemma dt_identity (hq0 : 0 ≤ q) (hq1 : q < 1) : dt t q - 1 + t ^ 2 = (t - q) ^ 2 / (1 - q ^ 2) := by
  have : 1 - q ^ 2 ≠ 0 := by nlinarith [sq_nonneg q]
  unfold dt; field_simp; ring

/-- **Theorem 1** (affine extremality), matrix model. With `E(ψ) = H(c²)` (Eq. S:entropy) and
`D_t(ψ) = G + 1 − t²` (Eq. S:Gdef), where `G = ∑ |t c_i X_ij − c_j Y_ij|²` (Eq. S:Gmatrix):
`E(ψ) + λ_{q,t} [D_t(ψ) − d_t(q)] ≥ 𝓔(q)` for all `0 < q < t ≤ 1`. -/
theorem theorem1 (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) (hB : BRK t (lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    EE q ≤ ∑' n, negMulLog (c n ^ 2) + lam q t * ((G + 1 - t ^ 2) - dt t q) := by
  have ht0 : 0 < t := hq.trans hqt
  obtain ⟨hDs, hDle⟩ := defect_inequality_tsum ht0 ht hc hc0 X Y G hG rX cX rY cY
    hrX hcX hrY hcY hX hY
  have h := thm1_spectrum hq hqt ht hB c hc hc0 hnorm hH hDs
  have hL := (lam_pos hq hqt ht).le
  have hq1 : q < 1 := lt_of_lt_of_le hqt ht
  unfold phi at h
  rw [show (G + 1 - t ^ 2) - dt t q = G - (dt t q - 1 + t ^ 2) by ring, dt_identity hq.le hq1]
  have := mul_le_mul_of_nonneg_left hDle hL
  nlinarith

/-- **Eq. (S:fixedE)**: `E(ψ) ≤ 𝓔(q)` implies `D_t(ψ) ≥ d_t(q)`. -/
theorem fixed_entanglement (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) {E D : ℝ}
    (h1 : EE q ≤ E + lam q t * (D - dt t q)) (hE : E ≤ EE q) : dt t q ≤ D := by
  have hL := lam_pos hq hqt ht
  by_contra hc
  rw [not_le] at hc
  have : lam q t * (D - dt t q) < 0 := mul_neg_of_pos_of_neg hL (by linarith)
  linarith

lemma EE_continuousAt {s : ℝ} (hs0 : 0 < s) (hs1 : s < 1) : ContinuousAt EE s := by
  have hd : 1 - s ^ 2 ≠ 0 := by nlinarith
  have heq : (fun x => -log (1 - x ^ 2) - 2 * (x ^ 2 / (1 - x ^ 2)) * log x) =ᶠ[𝓝 s] EE := by
    have : Set.Ioo (0:ℝ) 1 ∈ 𝓝 s := Ioo_mem_nhds hs0 hs1
    filter_upwards [this] with x hx
    rw [EE_formula hx.1 hx.2]
  refine ContinuousAt.congr ?_ heq
```

```
  have c1 : ContinuousAt (fun x : ℝ => 1 - x ^ 2) s := by fun_prop
  exact (c1.log hd).neg.sub ((continuousAt_const.mul
    ((continuousAt_id.pow 2).div c1 hd)).mul (Real.continuousAt_log hs0.ne'))

/-- **Eq. (S:endpoint)**: for `t < 1`, if the affine bound holds for all `q < t` and
`D_t(ψ) = d_t(t) = 1 − t²`, then `E(ψ) ≥ 𝓔(t)`. -/
theorem theorem1_endpoint (ht0 : 0 < t) (ht1 : t < 1) {E : ℝ}
    (h : ∀ q, 0 < q → q < t → EE q ≤ E + lam q t * (dt t t - dt t q)) : EE t ≤ E := by
  have hle : ∀ q, 0 < q → q < t → EE q ≤ E := by
    intro q hq hqt
    have hL := (lam_pos hq hqt ht1.le).le
    have hq1 : q < 1 := hqt.trans ht1
    have e : dt t t - dt t q = -((t - q) ^ 2 / (1 - q ^ 2)) := by
      have a1 : 1 - q ^ 2 ≠ 0 := by nlinarith
      have a2 : 1 - t ^ 2 ≠ 0 := by nlinarith
      unfold dt; field_simp; ring
    have : 0 ≤ (t - q) ^ 2 / (1 - q ^ 2) := div_nonneg (sq_nonneg _) (by nlinarith)
    have := h q hq hqt
    rw [e] at this
    nlinarith
  have hc := (EE_continuousAt ht0 ht1).tendsto.mono_left (nhdsWithin_le_nhds (s := Set.Ioo 0 t))
  have hev : ∀ᶠ q in 𝓝[Set.Ioo 0 t] t, EE q ≤ E :=
    eventually_nhdsWithin_of_forall (fun q hq => hle q hq.1 hq.2)
  have : (𝓝[Set.Ioo 0 t] t).NeBot := by
    rw [← mem_closure_iff_nhdsWithin_neBot, closure_Ioo ht0.ne]
    exact ⟨ht0.le, le_rfl⟩
  exact le_of_tendsto hc hev

end Thm1Cert
```

Thm1Cert/Upsilon.lean

```
/-
The Beigi–Rahimi-Keshari input in the form used in the Supplemental Material (Eqs. S:Upsilon,
S:parameters, S:Fmoments, S:UpsilonF), for diagonal states ρ_z = ∑ z_n² |n⟩⟨n|:

    Y(ρ_z) = 2ν₀[(ñ+1) − T_z] + ω ñ + H(z²),   ν₀ = λt, ν₁ = 0, ω = λ(1−t)²,
    ñ = ∑ n z_n²,   T_z = ∑ (n+1) z_n z_{n+1}.

Certified: 𝓕_t(z) = t²(ñ+1) + ñ − 2t T_z (S:Fmoments) and Y(ρ_z) = H(z²) + λ𝓕_t(z) + λ(2t − t²)
(S:UpsilonF), both for finitely supported z and for the thermal sequences z^{(s)}; hence the
statement "inf Y is attained on thermal states" (`BRKUpsilon`, restricted to diagonal states of
finite rank) implies the sequence form `BRK` used in `Assembly.lean`.
The diagonal-state moment formulas Tr(ρ_z a†a) = ñ, Tr(√ρ_z a √ρ_z a†) = T_z are inputs.
-/
import Thm1Cert.Assembly

open Finset Real Filter Topology

set_option autoImplicit false

noncomputable section

namespace Thm1Cert

/-- Mean photon number `ñ = ∑ n z_n²`. -/
def nbar (z : ℕ → ℝ) : ℝ := ∑' n : ℕ, (n : ℝ) * z n ^ 2

/-- `T_z = ∑ (n+1) z_n z_{n+1} = Tr(√ρ_z a √ρ_z a†)` (Eq. S:Tz). -/
def Tz (z : ℕ → ℝ) : ℝ := ∑' n : ℕ, ((n : ℝ) + 1) * z n * z (n + 1)

/-- `Y(ρ_z)` for `ν₀ = λt`, `ν₁ = 0`, `ω = λ(1−t)²` (Eqs. S:Upsilon, S:parameters). -/
def Ups (t L : ℝ) (z : ℕ → ℝ) : ℝ :=
  2 * (L * t) * ((nbar z + 1) - Tz z) + L * (1 - t) ^ 2 * nbar z + Hs z

/-- **Input (Beigi–Rahimi-Keshari)**, restricted to diagonal states of finite rank:
`Y(ρ_z) ≥ inf_{0 ≤ s < 1} Y(τ_s)`. -/
def BRKUpsilon (t L : ℝ) : Prop :=
  ∀ z : ℕ → ℝ, (∀ n, 0 ≤ z n) → (∃ M, ∀ n, M < n → z n = 0) → HasSum (fun n => z n ^ 2) 1 →
    ∀ c0 : ℝ, (∀ s, 0 ≤ s → s < 1 → c0 ≤ Ups t L (zs s)) → c0 ≤ Ups t L z

/-- **Eq. (S:Fmoments)** in `HasSum` form. -/
lemma Fmoments (t : ℝ) (z : ℕ → ℝ) {N T : ℝ} (h1 : HasSum (fun n => z n ^ 2) 1)
    (hN : HasSum (fun n : ℕ => (n : ℝ) * z n ^ 2) N)
    (hT : HasSum (fun n : ℕ => ((n : ℝ) + 1) * z n * z (n + 1)) T) :
    HasSum (fun n : ℕ => ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2)
      (t ^ 2 * (N + 1) + N - 2 * t * T) := by
  have hA : HasSum (fun n : ℕ => ((n : ℝ) + 1) * z n ^ 2) (N + 1) := by
    have := hN.add h1; simpa [add_mul] using this
  have hB : HasSum (fun n : ℕ => ((n : ℝ) + 1) * z (n + 1) ^ 2) N := by
    have := (hasSum_nat_add_iff' 1).mpr hN
    simp only [range_one, sum_singleton, Nat.cast_zero, zero_mul, sub_zero] at this
    simpa using this
  have := ((hA.mul_left (t ^ 2)).add hB).sub (hT.mul_left (2 * t))
  convert this using 1
  funext n; ring

/-- **Eq. (S:UpsilonF)**: `Y(ρ_z) = H(z²) + λ𝓕_t(z) + λ(2t − t²)`. -/
lemma Ups_eq (t L : ℝ) (z : ℕ → ℝ) (h1 : HasSum (fun n => z n ^ 2) 1)
```

```
    (hN : Summable (fun n : ℕ => (n : ℝ) * z n ^ 2))
    (hT : Summable (fun n : ℕ => ((n : ℝ) + 1) * z n * z (n + 1))) :
    Ups t L z = Hs z + L * Ft t z + L * (2 * t - t ^ 2) := by
  have hF := Fmoments t z h1 hN.hasSum hT.hasSum
  unfold Ups Ft nbar Tz
  rw [hF.tsum_eq]
  ring

lemma summable_of_finite_support (f : ℕ → ℝ) (M : ℕ) (hf : ∀ n, M < n → f n = 0) :
    Summable f :=
  summable_of_ne_finset_zero (s := range (M + 1)) (fun n hn => hf n (by simp at hn; omega))

lemma zs_summable {s : ℝ} (hs0 : 0 ≤ s) (hs1 : s < 1) :
    HasSum (fun n => zs s n ^ 2) 1 ∧ Summable (fun n : ℕ => (n : ℝ) * zs s n ^ 2) ∧
      Summable (fun n : ℕ => ((n : ℝ) + 1) * zs s n * zs s (n + 1)) := by
  have hx0 : 0 ≤ s ^ 2 := sq_nonneg s
  have hx1 : s ^ 2 < 1 := by nlinarith
  have ha : 0 < 1 - s ^ 2 := by linarith
  have hg := hasSum_geometric_of_lt_one hx0 hx1
  have hn := hasSum_coe_mul_geometric_of_norm_lt_one
    (show ‖s ^ 2‖ < 1 by rw [Real.norm_of_nonneg hx0]; exact hx1)
  refine ⟨?_, ?_, ?_⟩
  · have := hg.mul_left (1 - s ^ 2)
    rw [mul_inv_cancel₀ ha.ne'] at this
    simpa [zs_sq hs1 hs0] using this
  · have := (hn.mul_left (1 - s ^ 2)).summable
    refine this.congr (fun n => ?_)
    rw [zs_sq hs1 hs0]; ring
  · have := ((hn.add hg).mul_left ((1 - s ^ 2) * s)).summable
    refine this.congr (fun n => ?_)
    unfold zs
    have hsq : √(1 - s ^ 2) ^ 2 = 1 - s ^ 2 := Real.sq_sqrt ha.le
    linear_combination (-((n : ℝ) + 1) * s ^ n * s ^ (n + 1)) * hsq

/-- The Beigi–Rahimi-Keshari input in Y form implies the sequence form `BRK`. -/
theorem brk_of_upsilon {t L : ℝ} (hU : BRKUpsilon t L) : BRK t L := by
  intro z hz0 ⟨M, hM⟩ h1 c0 hc0
  have hNz := summable_of_finite_support (fun n : ℕ => (n : ℝ) * z n ^ 2) M
    (fun n hn => by simp [hM n hn])
  have hTz := summable_of_finite_support (fun n : ℕ => ((n : ℝ) + 1) * z n * z (n + 1)) M
    (fun n hn => by simp [hM n hn])
  have key := hU z hz0 ⟨M, hM⟩ h1 (c0 + L * (2 * t - t ^ 2)) (fun s hs0 hs1 => by
    obtain ⟨a, b, c⟩ := zs_summable hs0 hs1
    rw [Ups_eq t L (zs s) a b c]
    linarith [hc0 s hs0 hs1])
  rw [Ups_eq t L z h1 hNz hTz] at key
  linarith

/-- **Theorem 1** with the Beigi–Rahimi-Keshari input in its Y form. -/
theorem theorem1_upsilon {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1)
    (hU : BRKUpsilon t (Thm2Cert.lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    Thm2Cert.EE q ≤ ∑' n, negMulLog (c n ^ 2)
      + Thm2Cert.lam q t * ((G + 1 - t ^ 2) - Thm2Cert.dt t q) :=
  theorem1 hq hqt ht (brk_of_upsilon hU) c hc hc0 hnorm hH X Y G hG rX cX rY cY
    hrX hcX hrY hcY hX hY

/-- **Endpoint `q = t < 1`** (Eq. S:endpoint): a state with `D_t(ψ) = d_t(t) = 1 − t²`
(i.e. `G = 0`) has `E(ψ) ≥ 𝓛(t)`. -/
theorem theorem1_endpoint_state {t : ℝ} (ht0 : 0 < t) (ht1 : t < 1)
    (hU : ∀ q, 0 < q → q < t → BRKUpsilon t (Thm2Cert.lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (X Y : ℕ → ℕ → ℂ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) 0)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    Thm2Cert.EE t ≤ ∑' n, negMulLog (c n ^ 2) := by
  apply theorem1_endpoint ht0 ht1
  intro q hq hqt
  have h := theorem1_upsilon hq hqt ht1.le (hU q hq hqt) c hc hc0 hnorm hH X Y 0 hG
    rX cX rY cY hrX hcX hrY hcY hX hY
  have hdt : Thm2Cert.dt t t = 1 - t ^ 2 := by
    have : 1 - t ^ 2 ≠ 0 := by nlinarith
    unfold Thm2Cert.dt; field_simp
  rw [hdt]
```

```
  simpa using h

end Thm1Cert
```

Thm1Cert/Corollaries.lean

```
/-
Corollaries and the general-state limit for Theorem 1 of G. Adesso, "Optimality of Gaussian
entanglement of formation" (arXiv:2608.01909v3).

  * `theorem1_limit`: the affine bound passes to limits E_L → E, D_L → D. With the normalized
    Schmidt truncations C^{(L)} (finite rank, covered by `theorem1_upsilon` in the paired-basis
    matrix model), this gives Theorem 1 for every finite-energy state, the convergence
    G_t(C^{(L)}) → G_t(C), E(ψ_{C^{(L)}}) → E(ψ_C) being the Fock-space input of the SM
    (proof of Theorem S1, last paragraph);
  * `entropy_truncation`: E(ψ_{C^{(L)}}) → E(ψ_C) for the normalized truncations, at the level of
    Schmidt spectra (finite entropy);
  * `fixed_entanglement_endpoint` (q = t) and `separability_criterion` (q = 0, i.e. E(ψ) = 0
    implies D_t(ψ) ≥ 1), completing Eq. (S:fixedE) for 0 ≤ q ≤ t.
-/
import Thm1Cert.Upsilon

open Finset Real Filter Topology

set_option autoImplicit false

noncomputable section

namespace Thm1Cert

open Thm2Cert (EE gfun lam dt EE_formula lam_pos)

/-- **Limit step.** If the affine bound holds along a sequence of states with `E_L → E` and
`D_L → D`, it holds in the limit. -/
theorem theorem1_limit {q t : ℝ} (E D : ℝ) (EL DL : ℕ → ℝ)
    (h : ∀ L, EE q ≤ EL L + lam q t * (DL L - dt t q))
    (hE : Tendsto EL atTop (𝓝 E)) (hD : Tendsto DL atTop (𝓝 D)) :
    EE q ≤ E + lam q t * (D - dt t q) := by
  have hlim := hE.add ((hD.sub (tendsto_const_nhds (x := dt t q))).const_mul (lam q t))
  exact ge_of_tendsto hlim (Eventually.of_forall h)

/-- Entropy of the normalized Schmidt truncations converges: with `P_M = ∑_{n ≤ M} c_n²`,
`H((c^{(M)})²) = S_M/P_M + ln P_M → H(c²)`. -/
theorem entropy_truncation (c : ℕ → ℝ) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2))) :
    Tendsto (fun M : ℕ => (∑ n ∈ range (M + 1), negMulLog (c n ^ 2)) / (∑ n ∈ range (M + 1), c n ^ 2)
      + log (∑ n ∈ range (M + 1), c n ^ 2)) atTop (𝓝 (∑' n, negMulLog (c n ^ 2))) := by
  have hP : Tendsto (fun M : ℕ => ∑ n ∈ range (M + 1), c n ^ 2) atTop (𝓝 1) :=
    (hnorm.tendsto_sum_nat).comp (tendsto_add_atTop_nat 1)
  have hS : Tendsto (fun M : ℕ => ∑ n ∈ range (M + 1), negMulLog (c n ^ 2)) atTop
      (𝓝 (∑' n, negMulLog (c n ^ 2))) :=
    (hH.hasSum.tendsto_sum_nat).comp (tendsto_add_atTop_nat 1)
  have hlog : Tendsto (fun M : ℕ => log (∑ n ∈ range (M + 1), c n ^ 2)) atTop (𝓝 0) := by
    rw [← Real.log_one]
    exact (Real.continuousAt_log one_ne_zero).tendsto.comp hP
  have := (hS.div hP one_ne_zero).add hlog
  simpa using this

/-- **Eq. (S:fixedE) at `q = t`**: `D_t(ψ) = G + 1 − t² ≥ 1 − t² = d_t(t)` since `G ≥ 0`. -/
theorem fixed_entanglement_endpoint {t G : ℝ} (ht0 : 0 < t) (ht1 : t < 1) (hG : 0 ≤ G) :
    dt t t ≤ G + 1 - t ^ 2 := by
  have : 1 - t ^ 2 ≠ 0 := by nlinarith
  have hd : dt t t = 1 - t ^ 2 := by unfold dt; field_simp
  rw [hd]; linarith

lemma EE_nonneg {q : ℝ} (hq : 0 < q) (hq1 : q < 1) : 0 ≤ EE q := by
  unfold EE gfun
  set y := q ^ 2 / (1 - q ^ 2) with hy
  have hy0 : 0 < y := div_pos (by positivity) (by nlinarith)
  have h1 : log y ≤ log (y + 1) := Real.log_le_log hy0 (by linarith)
  have h2 : 0 ≤ log (y + 1) := Real.log_nonneg (by linarith)
  nlinarith

/-- **Separability criterion (`q = 0` in Eq. S:fixedE)**: if the affine bound holds for all
`0 < q < t` and `E(ψ) ≤ 0` (a product state), then `D_t(ψ) ≥ 1 = d_t(0)`. -/
theorem separability_criterion {t E D : ℝ} (ht0 : 0 < t) (ht : t ≤ 1)
    (h : ∀ q, 0 < q → q < t → EE q ≤ E + lam q t * (D - dt t q)) (hE : E ≤ 0) : 1 ≤ D := by
  have hle : ∀ q, 0 < q → q < t → dt t q ≤ D := by
    intro q hq hqt
    have hL := lam_pos hq hqt ht
    have h0 := EE_nonneg hq (lt_of_lt_of_le hqt ht)
    have := h q hq hqt
    by_contra hc
    rw [not_le] at hc
    have : lam q t * (D - dt t q) < 0 := mul_neg_of_pos_of_neg hL (by linarith)
    linarith
  have hcont : ContinuousAt (fun q => dt t q) 0 := by
    unfold dt
    exact ((continuousAt_const.sub (continuousAt_const.mul continuousAt_id)).pow 2).div
```

```
      (continuousAt_const.sub (continuousAt_id.pow 2)) (by norm_num)
  have hc := hcont.tendsto.mono_left (nhdsWithin_le_nhds (s := Set.Ioo 0 t))
  have hev : ∀ᶠ q in 𝓝[Set.Ioo 0 t] (0 : ℝ), dt t q ≤ D :=
    eventually_nhdsWithin_of_forall (fun q hq => hle q hq.1 hq.2)
  have : (𝓝[Set.Ioo 0 t] (0 : ℝ)).NeBot := by
    rw [← mem_closure_iff_nhdsWithin_neBot, closure_Ioo ht0.ne]
    exact ⟨le_rfl, ht0.le⟩
  have h1 := le_of_tendsto hc hev
  have hd0 : dt t 0 = 1 := by unfold dt; simp
  rwa [hd0] at h1

end Thm1Cert
```

Thm2Cert/MatrixLemma.lean

```
/-
Copyright (c) 2026. Machine-checked certificate for
G. Adesso, "Optimality of Gaussian entanglement of formation", Supplemental Material,
Lemma S5 (the matrix inequality behind Theorem 2).

Statement certified here (all real numbers):
  Let κ, α, β, w > 0 with κ² ≤ αβ and the calibration κ cosh w − w = (α+β)/2,
  and q = e^{−w}.  With f(x) = x / sinh x (f(0) = 1) and b(x) = x coth x = f(x) cosh x, put
    M₁₁(x) = q(κ − qα)eˣ + (α − qκ)e⁻ˣ + (1+q²) f(x)
    M₂₂(x) = q(κ − qβ)eˣ + (β − qκ)e⁻ˣ + (1+q²) f(x)
    M₁₂(x) = −κ(1 − q²) − 2q b(x).
  Then M(x) is positive semidefinite for every real x, positive definite for x ≠ w,
  and the kernel of M(w) is spanned by (1,1).
The comparison M ≥ γK of Lemma S5 (with explicit γ) is certified in `Comparison.lean`.
-/
import Mathlib.Analysis.SpecialFunctions.Trigonometric.DerivHyp
import Mathlib.Analysis.Convex.Deriv
import Mathlib.Analysis.Complex.Norm

open Real

set_option autoImplicit false

noncomputable section

namespace Thm2Cert

/-! ## The auxiliary functions `f x = x / sinh x` and `b x = x coth x` -/

/-- `f x = x / sinh x`, extended continuously by `f 0 = 1`. -/
def f (x : ℝ) : ℝ := if x = 0 then 1 else x / sinh x

/-- `b x = x coth x`, written as `f x * cosh x` (so that `b 0 = 1`). -/
def b (x : ℝ) : ℝ := f x * cosh x

@[simp] lemma f_zero : f 0 = 1 := by simp [f]

lemma f_mul_sinh (x : ℝ) : f x * sinh x = x := by
  unfold f
  split_ifs with h
  · simp [h]
  · have : sinh x ≠ 0 := sinh_ne_zero.mpr h
    field_simp

lemma f_pos (x : ℝ) : 0 < f x := by
  unfold f
  split_ifs with h
  · exact one_pos
  · rcases lt_or_gt_of_ne h with hx | hx
    · exact div_pos_of_neg_of_neg hx (sinh_neg_iff.mpr hx)
    · exact div_pos hx (sinh_pos_iff.mpr hx)

lemma f_neg (x : ℝ) : f (-x) = f x := by
  unfold f
  by_cases h : x = 0
  · simp [h]
  · have h' : -x ≠ 0 := neg_ne_zero.mpr h
    simp only [h, h', ite_false, sinh_neg, neg_div_neg_eq]

lemma b_eq {x : ℝ} (hx : x ≠ 0) : b x = x * cosh x / sinh x := by
  unfold b f; simp only [hx, ite_false]; ring

/-! ## Two monotonicity facts -/

lemma sinh_strictConvexOn : StrictConvexOn ℝ (Set.Ici (0:ℝ)) sinh := by
  apply StrictMonoOn.strictConvexOn_of_deriv (convex_Ici 0) continuous_sinh.continuousOn
  rw [interior_Ici, Real.deriv_sinh]
  exact cosh_strictMonoOn.mono Set.Ioi_subset_Ici_self

/-- (L1) `sinh t / t` is strictly increasing on `(0, ∞)`, in cross-multiplied form. -/
lemma L1 {a c : ℝ} (ha : 0 < a) (hac : a < c) : c * sinh a < a * sinh c := by
  have hc : 0 < c := ha.trans hac
  have h1 : (0:ℝ) < 1 - a / c := by rw [sub_pos, div_lt_one hc]; exact hac
  have h2 : (0:ℝ) < a / c := div_pos ha hc
```

```
  have h := sinh_strictConvexOn.2 (Set.self_mem_Ici) (Set.mem_Ici.mpr hc.le) hc.ne h1 h2
    (by ring)
  simp only [smul_eq_mul, mul_zero, zero_add, sinh_zero] at h
  rw [div_mul_cancel₀ a hc.ne'] at h
  calc c * sinh a < c * (a / c * sinh c) := mul_lt_mul_of_pos_left h hc
    _ = a * sinh c := by field_simp

/-- (L2) the identity behind the monotonicity of `sinh a / sinh (a + c)`. -/
lemma L2 (a a' c : ℝ) :
    sinh a' * sinh (a + c) - sinh a * sinh (a' + c) = sinh (a' - a) * sinh c := by
  simp only [sinh_add, sinh_sub]; ring

/-! ## Sign of `g(x) = w sinh x − sinh² w · f x` -/

lemma g_pos {w x : ℝ} (hw : 0 < w) (hx : w < x) : 0 < w * sinh x - sinh w ^ 2 * f x := by
  have hx0 : 0 < x := hw.trans hx
  have hs : 0 < sinh w := sinh_pos_iff.mpr hw
  have hsx : sinh w < sinh x := sinh_lt_sinh.mpr hx
  have hsxp : 0 < sinh x := hs.trans hsx
  have hL := L1 hw hx
  have key : (w * sinh x - sinh w ^ 2 * f x) * sinh x
      = w * sinh x * sinh x - x * sinh w * sinh w := by
    linear_combination (-(sinh w) ^ 2) * f_mul_sinh x
  have h1 := mul_lt_mul_of_pos_right hL hsxp
  have h2 := mul_lt_mul_of_pos_left hsx (mul_pos hx0 hs)
  have h3 : 0 < (w * sinh x - sinh w ^ 2 * f x) * sinh x := by rw [key]; linarith
  nlinarith

lemma g_neg {w x : ℝ} (hw : 0 < w) (hx : x < w) : w * sinh x - sinh w ^ 2 * f x < 0 := by
  have hs : 0 < sinh w := sinh_pos_iff.mpr hw
  have hfs : 0 < sinh w ^ 2 * f x := mul_pos (pow_pos hs 2) (f_pos x)
  rcases le_or_gt x 0 with hx0 | hx0
  · have : sinh x ≤ 0 := sinh_nonpos_iff.mpr hx0
    nlinarith
  · have hsx : sinh x < sinh w := sinh_lt_sinh.mpr hx
    have hsxp : 0 < sinh x := sinh_pos_iff.mpr hx0
    have hL := L1 hx0 hx
    have key : (w * sinh x - sinh w ^ 2 * f x) * sinh x
        = w * sinh x * sinh x - x * sinh w * sinh w := by
      linear_combination (-(sinh w) ^ 2) * f_mul_sinh x
    have h1 := mul_lt_mul_of_pos_right hL hsxp
    have h2 := mul_lt_mul_of_pos_left hsx (mul_pos hx0 hs)
    have h3 : (w * sinh x - sinh w ^ 2 * f x) * sinh x < 0 := by rw [key]; linarith
    nlinarith

/-! ## Sign of `J(x) = w sinh(x/2) − sinh(w/2) f(x) sinh((x+w)/2)` (that is, `w · I(x)`) -/

/-- `J w x = w * I(x)` with `I` as in the proof of Lemma S5. -/
def J (w x : ℝ) : ℝ := w * sinh (x / 2) - sinh (w / 2) * f x * sinh ((x + w) / 2)

lemma J_key (w x : ℝ) : J w x * sinh x * sinh w
    = (w * sinh x) * (sinh (x / 2) * sinh w)
      - (x * sinh w) * (sinh (w / 2) * sinh ((x + w) / 2)) := by
  unfold J
  linear_combination (-(sinh (w / 2)) * sinh ((x + w) / 2) * sinh w) * f_mul_sinh x

lemma J_pos {w x : ℝ} (hw : 0 < w) (hx : w < x) : 0 < J w x := by
  have hx0 : 0 < x := hw.trans hx
  have hS : 0 < sinh w := sinh_pos_iff.mpr hw
  have hs : 0 < sinh (w / 2) := sinh_pos_iff.mpr (by linarith)
  have hsx : 0 < sinh x := sinh_pos_iff.mpr hx0
  have hL := L1 hw hx
  have h2 := L2 (w / 2) (x / 2) (w / 2)
  rw [show w / 2 + w / 2 = w by ring, show x / 2 + w / 2 = (x + w) / 2 by ring] at h2
  have h2' : sinh (w / 2) * sinh ((x + w) / 2) < sinh (x / 2) * sinh w := by
    have : 0 < sinh (x / 2 - w / 2) * sinh (w / 2) :=
      mul_pos (sinh_pos_iff.mpr (by linarith)) hs
    linarith
  have hpos : 0 < sinh (w / 2) * sinh ((x + w) / 2) :=
    mul_pos hs (sinh_pos_iff.mpr (by linarith))
  have hprod := mul_lt_mul'' hL h2' (mul_pos hx0 hS).le hpos.le
  have : 0 < J w x * sinh x * sinh w := by rw [J_key]; linarith
  have hm : 0 < sinh x * sinh w := mul_pos hsx hS
  nlinarith

lemma J_neg {w x : ℝ} (hw : 0 < w) (hx : x < w) : J w x < 0 := by
  have hS : 0 < sinh w := sinh_pos_iff.mpr hw
  have hs : 0 < sinh (w / 2) := sinh_pos_iff.mpr (by linarith)
  rcases lt_trichotomy x 0 with hx0 | hx0 | hx0
  · rcases le_or_gt (-w) x with hxw | hxw
    · -- −w ≤ x < 0
      have h1 : sinh (x / 2) < 0 := sinh_neg_iff.mpr (by linarith)
      have h2 : 0 ≤ sinh ((x + w) / 2) := sinh_nonneg_iff.mpr (by linarith)
      have h3 : 0 < sinh (w / 2) * f x := mul_pos hs (f_pos x)
      unfold J
      nlinarith
    · -- x < −w : write x = −z with z > w
      obtain ⟨z, rfl⟩ : ∃ z, x = -z := ⟨-x, by ring⟩
      have hzw : w < z := by linarith
      have hz0 : 0 < z := hw.trans hzw
      have hsz : 0 < sinh z := sinh_pos_iff.mpr hz0
```

```
      have hL := L1 hw hzw
      have e : f z * sinh w * sinh z = z * sinh w := by
        linear_combination sinh w * f_mul_sinh z
      have hfz : f z * sinh w < w := by nlinarith
      have hsS : sinh (w / 2) < sinh w := sinh_lt_sinh.mpr (by linarith)
      have hsf : sinh (w / 2) * f z < w := by
        have := mul_lt_mul_of_pos_right hsS (f_pos z)
        nlinarith
      have hA : sinh ((-z + w) / 2) = -sinh ((z - w) / 2) := by
        rw [← sinh_neg]; congr 1; ring
      have hB : sinh (-z / 2) = -sinh (z / 2) := by
        rw [← sinh_neg]; congr 1; ring
      have hC0 : 0 < sinh ((z - w) / 2) := sinh_pos_iff.mpr (by linarith)
      have hC1 : sinh ((z - w) / 2) < sinh (z / 2) := sinh_lt_sinh.mpr (by linarith)
      have i1 := mul_lt_mul_of_pos_right hsf hC0
      have i2 := mul_lt_mul_of_pos_left hC1 hw
      unfold J
      rw [hA, hB, f_neg]
      nlinarith
  · subst hx0
    unfold J
    rw [f_zero]
    simp only [zero_div, sinh_zero, mul_zero, zero_add, mul_one]
    nlinarith
  · have hsx : 0 < sinh x := sinh_pos_iff.mpr hx0
    have hL := L1 hx0 hx
    have h2 := L2 (x / 2) (w / 2) (w / 2)
    rw [show w / 2 + w / 2 = w by ring, show x / 2 + w / 2 = (x + w) / 2 by ring] at h2
    have h2' : sinh (x / 2) * sinh w < sinh (w / 2) * sinh ((x + w) / 2) := by
      have : 0 < sinh (w / 2 - x / 2) * sinh (w / 2) :=
        mul_pos (sinh_pos_iff.mpr (by linarith)) hs
      linarith
    have hpos : 0 < sinh (x / 2) * sinh w := mul_pos (sinh_pos_iff.mpr (by linarith)) hS
    have hprod := mul_lt_mul'' hL h2' (mul_pos hw hsx).le hpos.le
    have : J w x * sinh x * sinh w < 0 := by rw [J_key]; linarith
    have hm : 0 < sinh x * sinh w := mul_pos hsx hS
    nlinarith

/-! ## Pure algebraic identities (certified by `linear_combination`) -/

/-- Eq. (S:detid): decomposition of the determinant of `M/(2q)`. Here
`P = κ cosh w − α`, `Q = κ cosh w − β` (so `P + Q = 2w` is the calibration). -/
lemma det_identity (κ κ0 w P Q S C sy cy sx cx F : ℝ)
    (hQ : Q = 2 * w - P) (hC : C ^ 2 - S ^ 2 = 1) (hy : cy ^ 2 - sy ^ 2 = 1)
    (hsx : sx = sy * C + cy * S) (hcx : cx = cy * C + sy * S) (h3 : κ0 * (C - 1) = w) :
    (κ * S * cy + P * sy + C * F) * (κ * S * cy + Q * sy + C * F) - (κ * S + F * cx) ^ 2
      = ((κ0 * S * cy + w * sy + C * F) ^ 2 - (κ0 * S + F * cx) ^ 2)
        + 2 * (κ - κ0) * sy * (w * sx - S ^ 2 * F)
        + ((κ * C - P) * (κ * C - Q) - κ ^ 2) * sy ^ 2 := by
  subst hQ hsx hcx
  linear_combination (sy ^ 2 * (κ0 - κ) * (κ + κ0)) * hC
    + (-S ^ 2 * (κ0 - κ) * (κ + κ0)) * hy + (-sy ^ 2 * (κ0 * (C + 1) - w)) * h3

/-- Half-angle form of the eigenvalue of `M/(2q)` along `(1,1)` at `κ = κ₀`, `δ = 0`. -/
lemma half_minus (κ0 w S C sy cy sxh cxh cx sxw sp cp sr cr F : ℝ)
    (hp : cp ^ 2 - sp ^ 2 = 1) (hr : cr ^ 2 - sr ^ 2 = 1)
    (hw : w = κ0 * (C - 1)) (hC : C = cr ^ 2 + sr ^ 2) (hS : S = 2 * sr * cr)
    (hsy : sy = 2 * sp * cp) (hcy : cy = cp ^ 2 + sp ^ 2)
    (hsxh : sxh = sp * cr + cp * sr) (hcxh : cxh = cp * cr + sp * sr)
    (hcx : cx = cxh ^ 2 + sxh ^ 2) (hsxw : sxw = sp * C + cp * S) :
    sr * ((κ0 * S * cy + w * sy + C * F) - (κ0 * S + F * cx))
      = 2 * sp * (w * sxh - sr * F * sxw) := by
  subst hw; subst hcx; subst hsxw; subst hC hS hsy hcy hsxh hcxh
  linear_combination (-sr * (F * cr ^ 2 + F * sr ^ 2 - 2 * cr * sr * κ0)) * hp
    + (-2 * cr * sp ^ 2 * κ0) * hr

/-- Half-angle form of the eigenvalue of `M/(2q)` along `(1,−1)` at `κ = κ₀`, `δ = 0`. -/
lemma half_plus (κ0 w S C sy cy sxh cxh cx sxw sp cp sr cr F : ℝ)
    (hp : cp ^ 2 - sp ^ 2 = 1) (hr : cr ^ 2 - sr ^ 2 = 1)
    (hw : w = κ0 * (C - 1)) (hC : C = cr ^ 2 + sr ^ 2) (hS : S = 2 * sr * cr)
    (hsy : sy = 2 * sp * cp) (hcy : cy = cp ^ 2 + sp ^ 2)
    (hsxh : sxh = sp * cr + cp * sr) (hcxh : cxh = cp * cr + sp * sr)
    (hcx : cx = cxh ^ 2 + sxh ^ 2) (_hsxw : sxw = sp * C + cp * S) :
    sr * ((κ0 * S * cy + w * sy + C * F) + (κ0 * S + F * cx))
      = 2 * w * cp * cxh + sr * F * (C + cx) := by
  subst hw; subst hcx; subst hC hS hsy hcy hsxh hcxh
  linear_combination (-2 * cr * κ0 * (cr - 1) * (cr + 1)) * hp
    + (-2 * cr * κ0 * (sp ^ 2 + 1)) * hr

/-! ## The matrix `M(x)` and its normalized form `M(x)/(2q)` -/

/-- Diagonal entry of `M(x)` for a general `q` (`a = α` or `a = β`). -/
def Mdiag (κ a q x : ℝ) : ℝ := q * (κ - q * a) * exp x + (a - q * κ) * exp (-x) + (1 + q ^ 2) * f x

/-- Off-diagonal entry of `M(x)` for a general `q`. -/
def Moff (κ q x : ℝ) : ℝ := -κ * (1 - q ^ 2) - 2 * q * b x

/-- Diagonal entry of `M(x)/(2q)`, `q = e^{−w}`: `m = κ S cosh(x−w) + (κC − a) sinh(x−w) + C f`. -/
def mdiag (κ a w x : ℝ) : ℝ :=
  κ * sinh w * cosh (x - w) + (κ * cosh w - a) * sinh (x - w) + cosh w * f x

```

```
/-- Minus the off-diagonal entry of `M(x)/(2q)`. -/
def moff (κ w x : ℝ) : ℝ := κ * sinh w + b x

lemma Mdiag_eq (κ a w x : ℝ) : Mdiag κ a (exp (-w)) x = 2 * exp (-w) * mdiag κ a w x := by
  unfold Mdiag mdiag
  simp only [cosh_eq, sinh_eq, Real.exp_neg, Real.exp_sub, neg_sub]
  field_simp
  ring

lemma Moff_eq (κ w x : ℝ) : Moff κ (exp (-w)) x = -(2 * exp (-w) * moff κ w x) := by
  unfold Moff moff
  simp only [sinh_eq, Real.exp_neg]
  field_simp
  ring

/-! ## Hypotheses of Lemma S5 -/

/-- Hypotheses of Lemma S5: `α, β, κ > 0`, `αβ ≥ κ²`, and `w > 0` solves the calibration
`κ cosh w − w = (α+β)/2` (then `q = e^{−w}`). -/
structure Hyp (κ α β w : ℝ) : Prop where
  hκ : 0 < κ
  hα : 0 < α
  hβ : 0 < β
  hw : 0 < w
  hκαβ : κ ^ 2 ≤ α * β
  hcal : κ * cosh w - w = (α + β) / 2

/-- Sanity check: the hypotheses are satisfiable (`w = 1`, `κ = 2/(cosh 1 − 1)`,
`α = β = κ cosh 1 − 1`). -/
theorem hyp_nonvacuous : ∃ κ α β w : ℝ, Hyp κ α β w := by
  have hC1 : 0 < cosh 1 - 1 := by have := one_lt_cosh.mpr (one_ne_zero (α := ℝ)); linarith
  set κ := 2 / (cosh 1 - 1) with hκ
  have hκpos : 0 < κ := div_pos two_pos hC1
  have hk : κ * (cosh 1 - 1) = 2 := div_mul_cancel₀ 2 hC1.ne'
  refine (κ, κ * cosh 1 - 1, κ * cosh 1 - 1, 1, hκpos, by nlinarith, by nlinarith, one_pos,
    by nlinarith, by ring)

variable {κ α β w : ℝ}

lemma kappa0_le (h : Hyp κ α β w) : w / (cosh w - 1) ≤ κ := by
  have hC1 : 0 < cosh w - 1 := by have := one_lt_cosh.mpr h.hw.ne'; linarith
  have h1 : (2 * κ) ^ 2 ≤ (α + β) ^ 2 := by nlinarith [sq_nonneg (α - β), h.hκαβ]
  have hab : 2 * κ ≤ α + β := by nlinarith [h.hκ, h.hα, h.hβ]
  rw [div_le_iff₀ hC1]
  linarith [h.hcal]

/-- `κ sinh w > w`. -/
lemma kS_gt (h : Hyp κ α β w) : w < κ * sinh w := by
  have hC1 : 0 < cosh w - 1 := by have := one_lt_cosh.mpr h.hw.ne'; linarith
  have hS : 0 < sinh w := sinh_pos_iff.mpr h.hw
  have hSC : cosh w - 1 < sinh w := by
    have h1 := cosh_sub_sinh w
    have h2 : exp (-w) < 1 := Real.exp_lt_one_iff.mpr (by linarith [h.hw])
    have h3 : 0 < exp (-w) := exp_pos _
    linarith
  have hk := kappa0_le h
  rw [div_le_iff₀ hC1] at hk
  nlinarith [h.hκ]

/-- The trace of `M(x)/(2q)` is positive. -/
lemma trace_pos (h : Hyp κ α β w) (x : ℝ) : 0 < mdiag κ α w x + mdiag κ β w x := by
  have hk := kS_gt h
  have hcy : 0 < cosh (x - w) := cosh_pos _
  have he : 0 < cosh (x - w) + sinh (x - w) := by rw [cosh_add_sinh]; exact exp_pos _
  have hCf : 0 < cosh w * f x := mul_pos (cosh_pos _) (f_pos x)
  have hsum : mdiag κ α w x + mdiag κ β w x
      = 2 * (κ * sinh w - w) * cosh (x - w) + 2 * w * (cosh (x - w) + sinh (x - w))
        + 2 * (cosh w * f x) := by
    unfold mdiag; linear_combination (2 * sinh (x - w)) * h.hcal
  rw [hsum]
  have : 0 < (κ * sinh w - w) * cosh (x - w) := mul_pos (by linarith) hcy
  nlinarith [h.hw]

/-- At `x = w` all four entries of `M(w)/(2q)` agree (up to the sign of the off-diagonal). -/
lemma at_w_alpha (κ a w : ℝ) : mdiag κ a w w = moff κ w w := by
  unfold mdiag moff b
  simp only [sub_self, cosh_zero, sinh_zero]
  ring

set_option maxHeartbeats 1000000 in
/-- Main step: `det(M(x)/(2q)) > 0` for `x ≠ w`. -/
theorem det_pos (h : Hyp κ α β w) {x : ℝ} (hx : x ≠ w) :
    0 < mdiag κ α w x * mdiag κ β w x - moff κ w x ^ 2 := by
  have hC1 : 0 < cosh w - 1 := by have := one_lt_cosh.mpr h.hw.ne'; linarith
  set κ0 := w / (cosh w - 1) with hκ0def
  have h3 : κ0 * (cosh w - 1) = w := div_mul_cancel₀ w hC1.ne'
  have hk : κ0 ≤ κ := kappa0_le h
  have hdet := det_identity κ κ0 w (κ * cosh w - α) (κ * cosh w - β) (sinh w) (cosh w)
    (sinh (x - w)) (cosh (x - w)) (sinh x) (cosh x) (f x)
    (by linarith [h.hcal]) (by rw [cosh_sq]; ring) (by rw [cosh_sq]; ring)
    (by rw [← sinh_add, sub_add_cancel]) (by rw [← cosh_add, sub_add_cancel]) h3
```

```
  -- half-angle data
  have hp : cosh ((x - w) / 2) ^ 2 - sinh ((x - w) / 2) ^ 2 = 1 := by rw [cosh_sq]; ring
  have hr : cosh (w / 2) ^ 2 - sinh (w / 2) ^ 2 = 1 := by rw [cosh_sq]; ring
  have hCr : cosh w = cosh (w / 2) ^ 2 + sinh (w / 2) ^ 2 := by
    rw [← cosh_two_mul]; congr 1; ring
  have hSr : sinh w = 2 * sinh (w / 2) * cosh (w / 2) := by
    rw [← sinh_two_mul]; congr 1; ring
  have hsy : sinh (x - w) = 2 * sinh ((x - w) / 2) * cosh ((x - w) / 2) := by
    rw [← sinh_two_mul]; congr 1; ring
  have hcy : cosh (x - w) = cosh ((x - w) / 2) ^ 2 + sinh ((x - w) / 2) ^ 2 := by
    rw [← cosh_two_mul]; congr 1; ring
  have hsxh : sinh (x / 2) = sinh ((x - w) / 2) * cosh (w / 2) + cosh ((x - w) / 2) * sinh (w / 2) := by
    rw [← sinh_add]; congr 1; ring
  have hcxh : cosh (x / 2) = cosh ((x - w) / 2) * cosh (w / 2) + sinh ((x - w) / 2) * sinh (w / 2) := by
    rw [← cosh_add]; congr 1; ring
  have hcx : cosh x = cosh (x / 2) ^ 2 + sinh (x / 2) ^ 2 := by
    rw [← cosh_two_mul]; congr 1; ring
  have hsxw : sinh ((x + w) / 2) = sinh ((x - w) / 2) * cosh w + cosh ((x - w) / 2) * sinh w := by
    rw [← sinh_add]; congr 1; ring
  have hm := half_minus κ0 w (sinh w) (cosh w) (sinh (x - w)) (cosh (x - w)) (sinh (x / 2))
    (cosh (x / 2)) (cosh x) (sinh ((x + w) / 2)) (sinh ((x - w) / 2)) (cosh ((x - w) / 2))
    (sinh (w / 2)) (cosh (w / 2)) (f x) hp hr h3.symm hCr hSr hsy hcy hsxh hcxh hcx hsxw
  have hpl := half_plus κ0 w (sinh w) (cosh w) (sinh (x - w)) (cosh (x - w)) (sinh (x / 2))
    (cosh (x / 2)) (cosh x) (sinh ((x + w) / 2)) (sinh ((x - w) / 2)) (cosh ((x - w) / 2))
    (sinh (w / 2)) (cosh (w / 2)) (f x) hp hr h3.symm hCr hSr hsy hcy hsxh hcxh hcx hsxw
  have hsr : 0 < sinh (w / 2) := sinh_pos_iff.mpr (by linarith [h.hw])
  -- sign facts on either side of x = w
  have hsJ : 0 < sinh ((x - w) / 2) * J w x := by
    rcases lt_or_gt_of_ne hx with hlt | hgt
    · exact mul_pos_of_neg_of_neg (sinh_neg_iff.mpr (by linarith)) (J_neg h.hw hlt)
    · exact mul_pos (sinh_pos_iff.mpr (by linarith)) (J_pos h.hw hgt)
  have hsg : 0 ≤ sinh (x - w) * (w * sinh x - sinh w ^ 2 * f x) := by
    rcases lt_or_gt_of_ne hx with hlt | hgt
    · exact (mul_pos_of_neg_of_neg (sinh_neg_iff.mpr (by linarith)) (g_neg h.hw hlt)).le
    · exact (mul_pos (sinh_pos_iff.mpr (by linarith)) (g_pos h.hw hgt)).le
  -- the two eigenvalues at κ = κ₀, δ = 0
  have hAm : 0 < (κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
      - (κ0 * sinh w + f x * cosh x) := by
    have e1 : 0 < sinh (w / 2) * ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
      - (κ0 * sinh w + f x * cosh x)) := by
      rw [hm]; unfold J at hsJ; linarith
    exact (mul_pos_iff_of_pos_left hsr).mp e1
  have hAp : 0 < (κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
      + (κ0 * sinh w + f x * cosh x) := by
    have q1 : 0 < 2 * w * cosh ((x - w) / 2) * cosh (x / 2) :=
      mul_pos (mul_pos (by linarith [h.hw]) (cosh_pos _)) (cosh_pos _)
    have q2 : 0 < sinh (w / 2) * f x * (cosh w + cosh x) :=
      mul_pos (mul_pos hsr (f_pos x)) (by linarith [cosh_pos w, cosh_pos x])
    have e1 : 0 < sinh (w / 2) * ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
      + (κ0 * sinh w + f x * cosh x)) := by rw [hpl]; linarith
    exact (mul_pos_iff_of_pos_left hsr).mp e1
  have hΔ0 : 0 < (κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x) ^ 2
      - (κ0 * sinh w + f x * cosh x) ^ 2 := by
    have e : (κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x) ^ 2
        - (κ0 * sinh w + f x * cosh x) ^ 2
        = ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
          - (κ0 * sinh w + f x * cosh x))
          * ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
          + (κ0 * sinh w + f x * cosh x)) := by ring
    rw [e]; exact mul_pos hAm hAp
  have hT2 : 0 ≤ 2 * (κ - κ0) * sinh (x - w) * (w * sinh x - sinh w ^ 2 * f x) := by
    have : 0 ≤ 2 * (κ - κ0) := by linarith
    have := mul_nonneg this hsg
    linarith
  have hT3 : 0 ≤ ((κ * cosh w - (κ * cosh w - α)) * (κ * cosh w - (κ * cosh w - β)) - κ ^ 2)
      * sinh (x - w) ^ 2 := by
    have : (κ * cosh w - (κ * cosh w - α)) * (κ * cosh w - (κ * cosh w - β)) - κ ^ 2
        = α * β - κ ^ 2 := by ring
    rw [this]
    exact mul_nonneg (by linarith [h.hκαβ]) (sq_nonneg _)
  unfold mdiag moff b
  linarith [hdet, hΔ0, hT2, hT3]

/-- `det(M(x)/(2q)) ≥ 0` for every real `x`. -/
theorem det_nonneg (h : Hyp κ α β w) (x : ℝ) :
    0 ≤ mdiag κ α w x * mdiag κ β w x - moff κ w x ^ 2 := by
  by_cases hx : x = w
  · rw [hx, at_w_alpha, at_w_alpha]; nlinarith [sq_nonneg (moff κ w w)]
  · exact (det_pos h hx).le

lemma diag_nonneg (h : Hyp κ α β w) (x : ℝ) :
    0 ≤ mdiag κ α w x ∧ 0 ≤ mdiag κ β w x := by
  have hd := det_nonneg h x
  have ht := trace_pos h x
  constructor <;> nlinarith [sq_nonneg (moff κ w x), sq_nonneg (mdiag κ α w x), sq_nonneg (mdiag κ β w x)]

/-! ## Quadratic forms of 2×2 symmetric matrices -/

lemma quad_nonneg {a c d : ℝ} (ha : 0 ≤ a) (hdet : d ^ 2 ≤ a * c) (hc : 0 ≤ c) (u v : ℝ) :
    0 ≤ a * u ^ 2 - 2 * d * u * v + c * v ^ 2 := by
  rcases ha.eq_or_lt with h0 | hpos
```

```
  · subst h0
    have hd : d = 0 := by
      have : d ^ 2 ≤ 0 := by linarith
      exact pow_eq_zero_iff (n := 2) (by norm_num) |>.mp (le_antisymm this (sq_nonneg d))
    subst hd
    nlinarith [sq_nonneg v]
  · nlinarith [sq_nonneg (a * u - d * v), mul_nonneg (sub_nonneg.2 hdet) (sq_nonneg v)]

lemma quad_pos {a c d : ℝ} (ha : 0 < a) (hdet : d ^ 2 < a * c) {u v : ℝ}
    (huv : u ≠ 0 ∨ v ≠ 0) : 0 < a * u ^ 2 - 2 * d * u * v + c * v ^ 2 := by
  by_cases hv : v = 0
  · subst hv
    have hu : u ≠ 0 := by tauto
    have : 0 < u ^ 2 := by positivity
    nlinarith
  · have : 0 < v ^ 2 := by positivity
    nlinarith [sq_nonneg (a * u - d * v), mul_pos (sub_pos.2 hdet) this]

/-! ## Lemma S5 (positivity and kernel of `M(x)`) -/

/-- The quadratic form of `M(x)`, `q = e^{-w}`. -/
def QM (κ α β w x u v : ℝ) : ℝ :=
  Mdiag κ α (exp (-w)) x * u ^ 2 + 2 * Moff κ (exp (-w)) x * u * v + Mdiag κ β (exp (-w)) x * v ^ 2

lemma QM_eq (κ α β w x u v : ℝ) : QM κ α β w x u v
    = 2 * exp (-w) * (mdiag κ α w x * u ^ 2 - 2 * moff κ w x * u * v + mdiag κ β w x * v ^ 2) := by
  unfold QM; rw [Mdiag_eq, Mdiag_eq, Moff_eq]; ring

/-- **Lemma S5 (i)**: `M(x) ≥ 0` for every real `x`. -/
theorem lemmaS5_psd (h : Hyp κ α β w) (x u v : ℝ) : 0 ≤ QM κ α β w x u v := by
  rw [QM_eq]
  have ⟨h1, h2⟩ := diag_nonneg h x
  have hd := det_nonneg h x
  exact mul_nonneg (by positivity) (quad_nonneg h1 (by linarith) h2 u v)

/-- **Lemma S5 (ii)**: `M(x)` is positive definite for `x ≠ w`, i.e. `ker M(x) = {0}`. -/
theorem lemmaS5_pd (h : Hyp κ α β w) {x : ℝ} (hx : x ≠ w) {u v : ℝ} (huv : u ≠ 0 ∨ v ≠ 0) :
    0 < QM κ α β w x u v := by
  rw [QM_eq]
  have hd := det_pos h hx
  have ⟨h1, h2⟩ := diag_nonneg h x
  have ha : 0 < mdiag κ α w x := by
    rcases h1.eq_or_lt with h0 | h0
    · rw [← h0] at hd; nlinarith [sq_nonneg (moff κ w x)]
    · exact h0
  exact mul_pos (by positivity) (quad_pos ha (by linarith) huv)

/-- **Lemma S5 (iii)**: at `x = w`, the quadratic form vanishes exactly on `span{(1,1)}`,
i.e. `ker M(w) = span{(1,1)}`. -/
theorem lemmaS5_kernel (h : Hyp κ α β w) (u v : ℝ) : QM κ α β w w u v = 0 ↔ u = v := by
  rw [QM_eq, at_w_alpha, at_w_alpha]
  have ht := trace_pos h w
  rw [at_w_alpha, at_w_alpha] at ht
  have hq : 0 < 2 * exp (-w) := by positivity
  have hc : 0 < moff κ w w := by linarith
  have e : moff κ w w * u ^ 2 - 2 * moff κ w w * u * v + moff κ w w * v ^ 2
      = moff κ w w * (u - v) ^ 2 := by ring
  rw [e]
  constructor
  · intro h0
    have : moff κ w w * (u - v) ^ 2 = 0 := by
      rcases mul_eq_zero.mp h0 with h1 | h1
      · linarith
      · exact h1
    rcases mul_eq_zero.mp this with h1 | h1
    · linarith
    · have := pow_eq_zero_iff (n := 2) (by norm_num) |>.mp h1
      linarith
  · intro huv; subst huv; ring

/-- `M(w) (1,1)ᵀ = 0`, entrywise. -/
theorem lemmaS5_null :
    Mdiag κ α (exp (-w)) w + Moff κ (exp (-w)) w = 0 ∧
    Moff κ (exp (-w)) w + Mdiag κ β (exp (-w)) w = 0 := by
  rw [Mdiag_eq, Mdiag_eq, Moff_eq, at_w_alpha, at_w_alpha]
  constructor <;> ring

/-- Complex form used in Eq. (S:mainid): for `z = (A, conj B)`,
`z† M(x) z = M₁₁|A|² + M₂₂|B|² + 2 M₁₂ Re(AB) ≥ 0`. -/
theorem lemmaS5_complex (h : Hyp κ α β w) (x : ℝ) (A B : ℂ) :
    0 ≤ Mdiag κ α (exp (-w)) x * Complex.normSq A + Mdiag κ β (exp (-w)) x * Complex.normSq B
      + 2 * Moff κ (exp (-w)) x * (A * B).re := by
  have hpsd := lemmaS5_psd h x ‖A‖ ‖B‖
  unfold QM at hpsd
  have hoff : Moff κ (exp (-w)) x ≤ 0 := by
    rw [Moff_eq]
    have : 0 < moff κ w x := by
      unfold moff b
      have := kS_gt h
      have : 0 < f x * cosh x := mul_pos (f_pos x) (cosh_pos x)
      nlinarith [h.hw]
```

```
    have : 0 < exp (-w) := exp_pos _
    nlinarith
  have hre : (A * B).re ≤ ‖A‖ * ‖B‖ := by
    rw [← Complex.norm_mul]; exact Complex.re_le_norm _
  rw [Complex.normSq_eq_norm_sq, Complex.normSq_eq_norm_sq]
  nlinarith [mul_le_mul_of_nonpos_left hre hoff]

/-- The off-diagonal entry is negative: `M₁₂(x) < 0`. -/
lemma Moff_neg (h : Hyp κ α β w) (x : ℝ) : Moff κ (exp (-w)) x < 0 := by
  rw [Moff_eq]
  have : 0 < moff κ w x := by
    unfold moff b
    have := kS_gt h
    have : 0 < f x * cosh x := mul_pos (f_pos x) (cosh_pos x)
    nlinarith [h.hw]
  have : 0 < exp (-w) := exp_pos _
  nlinarith

/-- Complex strict form: for `x ≠ w` and `z = (A, conj B) ≠ 0`, `z† M(x) z > 0`. -/
theorem lemmaS5_complex_pd (h : Hyp κ α β w) {x : ℝ} (hx : x ≠ w) {A B : ℂ}
    (hAB : A ≠ 0 ∨ B ≠ 0) :
    0 < Mdiag κ α (exp (-w)) x * Complex.normSq A + Mdiag κ β (exp (-w)) x * Complex.normSq B
      + 2 * Moff κ (exp (-w)) x * (A * B).re := by
  have huv : ‖A‖ ≠ 0 ∨ ‖B‖ ≠ 0 := by
    rcases hAB with hA | hB
    · exact Or.inl (norm_ne_zero_iff.mpr hA)
    · exact Or.inr (norm_ne_zero_iff.mpr hB)
  have hpd := lemmaS5_pd h hx huv
  unfold QM at hpd
  have hoff := (Moff_neg h x).le
  have hre : (A * B).re ≤ ‖A‖ * ‖B‖ := by
    rw [← Complex.norm_mul]; exact Complex.re_le_norm _
  rw [Complex.normSq_eq_norm_sq, Complex.normSq_eq_norm_sq]
  nlinarith [mul_le_mul_of_nonpos_left hre hoff]

/-- At `x = w`: `z† M(w) z = 2q (κ sinh w + b(w)) |A − conj B|²`. -/
theorem lemmaS5_complex_at_w (A B : ℂ) :
    Mdiag κ α (exp (-w)) w * Complex.normSq A + Mdiag κ β (exp (-w)) w * Complex.normSq B
      + 2 * Moff κ (exp (-w)) w * (A * B).re
      = 2 * exp (-w) * moff κ w w * Complex.normSq (A - (starRingEnd ℂ) B) := by
  rw [Mdiag_eq, Mdiag_eq, Moff_eq, at_w_alpha, at_w_alpha]
  simp only [Complex.normSq_apply, Complex.sub_re, Complex.sub_im, Complex.conj_re,
    Complex.conj_im, Complex.mul_re]
  ring

/-- Complex kernel at `x = w`: `z† M(w) z = 0` iff `z = (A, conj B)` lies in `span{(1,1)}`. -/
theorem lemmaS5_complex_kernel (h : Hyp κ α β w) (A B : ℂ) :
    Mdiag κ α (exp (-w)) w * Complex.normSq A + Mdiag κ β (exp (-w)) w * Complex.normSq B
      + 2 * Moff κ (exp (-w)) w * (A * B).re = 0 ↔ A = (starRingEnd ℂ) B := by
  rw [lemmaS5_complex_at_w]
  have ht := trace_pos h w
  rw [at_w_alpha, at_w_alpha] at ht
  have hc : 0 < 2 * exp (-w) * moff κ w w := mul_pos (by positivity) (by linarith)
  rw [mul_eq_zero, Complex.normSq_eq_zero, sub_eq_zero]
  constructor
  · rintro (h0 | h0)
    · linarith
    · exact h0
  · intro h0; exact Or.inr h0

lemma Hyp.swap (h : Hyp κ α β w) : Hyp κ β α w :=
  ⟨h.hκ, h.hβ, h.hα, h.hw, by linarith [h.hκαβ, mul_comm α β], by linarith [h.hcal]⟩

/-- `q κ < α < κ / q` (written as `q α < κ`), with `q = e^{−w}`. -/
lemma alpha_bounds (h : Hyp κ α β w) : exp (-w) * κ < α ∧ exp (-w) * α < κ := by
  have hE : exp w * exp (-w) = 1 := by rw [← exp_add]; simp
  have hEe : exp (-w) < exp w := exp_lt_exp.mpr (by linarith [h.hw])
  have he : 0 < exp (-w) := exp_pos _
  have hβ : β = κ * (exp w + exp (-w)) - 2 * w - α := by
    have := h.hcal; rw [cosh_eq] at this; linarith
  have hk := h.hκαβ
  rw [hβ] at hk
  have hwα : 0 < w * α := mul_pos h.hw h.hα
  have hE2 : κ ^ 2 * (exp w * exp (-w)) = κ ^ 2 := by rw [hE, mul_one]
  constructor
  · by_contra hc
    replace hc := not_lt.mp hc
    have h1 : 0 ≤ (exp (-w) * κ - α) * (exp w * κ - α) :=
      mul_nonneg (by linarith) (by nlinarith [h.hκ])
    nlinarith
  · by_contra hc
    replace hc := not_lt.mp hc
    have hα' : exp w * κ ≤ α := by
      have h1 := mul_le_mul_of_nonneg_left hc (exp_pos w).le
      have h2 : exp w * (exp (-w) * α) = α := by rw [← mul_assoc, hE, one_mul]
      linarith
    have h1 : 0 ≤ (α - exp w * κ) * (α - exp (-w) * κ) :=
      mul_nonneg (by linarith) (by nlinarith [h.hκ])
    nlinarith

end Thm2Cert
```

Thm2Cert/Identity.lean

```
/-
Certificate for the exact identity Eq. (S:mainid) of the Supplemental Material:
adding the two second variations Q(O₁ψ), Q(O₂ψ) of Eq. (S:Q12), and using the sum rules
Eq. (S:wccr), gives  −∑_{l,r} c_l c_r z†_{lr} M(x_{lr}) z_{lr}.

Input (taken as hypotheses / definitions, NOT derived here): the expressions (S:Q12) for the
second variations, and the sum rules ⟨N⟩ = ∑ p_r|A_lr|², ⟨N⟩+1 = ∑ p_l|A_lr|².
Setting: a finite index set. `mainid` is Eq. (S:mainid) verbatim, for Schmidt coefficients
c_l > 0, with p_l = c_l², L_lr = (ln p_l − ln p_r)/(p_l − p_r) (= 1/p_l if p_l = p_r),
x_lr = ln(c_l/c_r). `mainid_general` / `sumQ_nonpos_general` allow c_l ≥ 0, so that kernel
directions (support-to-kernel entries, retained in the text as continuous limits) are included;
this covers every state of finite Schmidt rank. Infinite Schmidt rank (the limiting argument of
the SM) is not covered.
-/
import Thm2Cert.MatrixLemma
import Mathlib.Analysis.SpecialFunctions.Log.Basic

open Real Finset

set_option autoImplicit false

noncomputable section

namespace Thm2Cert

/-- The logarithmic mean kernel `L(p,p') = (ln p − ln p')/(p − p')`, `= 1/p` on the diagonal. -/
def Lker (p p' : ℝ) : ℝ := if p = p' then 1 / p else (log p - log p') / (p - p')

section scalar

variable {cl cr : ℝ}

lemma E1 (hl : 0 < cl) (hr : 0 < cr) : cl * cr * exp (log (cl / cr)) = cl ^ 2 := by
  rw [exp_log (div_pos hl hr)]; field_simp

lemma E2 (hl : 0 < cl) (hr : 0 < cr) : cl * cr * exp (-log (cl / cr)) = cr ^ 2 := by
  rw [exp_neg, exp_log (div_pos hl hr)]; field_simp

lemma E3 (hl : 0 < cl) (hr : 0 < cr) :
    cl ^ 2 * cr ^ 2 * Lker (cl ^ 2) (cr ^ 2) = cl * cr * f (log (cl / cr)) := by
  unfold Lker
  by_cases h : cl ^ 2 = cr ^ 2
  · have hlr : cl = cr := (pow_left_inj₀ hl.le hr.le two_ne_zero).mp h
    subst hlr
    simp only [↓reduceIte]
    rw [div_self hl.ne', log_one, f_zero]
    field_simp
  · simp only [h, ↓reduceIte]
    have hlr : cl ≠ cr := fun e => h (by rw [e])
    have hq : cl / cr ≠ 1 := by
      intro e; apply hlr; field_simp at e; linarith
    have hX : log (cl / cr) ≠ 0 := log_ne_zero_of_pos_of_ne_one (div_pos hl hr) hq
    unfold f
    simp only [hX, ↓reduceIte]
    rw [sinh_eq, exp_neg, exp_log (div_pos hl hr), log_pow, log_pow,
      log_div hl.ne' hr.ne']
    have h2 : cl ^ 2 - cr ^ 2 ≠ 0 := sub_ne_zero.mpr h
    have h3 : cl / cr - (cl / cr)⁻¹ ≠ 0 := by
      intro e
      apply h2
      field_simp at e
      linarith
    field_simp
    ring

lemma E3' (hl : 0 < cl) (hr : 0 < cr) :
    cl * cr * Lker (cl ^ 2) (cr ^ 2) = f (log (cl / cr)) := by
  have h := E3 hl hr
  have hc : 0 < cl * cr := mul_pos hl hr
  have : cl * cr * (cl * cr * Lker (cl ^ 2) (cr ^ 2)) = cl * cr * f (log (cl / cr)) := by
    rw [← h]; ring
  exact mul_left_cancel₀ hc.ne' this

lemma E4 (hl : 0 < cl) (hr : 0 < cr) :
    √(cl ^ 2 * cr ^ 2) * (cl ^ 2 + cr ^ 2) * Lker (cl ^ 2) (cr ^ 2)
      = 2 * cl * cr * b (log (cl / cr)) := by
  have hs : √(cl ^ 2 * cr ^ 2) = cl * cr := by
    rw [show cl ^ 2 * cr ^ 2 = (cl * cr) * (cl * cr) by ring]
    exact sqrt_mul_self (mul_pos hl hr).le
  rw [hs]
  unfold b
  rw [← E3' hl hr, cosh_eq, exp_neg, exp_log (div_pos hl hr)]
  field_simp

end scalar

variable {ι : Type*} [Fintype ι]

/-- `k = Re(a b) = ∑ c_l c_r Re(A_lr B_lr)`. -/
```

```
def kk (c : ι → ℝ) (A B : ι → ι → ℂ) : ℝ := ∑ l, ∑ r, c l * c r * (A l r * B l r).re

/-- `A = ∑ p_l p_r L_lr |A_lr|²` (and `B` with `B`). -/
def sfA (c : ι → ℝ) (A : ι → ι → ℂ) : ℝ :=
  ∑ l, ∑ r, c l ^ 2 * c r ^ 2 * Lker (c l ^ 2) (c r ^ 2) * Complex.normSq (A l r)

/-- `C = ∑ √(p_l p_r) (p_l + p_r) L_lr Re(A_lr B_lr)`. -/
def sfC (c : ι → ℝ) (A B : ι → ι → ℂ) : ℝ :=
  ∑ l, ∑ r, √(c l ^ 2 * c r ^ 2) * (c l ^ 2 + c r ^ 2) * Lker (c l ^ 2) (c r ^ 2) * (A l r * B l r).re

/-- Eq. (S:Q12), first line: `Q(O₁ψ)`. -/
def Q1 (α β κ q nA nB k sA sB sC : ℝ) : ℝ :=
  (q * κ - α) * nA + (q ^ 2 * β - q * κ) * (nB + 1) + (κ * (1 - q ^ 2) + q * (α - β)) * k
    - sA - q ^ 2 * sB + q * sC

/-- Eq. (S:Q12), second line: `Q(O₂ψ)`. -/
def Q2 (α β κ q nA nB k sA sB sC : ℝ) : ℝ :=
  (q ^ 2 * α - q * κ) * (nA + 1) + (q * κ - β) * nB + (κ * (1 - q ^ 2) - q * (α - β)) * k
    - q ^ 2 * sA - sB + q * sC

/-- **Eq. (S:mainid)** (exact bookkeeping identity), for an arbitrary real `q`. -/
theorem mainid (c : ι → ℝ) (hc : ∀ l, 0 < c l) (A B : ι → ι → ℂ) (α β κ q nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
    = -∑ l, ∑ r, c l * c r *
        (Mdiag κ α q (log (c l / c r)) * Complex.normSq (A l r)
          + Mdiag κ β q (log (c l / c r)) * Complex.normSq (B l r)
          + 2 * Moff κ q (log (c l / c r)) * (A l r * B l r).re) := by
  have step : Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      = (q * κ - α) * nA + (q ^ 2 * α - q * κ) * (nA + 1)
        + (q ^ 2 * β - q * κ) * (nB + 1) + (q * κ - β) * nB
        + 2 * κ * (1 - q ^ 2) * kk c A B - (1 + q ^ 2) * sfA c A - (1 + q ^ 2) * sfA c B
        + 2 * q * sfC c A B := by
    unfold Q1 Q2; ring
  rw [step, hA1, hB1, hA0, hB0]
  unfold kk sfA sfC
  simp only [Finset.mul_sum, ← Finset.sum_add_distrib, ← Finset.sum_sub_distrib,
    ← Finset.sum_neg_distrib]
  refine Finset.sum_congr rfl (fun l _ => Finset.sum_congr rfl (fun r _ => ?_))
  have e1 := E1 (hc l) (hc r)
  have e2 := E2 (hc l) (hc r)
  have e3 := E3 (hc l) (hc r)
  have e4 := E4 (hc l) (hc r)
  unfold Mdiag Moff
  linear_combination
    ((α - q * κ) * Complex.normSq (A l r) + (β - q * κ) * Complex.normSq (B l r)) * e2
    + (q * (κ - q * α) * Complex.normSq (A l r) + q * (κ - q * β) * Complex.normSq (B l r)) * e1
    - (1 + q ^ 2) * (Complex.normSq (A l r) + Complex.normSq (B l r)) * e3
    + 2 * q * (A l r * B l r).re * e4

/-- **Consequence**: with `q = e^{−w}` and the hypotheses of Lemma S5,
`Q(O₁ψ) + Q(O₂ψ) ≤ 0`. -/
theorem sumQ_nonpos {κ α β w : ℝ} (h : Hyp κ α β w) (c : ι → ℝ) (hc : ∀ l, 0 < c l)
    (A B : ι → ι → ℂ) (nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B) ≤ 0 := by
  rw [mainid c hc A B α β κ (exp (-w)) nA nB hA0 hA1 hB0 hB1, neg_nonpos]
  refine Finset.sum_nonneg (fun l _ => Finset.sum_nonneg (fun r _ => ?_))
  exact mul_nonneg (mul_pos (hc l) (hc r)).le (lemmaS5_complex h _ (A l r) (B l r))

/-! ## The identity for arbitrary `c_l ≥ 0` (kernel directions included)

For a state of finite Schmidt rank, the sum rules (S:wccr) also involve pairs `(l, r)` with
one vanishing Schmidt coefficient (support-to-kernel entries of `a`, `b`). Indexing the support
together with a finite set of kernel directions (`c_l = 0`) covers this case. Here the
coefficients are written through `p_l = c_l²`, `Φ_lr = p_l p_r L_lr` and
`Ψ_lr = √(p_l p_r)(p_l + p_r) L_lr`, which equal `0` when `p_l p_r = 0`
(the "continuous limits" of the text). -/

/-- Diagonal weight `c_l c_r M₁₁(x_lr)` written in the `p`-variables. -/
def wdiag (κ a q pl pr Φ : ℝ) : ℝ := q * (κ - q * a) * pl + (a - q * κ) * pr + (1 + q ^ 2) * Φ

/-- Off-diagonal weight `c_l c_r M₁₂(x_lr)` written in the `p`-variables. -/
def woff (κ q cc Ψ : ℝ) : ℝ := -κ * (1 - q ^ 2) * cc - q * Ψ

/-- The summand of Eq. (S:mainid), for Schmidt coefficients `cl, cr ≥ 0`. -/
def term (κ α β q cl cr : ℝ) (A B : ℂ) : ℝ :=
  wdiag κ α q (cl ^ 2) (cr ^ 2) (cl ^ 2 * cr ^ 2 * Lker (cl ^ 2) (cr ^ 2)) * Complex.normSq A
  + wdiag κ β q (cl ^ 2) (cr ^ 2) (cl ^ 2 * cr ^ 2 * Lker (cl ^ 2) (cr ^ 2)) * Complex.normSq B
  + 2 * woff κ q (cl * cr) (√(cl ^ 2 * cr ^ 2) * (cl ^ 2 + cr ^ 2) * Lker (cl ^ 2) (cr ^ 2))
      * (A * B).re

```

```
/-- **Eq. (S:mainid), general form**: valid for all real `c_l` (no positivity needed). -/
theorem mainid_general (c : ι → ℝ) (A B : ι → ι → ℂ) (α β κ q nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
    = -∑ l, ∑ r, term κ α β q (c l) (c r) (A l r) (B l r) := by
  have step : Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      = (q * κ - α) * nA + (q ^ 2 * α - q * κ) * (nA + 1)
        + (q ^ 2 * β - q * κ) * (nB + 1) + (q * κ - β) * nB
        + 2 * κ * (1 - q ^ 2) * kk c A B - (1 + q ^ 2) * sfA c A - (1 + q ^ 2) * sfA c B
        + 2 * q * sfC c A B := by
    unfold Q1 Q2; ring
  rw [step, hA1, hB1, hA0, hB0]
  unfold kk sfA sfC
  simp only [Finset.mul_sum, ← Finset.sum_add_distrib, ← Finset.sum_sub_distrib,
    ← Finset.sum_neg_distrib]
  refine Finset.sum_congr rfl (fun l _ => Finset.sum_congr rfl (fun r _ => ?_))
  unfold term wdiag woff
  ring

/-- Each summand is nonnegative under the hypotheses of Lemma S5, for all `cl, cr ≥ 0`:
for `cl, cr > 0` it is `c_l c_r z† M(x_lr) z` (Lemma S5); if one coefficient vanishes it
reduces to the boundary weights, nonnegative because `qκ < α, β < κ/q`. -/
theorem term_nonneg {κ α β w : ℝ} (h : Hyp κ α β w) {cl cr : ℝ} (hl : 0 ≤ cl) (hr : 0 ≤ cr)
    (A B : ℂ) : 0 ≤ term κ α β (exp (-w)) cl cr A B := by
  have ⟨ha1, ha2⟩ := alpha_bounds h
  have ⟨hb1, hb2⟩ := alpha_bounds h.swap
  have he : 0 < exp (-w) := exp_pos _
  have nA := Complex.normSq_nonneg A
  have nB := Complex.normSq_nonneg B
  rcases hl.eq_or_lt with hl0 | hlp
  · subst hl0
    unfold term wdiag woff
    simp only [ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, zero_pow, zero_mul, mul_zero,
      Real.sqrt_zero, zero_add, add_zero, sub_zero]
    have : 0 ≤ (α - exp (-w) * κ) * cr ^ 2 * Complex.normSq A :=
      mul_nonneg (mul_nonneg (by linarith) (sq_nonneg _)) nA
    have : 0 ≤ (β - exp (-w) * κ) * cr ^ 2 * Complex.normSq B :=
      mul_nonneg (mul_nonneg (by linarith) (sq_nonneg _)) nB
    nlinarith
  rcases hr.eq_or_lt with hr0 | hrp
  · subst hr0
    unfold term wdiag woff
    simp only [ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, zero_pow, zero_mul, mul_zero,
      Real.sqrt_zero, add_zero, sub_zero]
    have : 0 ≤ exp (-w) * (κ - exp (-w) * α) * cl ^ 2 * Complex.normSq A :=
      mul_nonneg (mul_nonneg (mul_nonneg he.le (by linarith)) (sq_nonneg _)) nA
    have : 0 ≤ exp (-w) * (κ - exp (-w) * β) * cl ^ 2 * Complex.normSq B :=
      mul_nonneg (mul_nonneg (mul_nonneg he.le (by linarith)) (sq_nonneg _)) nB
    nlinarith
  · have e1 := E1 hlp hrp
    have e2 := E2 hlp hrp
    have e3 := E3 hlp hrp
    have e4 := E4 hlp hrp
    have key : term κ α β (exp (-w)) cl cr A B = cl * cr *
        (Mdiag κ α (exp (-w)) (log (cl / cr)) * Complex.normSq A
          + Mdiag κ β (exp (-w)) (log (cl / cr)) * Complex.normSq B
          + 2 * Moff κ (exp (-w)) (log (cl / cr)) * (A * B).re) := by
      unfold term wdiag woff Mdiag Moff
      linear_combination
        (-exp (-w) * (κ - exp (-w) * α) * Complex.normSq A
          - exp (-w) * (κ - exp (-w) * β) * Complex.normSq B) * e1
        + (-(α - exp (-w) * κ) * Complex.normSq A - (β - exp (-w) * κ) * Complex.normSq B) * e2
        + (1 + exp (-w) ^ 2) * (Complex.normSq A + Complex.normSq B) * e3
        - 2 * exp (-w) * (A * B).re * e4
    rw [key]
    exact mul_nonneg (mul_pos hlp hrp).le (lemmaS5_complex h _ A B)

/-- **Consequence, general form**: under the hypotheses of Lemma S5, `Q(O₁ψ) + Q(O₂ψ) ≤ 0`
for all Schmidt coefficients `c_l ≥ 0` (finite index set). -/
theorem sumQ_nonpos_general {κ α β w : ℝ} (h : Hyp κ α β w) (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l)
    (A B : ι → ι → ℂ) (nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B) ≤ 0 := by
  rw [mainid_general c A B α β κ (exp (-w)) nA nB hA0 hA1 hB0 hB1, neg_nonpos]
  exact Finset.sum_nonneg (fun l _ => Finset.sum_nonneg (fun r _ =>
    term_nonneg h (hc l) (hc r) (A l r) (B l r)))

end Thm2Cert
```

Thm2Cert/Reduction.lean

```
/-
Certificate for the scalar steps connecting the minimum principle (Theorem S4 in the SM)
to Theorem 2 of the Letter:
  * the calibration κ cosh w − w = (α+β)/2 with (α,β,κ) = λ(t²,1,t), q = e^{−w},
    is equivalent to λ = λ_{q,t} = −2q ln q / [(t−q)(1−tq)], and λ_{q,t} > 0;
  * 𝓔(q) = g(q²/(1−q²)) = −ln(1−q²) − 2 n_q ln q;
  * the equality value on the two-mode squeezed vacuum:
      𝓔(q) + (α+β) n_q − 2κ q/(1−q²) = −ln(1−q²) − κ q   (under the calibration);
  * 𝓔(q) + λ_{q,t} d_t(q) = −ln(1−q²) − λ_{q,t} t q + λ_{q,t};
  * the m-pair inequality of Theorem 2 follows from the minimum principle.
The TMSV moments ⟨N_A⟩ = ⟨N_B⟩ = q²/(1−q²), ⟨ab⟩ = q/(1−q²), and the operator identity
λ D̂_t = T + λ, are inputs (standard), not derived here.
-/
import Thm2Cert.MatrixLemma
import Mathlib.Analysis.SpecialFunctions.Log.Basic

open Real Finset

set_option autoImplicit false

noncomputable section

namespace Thm2Cert

/-- `λ_{q,t} = −2 q ln q / [(t − q)(1 − t q)]`. -/
def lam (q t : ℝ) : ℝ := -2 * q * log q / ((t - q) * (1 - t * q))

/-- `d_t(q) = (1 − t q)² / (1 − q²)`. -/
def dt (t q : ℝ) : ℝ := (1 - t * q) ^ 2 / (1 - q ^ 2)

/-- `g(y) = (y+1) ln(y+1) − y ln y`. -/
def gfun (y : ℝ) : ℝ := (y + 1) * log (y + 1) - y * log y

/-- `𝓔(q) = g(q²/(1−q²))`, the entanglement of the TMSV. -/
def EE (q : ℝ) : ℝ := gfun (q ^ 2 / (1 - q ^ 2))

lemma calib_factor (w t : ℝ) :
    t * cosh w - (1 + t ^ 2) / 2 = (t - exp (-w)) * (1 - t * exp (-w)) / (2 * exp (-w)) := by
  rw [cosh_eq, exp_neg]
  field_simp
  ring

lemma lam_pos {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) : 0 < lam q t := by
  unfold lam
  have hq1 : q < 1 := lt_of_lt_of_le hqt ht
  have hl : log q < 0 := log_neg hq hq1
  have h1 : 0 < t - q := by linarith
  have h2 : 0 < 1 - t * q := by nlinarith
  apply div_pos _ (mul_pos h1 h2)
  nlinarith

/-- The calibration for `(α, β, κ) = λ(t², 1, t)` holds iff `λ = λ_{q,t}`, `q = e^{−w}`. -/
theorem calib_iff {w t l : ℝ} (hw : 0 < w) (hqt : exp (-w) < t) (ht : t ≤ 1) :
    (l * t) * cosh w - w = (l * t ^ 2 + l * 1) / 2 ↔ l = lam (exp (-w)) t := by
  have hq : 0 < exp (-w) := exp_pos _
  have hq1 : exp (-w) < 1 := exp_lt_one_iff.mpr (by linarith)
  have h1 : 0 < t - exp (-w) := by linarith
  have h2 : 0 < 1 - t * exp (-w) := by nlinarith
  set D := (t - exp (-w)) * (1 - t * exp (-w)) / (2 * exp (-w)) with hDdef
  have hD : 0 < D := div_pos (mul_pos h1 h2) (by positivity)
  have key : (l * t) * cosh w - w - (l * t ^ 2 + l * 1) / 2 = l * D - w := by
    rw [hDdef, ← calib_factor w t]; ring
  have hlam : lam (exp (-w)) t = w / D := by
    unfold lam; rw [log_exp, hDdef]; field_simp
  rw [hlam, eq_div_iff hD.ne']
  constructor <;> intro h <;> linarith

lemma EE_formula {q : ℝ} (hq : 0 < q) (hq1 : q < 1) :
    EE q = -log (1 - q ^ 2) - 2 * (q ^ 2 / (1 - q ^ 2)) * log q := by
  have hd : 0 < 1 - q ^ 2 := by nlinarith
  unfold EE gfun
  have e1 : q ^ 2 / (1 - q ^ 2) + 1 = (1 - q ^ 2)⁻¹ := by field_simp; ring
  rw [e1, log_inv, log_div (pow_pos hq 2).ne' hd.ne', log_pow]
  push_cast
  field_simp
  ring

/-- Equality in the minimum principle on the two-mode squeezed vacuum (scalar part). -/
theorem tmsv_value {κ α β w : ℝ} (hcal : κ * cosh w - w = (α + β) / 2) (hw : 0 < w) :
    EE (exp (-w)) + (α + β) * (exp (-w) ^ 2 / (1 - exp (-w) ^ 2))
      - κ * (2 * exp (-w) / (1 - exp (-w) ^ 2))
      = -log (1 - exp (-w) ^ 2) - κ * exp (-w) := by
  have hq : 0 < exp (-w) := exp_pos _
  have hq1 : exp (-w) < 1 := exp_lt_one_iff.mpr (by linarith)
  have hd : 0 < 1 - exp (-w) ^ 2 := by nlinarith
  have hab : α + β = κ * (exp w + exp (-w)) - 2 * w := by rw [cosh_eq] at hcal; linarith
  have hinv : exp w = (exp (-w))⁻¹ := by rw [exp_neg, inv_inv]
  rw [EE_formula hq hq1, log_exp, hab, hinv]
  generalize exp (-w) = q at hq hq1 hd ⊢
```

```
  field_simp
  ring

/-- `𝓔(q) + λ_{q,t} d_t(q) = −ln(1−q²) − λ_{q,t} t q + λ_{q,t}`. -/
theorem energy_identity {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) :
    EE q + lam q t * dt t q = -log (1 - q ^ 2) - lam q t * t * q + lam q t := by
  have hq1 : q < 1 := lt_of_lt_of_le hqt ht
  have hd : 0 < 1 - q ^ 2 := by nlinarith
  have h1 : 0 < t - q := by linarith
  have h2 : 0 < 1 - t * q := by nlinarith
  have h2' : 1 - q * t ≠ 0 := by rw [mul_comm]; exact h2.ne'
  rw [EE_formula hq hq1]
  unfold lam dt
  field_simp
  ring

/-- **Theorem 2 from the minimum principle.** For `m` pairs with `0 < q_j < t_j ≤ 1`,
`λ_j = λ_{q_j,t_j}`, `κ_j = λ_j t_j`, and `⟨T_j⟩ = λ_j(⟨D̂^{(j)}⟩ − 1)`, the minimum principle
`S + ∑ ⟨T_j⟩ ≥ ∑ [−ln(1−q_j²) − κ_j q_j]` implies
`S + ∑ λ_j [⟨D̂^{(j)}⟩ − d_{t_j}(q_j)] ≥ ∑ 𝓔(q_j)`. -/
theorem thm2_of_minprinciple {m : ℕ} (q t D : Fin m → ℝ) (Sent : ℝ)
    (hq : ∀ j, 0 < q j) (hqt : ∀ j, q j < t j) (ht : ∀ j, t j ≤ 1)
    (hmin : Sent + ∑ j, lam (q j) (t j) * (D j - 1)
      ≥ ∑ j, (-log (1 - q j ^ 2) - (lam (q j) (t j) * t j) * q j)) :
    Sent + ∑ j, lam (q j) (t j) * (D j - dt (t j) (q j)) ≥ ∑ j, EE (q j) := by
  have hE : ∑ j, EE (q j) = ∑ j, ((-log (1 - q j ^ 2) - (lam (q j) (t j) * t j) * q j)
      + lam (q j) (t j) - lam (q j) (t j) * dt (t j) (q j)) := by
    refine Finset.sum_congr rfl (fun j _ => ?_)
    have := energy_identity (hq j) (hqt j) (ht j)
    linarith
  have hD : ∑ j, lam (q j) (t j) * (D j - dt (t j) (q j))
      = ∑ j, (lam (q j) (t j) * (D j - 1) + lam (q j) (t j)
        - lam (q j) (t j) * dt (t j) (q j)) := by
    refine Finset.sum_congr rfl (fun j _ => ?_); ring
  rw [hE, hD]
  simp only [Finset.sum_add_distrib, Finset.sum_sub_distrib] at hmin ⊢
  linarith

/-- The parameters of Theorem 2 satisfy the hypotheses of Lemma S5: for `0 < q < t ≤ 1`,
`w = −ln q`, `λ = λ_{q,t}`, the triple `(α, β, κ) = λ(t², 1, t)` (and the swapped one
`λ(1, t², t)`) satisfies `Hyp`, with `κ² = αβ`. -/
theorem hyp_of_lam {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) :
    Hyp (lam q t * t) (lam q t * t ^ 2) (lam q t * 1) (-log q) ∧
    Hyp (lam q t * t) (lam q t * 1) (lam q t * t ^ 2) (-log q) := by
  have hq1 : q < 1 := lt_of_lt_of_le hqt ht
  have hl := lam_pos hq hqt ht
  have ht0 : 0 < t := hq.trans hqt
  have hw : 0 < -log q := neg_pos.mpr (log_neg hq hq1)
  have hexp : exp (-(-log q)) = q := by rw [neg_neg, exp_log hq]
  have hcal := (calib_iff (w := -log q) (t := t) (l := lam q t) hw (by rw [hexp]; exact hqt) ht).mpr
    (by rw [hexp])
  refine ⟨⟨by positivity, by positivity, by positivity, hw, by nlinarith, by linarith⟩,
    ⟨by positivity, by positivity, by positivity, hw, by nlinarith, by linarith⟩⟩

end Thm2Cert
```

Thm2Cert/Comparison.lean

```
/-
Certificate for the comparison `M(x) ≥ γ K(x)` of Lemma S5, with the EXPLICIT constant
    γ = κ sinh w − w coth w − |α − β| / 2  > 0,
and for the rigidity inequality of the Supplemental Material:
    Q(O₁ψ) + Q(O₂ψ) ≤ −γ ∑_{l,r} c_l c_r z†_{lr} K(x_{lr}) z_{lr}   (= −γ (‖O₁ψ‖² + ‖O₂ψ‖²)),
for every finite Schmidt spectrum (kernel directions included), and its many-pair version.
Here Q(O_iψ) are the expressions (S:Q12); their derivation from the Euler equation, and the
identity ∑ c_l c_r z† K z = ‖O₁ψ‖² + ‖O₂ψ‖² (S:defects), are certified in `Derivation.lean`.
-/
import Thm2Cert.Identity

open Real Finset

set_option autoImplicit false

noncomputable section

namespace Thm2Cert

/-! ## `f` is even and nonincreasing in `|x|` -/

lemma f_anti {a c : ℝ} (ha : 0 ≤ a) (hac : a ≤ c) : f c ≤ f a := by
  rcases hac.eq_or_lt with h | h
  · subst h; exact le_rfl
  rcases ha.eq_or_lt with h0 | h0
  · subst h0
    rw [f_zero]
    have h1 := f_mul_sinh c
    have h2 : c < sinh c := self_lt_sinh_iff.mpr h
    have h3 : 0 < sinh c := sinh_pos_iff.mpr h
    nlinarith [f_pos c]
```

```
  · have hc : 0 < c := h0.trans h
    have hL := L1 h0 h
    have e1 := f_mul_sinh a
    have e2 := f_mul_sinh c
    have hsa : 0 < sinh a := sinh_pos_iff.mpr h0
    have hsc : 0 < sinh c := sinh_pos_iff.mpr hc
    have key : (f c - f a) * (sinh a * sinh c) = c * sinh a - a * sinh c := by
      linear_combination sinh a * e2 - sinh c * e1
    have : (f c - f a) * (sinh a * sinh c) < 0 := by rw [key]; linarith
    nlinarith [mul_pos hsa hsc]

lemma f_abs (x : ℝ) : f |x| = f x := by
  rcases abs_choice x with h | h
  · rw [h]
  · rw [h, f_neg]

/-! ## Two more half-angle identities -/

lemma minus_identity (κ0 w S C sy cy sxh cxh cx sxw sp cp sr cr F : ℝ)
    (hp : cp ^ 2 - sp ^ 2 = 1) (hr : cr ^ 2 - sr ^ 2 = 1)
    (hw : w = κ0 * (C - 1)) (hC : C = cr ^ 2 + sr ^ 2) (hS : S = 2 * sr * cr)
    (hsy : sy = 2 * sp * cp) (hcy : cy = cp ^ 2 + sp ^ 2)
    (hsxh : sxh = sp * cr + cp * sr) (hcxh : cxh = cp * cr + sp * sr)
    (hcx : cx = cxh ^ 2 + sxh ^ 2) (hsxw : sxw = sp * C + cp * S) :
    S * ((κ0 * S * cy + w * sy + C * F) - (κ0 * S + F * cx))
      = 2 * w * sp ^ 2 + 2 * sp * (sxw * (w - S * F)) := by
  subst hw; subst hcx; subst hsxw; subst hS; subst hC hsy hcy hsxh hcxh
  linear_combination (-2 * cr * sr * (F * cr ^ 2 + F * sr ^ 2 - 2 * cr * sr * κ0)) * hp
    + (-2 * sp ^ 2 * κ0 * (cr ^ 2 - sr ^ 2 + 1)) * hr

lemma plus_identity (κ0 w S C sy cy sxh cxh cx cxw sp cp sr cr F : ℝ)
    (hp : cp ^ 2 - sp ^ 2 = 1) (hr : cr ^ 2 - sr ^ 2 = 1)
    (hw : w = κ0 * (C - 1)) (hC : C = cr ^ 2 + sr ^ 2) (hS : S = 2 * sr * cr)
    (hsy : sy = 2 * sp * cp) (hcy : cy = cp ^ 2 + sp ^ 2)
    (hsxh : sxh = sp * cr + cp * sr) (hcxh : cxh = cp * cr + sp * sr)
    (hcx : cx = cxh ^ 2 + sxh ^ 2) (hcxw : cxw = cp * C + sp * S) :
    S * ((κ0 * S * cy + w * sy + C * F) + (κ0 * S + F * cx))
      = 2 * w * cp ^ 2 + 2 * w * cp * cxw + S * F * (C + cx) := by
  subst hw; subst hcx; subst hcxw; subst hS; subst hC hsy hcy hsxh hcxh
  linear_combination (-2 * κ0 * (cr ^ 4 + sr ^ 4 - 1)) * hp
    + (-2 * κ0 * (sp ^ 2 + 1) * (cr ^ 2 - sr ^ 2 + 1)) * hr

/-! ## Eigenvalue bounds at `κ = κ₀`, `δ = 0`: `N₀ ≥ (w / sinh w) K̃` -/

/-- Both eigenvalues of `M/(2q)` at `κ = κ₀`, `δ = 0` dominate `w / sinh w` times those of
`K/(2q) = [[cosh(x−w), −1], [−1, cosh(x−w)]]`. -/
lemma eig_bounds {w : ℝ} (hw : 0 < w) (κ0 : ℝ) (h3 : κ0 * (cosh w - 1) = w) (x : ℝ) :
    w * (cosh (x - w) - 1) ≤ sinh w * ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w)
        + cosh w * f x) - (κ0 * sinh w + f x * cosh x)) ∧
    w * (cosh (x - w) + 1) ≤ sinh w * ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w)
        + cosh w * f x) + (κ0 * sinh w + f x * cosh x)) := by
  have hp : cosh ((x - w) / 2) ^ 2 - sinh ((x - w) / 2) ^ 2 = 1 := by rw [cosh_sq]; ring
  have hr : cosh (w / 2) ^ 2 - sinh (w / 2) ^ 2 = 1 := by rw [cosh_sq]; ring
  have hCr : cosh w = cosh (w / 2) ^ 2 + sinh (w / 2) ^ 2 := by
    rw [← cosh_two_mul]; congr 1; ring
  have hSr : sinh w = 2 * sinh (w / 2) * cosh (w / 2) := by
    rw [← sinh_two_mul]; congr 1; ring
  have hsy : sinh (x - w) = 2 * sinh ((x - w) / 2) * cosh ((x - w) / 2) := by
    rw [← sinh_two_mul]; congr 1; ring
  have hcy : cosh (x - w) = cosh ((x - w) / 2) ^ 2 + sinh ((x - w) / 2) ^ 2 := by
    rw [← cosh_two_mul]; congr 1; ring
  have hsxh : sinh (x / 2) = sinh ((x - w) / 2) * cosh (w / 2)
      + cosh ((x - w) / 2) * sinh (w / 2) := by
    rw [← sinh_add]; congr 1; ring
  have hcxh : cosh (x / 2) = cosh ((x - w) / 2) * cosh (w / 2)
      + sinh ((x - w) / 2) * sinh (w / 2) := by
    rw [← cosh_add]; congr 1; ring
  have hcx : cosh x = cosh (x / 2) ^ 2 + sinh (x / 2) ^ 2 := by
    rw [← cosh_two_mul]; congr 1; ring
  have hsxw : sinh ((x + w) / 2) = sinh ((x - w) / 2) * cosh w
      + cosh ((x - w) / 2) * sinh w := by
    rw [← sinh_add]; congr 1; ring
  have hcxw : cosh ((x + w) / 2) = cosh ((x - w) / 2) * cosh w
      + sinh ((x - w) / 2) * sinh w := by
    rw [← cosh_add]; congr 1; ring
  have hm := minus_identity κ0 w (sinh w) (cosh w) (sinh (x - w)) (cosh (x - w)) (sinh (x / 2))
    (cosh (x / 2)) (cosh x) (sinh ((x + w) / 2)) (sinh ((x - w) / 2)) (cosh ((x - w) / 2))
    (sinh (w / 2)) (cosh (w / 2)) (f x) hp hr h3.symm hCr hSr hsy hcy hsxh hcxh hcx hsxw
  have hpl := plus_identity κ0 w (sinh w) (cosh w) (sinh (x - w)) (cosh (x - w)) (sinh (x / 2))
    (cosh (x / 2)) (cosh x) (cosh ((x + w) / 2)) (sinh ((x - w) / 2)) (cosh ((x - w) / 2))
    (sinh (w / 2)) (cosh (w / 2)) (f x) hp hr h3.symm hCr hSr hsy hcy hsxh hcxh hcx hcxw
  have hS : 0 < sinh w := sinh_pos_iff.mpr hw
  have hfw : f w * sinh w = w := f_mul_sinh w
  -- sign: sinh((x−w)/2) · sinh((x+w)/2) · (w − sinh w · f x) ≥ 0
  have hsign : 0 ≤ sinh ((x - w) / 2) * (sinh ((x + w) / 2) * (w - sinh w * f x)) := by
    rcases le_total w x with hxw | hxw
    · -- x ≥ w
      have h1 : 0 ≤ sinh ((x - w) / 2) := sinh_nonneg_iff.mpr (by linarith)
      have h2 : 0 ≤ sinh ((x + w) / 2) := sinh_nonneg_iff.mpr (by linarith)
      have h3' : f x ≤ f w := by
        rw [← f_abs x]; exact f_anti hw.le (le_trans hxw (le_abs_self x))
```

```
      have h4 : 0 ≤ w - sinh w * f x := by nlinarith
      exact mul_nonneg h1 (mul_nonneg h2 h4)
    · have h1 : sinh ((x - w) / 2) ≤ 0 := sinh_nonpos_iff.mpr (by linarith)
      rcases le_total (-w) x with hxw' | hxw'
      · -- −w ≤ x ≤ w
        have h2 : 0 ≤ sinh ((x + w) / 2) := sinh_nonneg_iff.mpr (by linarith)
        have h3' : f w ≤ f x := by
          rw [← f_abs x]; exact f_anti (abs_nonneg x) (abs_le.mpr (hxw', hxw))
        have h4 : w - sinh w * f x ≤ 0 := by nlinarith
        have : sinh ((x + w) / 2) * (w - sinh w * f x) ≤ 0 :=
          mul_nonpos_of_nonneg_of_nonpos h2 h4
        exact mul_nonneg_of_nonpos_of_nonpos h1 this
      · -- x ≤ −w
        have h2 : sinh ((x + w) / 2) ≤ 0 := sinh_nonpos_iff.mpr (by linarith)
        have h3' : f x ≤ f w := by
          rw [← f_abs x]; exact f_anti hw.le (by rw [abs_of_nonpos (by linarith)]; linarith)
        have h4 : 0 ≤ w - sinh w * f x := by nlinarith
        have : sinh ((x + w) / 2) * (w - sinh w * f x) ≤ 0 :=
          mul_nonpos_of_nonpos_of_nonneg h2 h4
        exact mul_nonneg_of_nonpos_of_nonpos h1 this
  have hcxw_pos : 0 < cosh ((x + w) / 2) := cosh_pos _
  have hcp : 0 < cosh ((x - w) / 2) := cosh_pos _
  have hCx : 0 < cosh w + cosh x := by linarith [cosh_pos w, cosh_pos x]
  have hw2 : w * (cosh ((x - w) / 2) ^ 2 - sinh ((x - w) / 2) ^ 2) = w := by rw [hp, mul_one]
  constructor
  · rw [hm, hcy]; linarith
  · rw [hpl, hcy]
    have : 0 ≤ sinh w * f x * (cosh w + cosh x) :=
      (mul_pos (mul_pos hS (f_pos x)) hCx).le
    have : 0 ≤ 2 * w * cosh ((x - w) / 2) * cosh ((x + w) / 2) :=
      (mul_pos (mul_pos (by linarith) hcp) hcxw_pos).le
    linarith

/-! ## The explicit constant `γ` -/

/-- `γ = κ sinh w − w coth w − |α − β|/2`. -/
def gam (κ α β w : ℝ) : ℝ := κ * sinh w - w * cosh w / sinh w - |α - β| / 2

variable {κ α β w : ℝ}

lemma gam_mul_sinh (hw : 0 < w) :
    gam κ α β w * sinh w = κ * sinh w ^ 2 - w * cosh w - |α - β| / 2 * sinh w := by
  have hS : sinh w ≠ 0 := (sinh_pos_iff.mpr hw).ne'
  unfold gam
  rw [sub_mul, sub_mul, div_mul_cancel₀ _ hS]; ring

theorem gam_pos (h : Hyp κ α β w) : 0 < gam κ α β w := by
  have hw := h.hw
  have hS : 0 < sinh w := sinh_pos_iff.mpr hw
  have hC1 : 0 < cosh w - 1 := by have := one_lt_cosh.mpr hw.ne'; linarith
  have hSC : sinh w ^ 2 = cosh w ^ 2 - 1 := by rw [cosh_sq]; ring
  have hk := kappa0_le h
  rw [div_le_iff₀ hC1] at hk
  -- R := κ S² − w C > 0
  have hR : 0 < κ * sinh w ^ 2 - w * cosh w := by
    have : κ * sinh w ^ 2 = κ * (cosh w - 1) * (cosh w + 1) := by rw [hSC]; ring
    nlinarith [cosh_pos w]
  -- (|δ| S / 2)² < R²
  have hδ : (α - β) ^ 2 ≤ 4 * ((κ * cosh w - w) ^ 2 - κ ^ 2) := by
    have hs : α + β = 2 * (κ * cosh w - w) := by linarith [h.hcal]
    have e : (α - β) ^ 2 = (α + β) ^ 2 - 4 * (α * β) := by ring
    rw [e, hs]; nlinarith [h.hκαβ]
  have hsq : (|α - β| / 2 * sinh w) ^ 2 < (κ * sinh w ^ 2 - w * cosh w) ^ 2 := by
    have e : (κ * sinh w ^ 2 - w * cosh w) ^ 2
        = ((κ * cosh w - w) ^ 2 - κ ^ 2) * sinh w ^ 2 + w ^ 2 := by
      rw [hSC]; ring
    rw [e, mul_pow, div_pow, sq_abs]
    have : 0 < w ^ 2 := by positivity
    have := mul_le_mul_of_nonneg_right hδ (sq_nonneg (sinh w))
    nlinarith
  have hlt := abs_lt_of_sq_lt_sq hsq hR.le
  have hlt' : |α - β| / 2 * sinh w < κ * sinh w ^ 2 - w * cosh w := lt_of_abs_lt hlt
  have hpos : 0 < gam κ α β w * sinh w := by rw [gam_mul_sinh hw]; linarith
  exact (mul_pos_iff_of_pos_right hS).mp hpos

/-- `γ ≤ α − qκ` and `q γ ≤ κ − q α` (the boundary weights dominate `γ`). -/
lemma gam_le (h : Hyp κ α β w) :
    gam κ α β w ≤ α - exp (-w) * κ ∧ exp (-w) * gam κ α β w ≤ κ - exp (-w) * α := by
  have hw := h.hw
  have hS : 0 < sinh w := sinh_pos_iff.mpr hw
  have hq : cosh w - sinh w = exp (-w) := cosh_sub_sinh w
  have hE : exp (-w) * (cosh w + sinh w) = 1 := by
    rw [cosh_add_sinh, ← exp_add]; simp
  have hwC : w ≤ w * cosh w / sinh w := by
    rw [le_div_iff₀ hS]; nlinarith [sinh_lt_cosh w]
  have hwC0 : 0 ≤ w * cosh w / sinh w := by linarith
  have hab := h.hcal
  have hd1 : -(α - β) ≤ |α - β| := neg_le_abs _
  have hd2 : α - β ≤ |α - β| := le_abs_self _
  have hqk : κ * cosh w - κ * sinh w = exp (-w) * κ := by rw [← mul_sub, hq, mul_comm]
  unfold gam
```

```
  constructor
  · linarith
  · have e : κ - exp (-w) * α - exp (-w) * (κ * sinh w - w * cosh w / sinh w - |α - β| / 2)
        = exp (-w) * (κ * (cosh w + sinh w) - α
          - (κ * sinh w - w * cosh w / sinh w - |α - β| / 2)) := by
      linear_combination (-κ) * hE
    have : 0 ≤ κ * (cosh w + sinh w) - α - (κ * sinh w - w * cosh w / sinh w - |α - β| / 2) := by
      linarith
    nlinarith [exp_pos (-w)]

lemma gam_swap : gam κ α β w = gam κ β α w := by unfold gam; rw [abs_sub_comm]

/-! ## `M(x)/(2q) ≥ γ K(x)/(2q)` -/

lemma amgm_cross (δ sy cy s d : ℝ) (hy : cy ^ 2 - sy ^ 2 = 1) (hc : 1 ≤ cy) :
    2 * δ * sy * s * d ≤ |δ| * ((cy - 1) * s ^ 2 + (cy + 1) * d ^ 2) := by
  have hX : 0 ≤ (cy - 1) * s ^ 2 + (cy + 1) * d ^ 2 := by
    have := sq_nonneg s; have := sq_nonneg d; nlinarith
  have hsq : (2 * sy * s * d) ^ 2 ≤ ((cy - 1) * s ^ 2 + (cy + 1) * d ^ 2) ^ 2 := by
    have : ((cy - 1) * s ^ 2 + (cy + 1) * d ^ 2) ^ 2 - (2 * sy * s * d) ^ 2
        = ((cy - 1) * s ^ 2 - (cy + 1) * d ^ 2) ^ 2 + 4 * s ^ 2 * d ^ 2 * (cy ^ 2 - sy ^ 2 - 1) := by
      ring
    rw [hy, sub_self, mul_zero, add_zero] at this
    nlinarith [sq_nonneg ((cy - 1) * s ^ 2 - (cy + 1) * d ^ 2)]
  have habs := abs_le_of_sq_le_sq hsq hX
  calc 2 * δ * sy * s * d = δ * (2 * sy * s * d) := by ring
    _ ≤ |δ * (2 * sy * s * d)| := le_abs_self _
    _ = |δ| * |2 * sy * s * d| := abs_mul _ _
    _ ≤ |δ| * ((cy - 1) * s ^ 2 + (cy + 1) * d ^ 2) :=
        mul_le_mul_of_nonneg_left habs (abs_nonneg _)

theorem N_comparison (h : Hyp κ α β w) (x u v : ℝ) :
    gam κ α β w * (cosh (x - w) * u ^ 2 - 2 * u * v + cosh (x - w) * v ^ 2)
      ≤ mdiag κ α w x * u ^ 2 - 2 * moff κ w x * u * v + mdiag κ β w x * v ^ 2 := by
  have hw := h.hw
  have hS : 0 < sinh w := sinh_pos_iff.mpr hw
  have hC1 : 0 < cosh w - 1 := by have := one_lt_cosh.mpr hw.ne'; linarith
  set κ0 := w / (cosh w - 1) with hκ0def
  have h3 : κ0 * (cosh w - 1) = w := div_mul_cancel₀ w hC1.ne'
  have hSC : sinh w ^ 2 = cosh w ^ 2 - 1 := by rw [cosh_sq]; ring
  obtain (hAm, hAp) := eig_bounds hw κ0 h3 x
  have hg := gam_mul_sinh (κ := κ) (α := α) (β := β) hw
  -- Γ S = γ S + |δ| S / 2, with Γ = w/S + (κ − κ₀) S
  have hΓ : w + (κ - κ0) * sinh w ^ 2 = gam κ α β w * sinh w + |α - β| / 2 * sinh w := by
    rw [hg, hSC]; linear_combination (-(cosh w + 1)) * h3
  have hy : cosh (x - w) ^ 2 - sinh (x - w) ^ 2 = 1 := by rw [cosh_sq]; ring
  have hcy1 : 1 ≤ cosh (x - w) := one_le_cosh _
  -- P₁ = A − c − γ (cosh y − 1) ≥ |δ|/2 (cosh y − 1), P₂ likewise
  set A := κ * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x with hA
  set c := κ * sinh w + f x * cosh x with hc
  have hP1 : sinh w * (|α - β| / 2 * (cosh (x - w) - 1))
      ≤ sinh w * (A - c - gam κ α β w * (cosh (x - w) - 1)) := by
    have e : sinh w * (A - c - gam κ α β w * (cosh (x - w) - 1))
        = sinh w * ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
          - (κ0 * sinh w + f x * cosh x)) - w * (cosh (x - w) - 1)
          + (cosh (x - w) - 1) * (w + (κ - κ0) * sinh w ^ 2 - gam κ α β w * sinh w) := by
      rw [hA, hc]; ring
    rw [e, hΓ]; nlinarith
  have hP2 : sinh w * (|α - β| / 2 * (cosh (x - w) + 1))
      ≤ sinh w * (A + c - gam κ α β w * (cosh (x - w) + 1)) := by
    have e : sinh w * (A + c - gam κ α β w * (cosh (x - w) + 1))
        = sinh w * ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
          + (κ0 * sinh w + f x * cosh x)) - w * (cosh (x - w) + 1)
          + (cosh (x - w) + 1) * (w + (κ - κ0) * sinh w ^ 2 - gam κ α β w * sinh w) := by
      rw [hA, hc]; ring
    rw [e, hΓ]; nlinarith
  have hP1' := le_of_mul_le_mul_left hP1 hS
  have hP2' := le_of_mul_le_mul_left hP2 hS
  -- diagonal entries through A, using the calibration
  have hmα : mdiag κ α w x = A - (α - β) / 2 * sinh (x - w) := by
    unfold mdiag; rw [hA]; linear_combination sinh (x - w) * h.hcal
  have hmβ : mdiag κ β w x = A + (α - β) / 2 * sinh (x - w) := by
    unfold mdiag; rw [hA]; linear_combination sinh (x - w) * h.hcal
  have hmo : moff κ w x = c := by unfold moff b; rw [hc]
  rw [hmα, hmβ, hmo]
  -- change of variables s = (u+v)/2, d = (u−v)/2
  have e : (A - (α - β) / 2 * sinh (x - w)) * u ^ 2 - 2 * c * u * v
      + (A + (α - β) / 2 * sinh (x - w)) * v ^ 2
      - gam κ α β w * (cosh (x - w) * u ^ 2 - 2 * u * v + cosh (x - w) * v ^ 2)
      = 2 * (A - c - gam κ α β w * (cosh (x - w) - 1)) * ((u + v) / 2) ^ 2
        - 2 * (α - β) * sinh (x - w) * ((u + v) / 2) * ((u - v) / 2)
        + 2 * (A + c - gam κ α β w * (cosh (x - w) + 1)) * ((u - v) / 2) ^ 2 := by ring
  have hcross := amgm_cross (α - β) (sinh (x - w)) (cosh (x - w)) ((u + v) / 2) ((u - v) / 2)
    hy hcy1
  have i1 := mul_le_mul_of_nonneg_right hP1' (sq_nonneg ((u + v) / 2))
  have i2 := mul_le_mul_of_nonneg_right hP2' (sq_nonneg ((u - v) / 2))
  have : 0 ≤ (A - (α - β) / 2 * sinh (x - w)) * u ^ 2 - 2 * c * u * v
      + (A + (α - β) / 2 * sinh (x - w)) * v ^ 2
      - gam κ α β w * (cosh (x - w) * u ^ 2 - 2 * u * v + cosh (x - w) * v ^ 2) := by
    rw [e]; linarith
  linarith
```

```

/-! ## Lemma S5, comparison part: `M(x) ≥ γ K(x)` -/

/-- Diagonal entry of `K(x)`: `e^{−x} + q² eˣ` (the off-diagonal entry is `−2q`). -/
def Kdiag (q x : ℝ) : ℝ := exp (-x) + q ^ 2 * exp x

lemma Kdiag_eq (w x : ℝ) : Kdiag (exp (-w)) x = 2 * exp (-w) * cosh (x - w) := by
  unfold Kdiag
  simp only [cosh_eq, Real.exp_neg, Real.exp_sub, neg_sub]
  field_simp
  ring

/-- The quadratic form of `K(x)`. -/
def QK (w x u v : ℝ) : ℝ :=
  Kdiag (exp (-w)) x * u ^ 2 + 2 * (-2 * exp (-w)) * u * v + Kdiag (exp (-w)) x * v ^ 2

/-- **Lemma S5 (iv)**: `M(x) ≥ γ K(x)` for every real `x`, with the explicit
`γ = κ sinh w − w coth w − |α−β|/2 > 0` (`gam_pos`). -/
theorem lemmaS5_gamma (h : Hyp κ α β w) (x u v : ℝ) :
    gam κ α β w * QK w x u v ≤ QM κ α β w x u v := by
  rw [QM_eq]
  unfold QK
  rw [Kdiag_eq]
  have := N_comparison h x u v
  have hq : 0 < 2 * exp (-w) := by positivity
  have := mul_le_mul_of_nonneg_left this hq.le
  linarith

/-- A real symmetric 2×2 form that is nonnegative on `ℝ²` is nonnegative on `z = (A, conj B)`. -/
lemma psd_complex {p11 p12 p22 : ℝ}
    (hP : ∀ u v : ℝ, 0 ≤ p11 * u ^ 2 + 2 * p12 * u * v + p22 * v ^ 2) (A B : ℂ) :
    0 ≤ p11 * Complex.normSq A + p22 * Complex.normSq B + 2 * p12 * (A * B).re := by
  have hre1 : (A * B).re ≤ ‖A‖ * ‖B‖ := by
    rw [← Complex.norm_mul]; exact Complex.re_le_norm _
  have hre2 : -(‖A‖ * ‖B‖) ≤ (A * B).re := by
    have := Complex.re_le_norm (-(A * B))
    rw [norm_neg, Complex.norm_mul, Complex.neg_re] at this
    linarith
  rw [Complex.normSq_eq_norm_sq, Complex.normSq_eq_norm_sq]
  rcases le_total 0 p12 with hp | hp
  · have := hP ‖A‖ (-‖B‖)
    nlinarith [mul_le_mul_of_nonneg_left hre2 hp]
  · have := hP ‖A‖ ‖B‖
    nlinarith [mul_le_mul_of_nonpos_left hre1 hp]

/-- Complex form of `M(x) ≥ γ K(x)`, for `z = (A, conj B)`. -/
theorem lemmaS5_gamma_complex (h : Hyp κ α β w) (x : ℝ) (A B : ℂ) :
    gam κ α β w * (Kdiag (exp (-w)) x * (Complex.normSq A + Complex.normSq B)
        + 2 * (-2 * exp (-w)) * (A * B).re)
      ≤ Mdiag κ α (exp (-w)) x * Complex.normSq A + Mdiag κ β (exp (-w)) x * Complex.normSq B
        + 2 * Moff κ (exp (-w)) x * (A * B).re := by
  have hP : ∀ u v : ℝ, 0 ≤ (Mdiag κ α (exp (-w)) x - gam κ α β w * Kdiag (exp (-w)) x) * u ^ 2
      + 2 * (Moff κ (exp (-w)) x - gam κ α β w * (-2 * exp (-w))) * u * v
      + (Mdiag κ β (exp (-w)) x - gam κ α β w * Kdiag (exp (-w)) x) * v ^ 2 := by
    intro u v
    have := lemmaS5_gamma h x u v
    unfold QM QK at this
    linarith
  have := psd_complex hP A B
  linarith

/-! ## Rigidity: `Q(O₁ψ) + Q(O₂ψ) ≤ −γ (‖O₁ψ‖² + ‖O₂ψ‖²)` -/

/-- Summand of `‖O₁ψ‖² + ‖O₂ψ‖² = ∑ c_l c_r z† K(x_lr) z` (Eq. S:defects) in the
`p`-variables: `c_l c_r K₁₁ = p_r + q² p_l`, `c_l c_r K₁₂ = −2q c_l c_r`. -/
def kterm (q cl cr : ℝ) (A B : ℂ) : ℝ :=
  (cr ^ 2 + q ^ 2 * cl ^ 2) * (Complex.normSq A + Complex.normSq B)
    - 4 * q * (cl * cr) * (A * B).re

lemma kterm_nonneg {q cl cr : ℝ} (hq : 0 ≤ q) (hl : 0 ≤ cl) (hr : 0 ≤ cr) (A B : ℂ) :
    0 ≤ kterm q cl cr A B := by
  unfold kterm
  have hre : (A * B).re ≤ ‖A‖ * ‖B‖ := by
    rw [← Complex.norm_mul]; exact Complex.re_le_norm _
  have hm : 0 ≤ q * (cl * cr) := mul_nonneg hq (mul_nonneg hl hr)
  have hP : 2 * (q * (cl * cr)) ≤ cr ^ 2 + q ^ 2 * cl ^ 2 := by nlinarith [sq_nonneg (cr - q * cl)]
  have hN : 2 * (‖A‖ * ‖B‖) ≤ ‖A‖ ^ 2 + ‖B‖ ^ 2 := by nlinarith [sq_nonneg (‖A‖ - ‖B‖)]
  have hab : 0 ≤ ‖A‖ * ‖B‖ := by positivity
  rw [Complex.normSq_eq_norm_sq, Complex.normSq_eq_norm_sq]
  have i1 := mul_le_mul_of_nonneg_left hre hm
  have i2 := mul_le_mul_of_nonneg_right hP hab
  have i3 := mul_le_mul_of_nonneg_left hN (by positivity : (0:ℝ) ≤ cr ^ 2 + q ^ 2 * cl ^ 2)
  nlinarith

/-- Termwise comparison, for all Schmidt coefficients `cl, cr ≥ 0`. -/
theorem term_ge (h : Hyp κ α β w) {cl cr : ℝ} (hl : 0 ≤ cl) (hr : 0 ≤ cr) (A B : ℂ) :
    gam κ α β w * kterm (exp (-w)) cl cr A B ≤ term κ α β (exp (-w)) cl cr A B := by
  obtain (ga1, ga2) := gam_le h
  obtain (gb1, gb2) := gam_le h.swap
  rw [← gam_swap] at gb1 gb2
  have he : 0 < exp (-w) := exp_pos _
```

```
  have nA := Complex.normSq_nonneg A
  have nB := Complex.normSq_nonneg B
  rcases hl.eq_or_lt with hl0 | hlp
  · subst hl0
    unfold term wdiag woff kterm
    simp only [ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, zero_pow, zero_mul, mul_zero,
      Real.sqrt_zero, add_zero, sub_zero]
    have i1 := mul_le_mul_of_nonneg_right ga1 (mul_nonneg (sq_nonneg cr) nA)
    have i2 := mul_le_mul_of_nonneg_right gb1 (mul_nonneg (sq_nonneg cr) nB)
    nlinarith
  rcases hr.eq_or_lt with hr0 | hrp
  · subst hr0
    unfold term wdiag woff kterm
    simp only [ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, zero_pow, zero_mul, mul_zero,
      Real.sqrt_zero, add_zero, sub_zero, zero_add]
    have i1 := mul_le_mul_of_nonneg_right ga2 (mul_nonneg (mul_nonneg he.le (sq_nonneg cl)) nA)
    have i2 := mul_le_mul_of_nonneg_right gb2 (mul_nonneg (mul_nonneg he.le (sq_nonneg cl)) nB)
    nlinarith
  · have e1 := E1 hlp hrp
    have e2 := E2 hlp hrp
    have e3 := E3 hlp hrp
    have e4 := E4 hlp hrp
    have keyM : term κ α β (exp (-w)) cl cr A B = cl * cr *
        (Mdiag κ α (exp (-w)) (log (cl / cr)) * Complex.normSq A
          + Mdiag κ β (exp (-w)) (log (cl / cr)) * Complex.normSq B
          + 2 * Moff κ (exp (-w)) (log (cl / cr)) * (A * B).re) := by
      unfold term wdiag woff Mdiag Moff
      linear_combination
        (-exp (-w) * (κ - exp (-w) * α) * Complex.normSq A
          - exp (-w) * (κ - exp (-w) * β) * Complex.normSq B) * e1
        + (-(α - exp (-w) * κ) * Complex.normSq A - (β - exp (-w) * κ) * Complex.normSq B) * e2
        + (1 + exp (-w) ^ 2) * (Complex.normSq A + Complex.normSq B) * e3
        - 2 * exp (-w) * (A * B).re * e4
    have keyK : kterm (exp (-w)) cl cr A B = cl * cr *
        (Kdiag (exp (-w)) (log (cl / cr)) * (Complex.normSq A + Complex.normSq B)
          + 2 * (-2 * exp (-w)) * (A * B).re) := by
      unfold kterm Kdiag
      linear_combination
        (-(Complex.normSq A + Complex.normSq B)) * e2
        + (-exp (-w) ^ 2 * (Complex.normSq A + Complex.normSq B)) * e1
    rw [keyM, keyK]
    have := mul_le_mul_of_nonneg_left (lemmaS5_gamma_complex h (log (cl / cr)) A B)
      (mul_pos hlp hrp).le
    linarith

variable {ι : Type*} [Fintype ι]

/-- **Rigidity inequality** for one pair: `Q(O₁ψ) + Q(O₂ψ) ≤ −γ ∑ c_l c_r z† K z`. -/
theorem rigidity (h : Hyp κ α β w) (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l)
    (A B : ι → ι → ℂ) (nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      ≤ -(gam κ α β w * ∑ l, ∑ r, kterm (exp (-w)) (c l) (c r) (A l r) (B l r)) := by
  rw [mainid_general c A B α β κ (exp (-w)) nA nB hA0 hA1 hB0 hB1, neg_le_neg_iff,
    Finset.mul_sum]
  refine Finset.sum_le_sum (fun l _ => ?_)
  rw [Finset.mul_sum]
  exact Finset.sum_le_sum (fun r _ => term_ge h (hc l) (hc r) (A l r) (B l r))

/-- **Rigidity, many pairs.** For `m` pairs `j` (all sharing the Schmidt coefficients `c`)
with hypotheses `Hyp κ_j α_j β_j w_j`, and any `0 < γ₀ ≤ min_j γ_j`:
`∑_j [Q(O₁,ⱼψ) + Q(O₂,ⱼψ)] ≤ −γ₀ ∑_j ∑ c_l c_r z†_j K_j z_j`. In particular, if the left side
is `≥ 0` (second-order stability with `⟨O_{i,j}⟩ = 0`), every defect sum vanishes. -/
theorem rigidity_multi {m : ℕ} (κ α β w : Fin m → ℝ) (h : ∀ j, Hyp (κ j) (α j) (β j) (w j))
    (γ0 : ℝ) (hγ0 : 0 < γ0) (hγ : ∀ j, γ0 ≤ gam (κ j) (α j) (β j) (w j))
    (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : Fin m → ι → ι → ℂ) (nA nB : Fin m → ℝ)
    (hA0 : ∀ j, nA j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A j l r))
    (hA1 : ∀ j, nA j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A j l r))
    (hB0 : ∀ j, nB j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B j l r))
    (hB1 : ∀ j, nB j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B j l r)) :
    (∑ j, (Q1 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))
      + Q2 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))))
      ≤ -(γ0 * ∑ j, ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r)) ∧
    (0 ≤ ∑ j, (Q1 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))
      + Q2 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))) →
      ∀ j, ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r) = 0) := by
  have hK : ∀ j, 0 ≤ ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r) := fun j =>
    Finset.sum_nonneg (fun l _ => Finset.sum_nonneg (fun r _ =>
      kterm_nonneg (exp_pos _).le (hc l) (hc r) _ _))
  have hj : ∀ j, (Q1 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j))
        (sfA c (A j)) (sfA c (B j)) (sfC c (A j) (B j))
      + Q2 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j)))
```

```
      ≤ -(γ0 * ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r)) := by
    intro j
    have := rigidity (h j) c hc (A j) (B j) (nA j) (nB j) (hA0 j) (hA1 j) (hB0 j) (hB1 j)
    have := mul_le_mul_of_nonneg_right (hγ j) (hK j)
    linarith
  have hsum := Finset.sum_le_sum (fun j (_ : j ∈ Finset.univ) => hj j)
  rw [Finset.sum_neg_distrib, ← Finset.mul_sum] at hsum
  refine (hsum, fun h0 j => ?_)
  have htot : γ0 * ∑ j, ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r) ≤ 0 := by
    linarith
  have hnn : 0 ≤ ∑ j, ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r) :=
    Finset.sum_nonneg (fun j _ => hK j)
  have hle : ∑ j, ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r) ≤ 0 := by
    by_contra hcon
    have := mul_pos hγ0 (not_le.mp hcon)
    linarith
  have hzero := le_antisymm hle hnn
  exact (Finset.sum_eq_zero_iff_of_nonneg (fun j _ => hK j)).mp hzero j (Finset.mem_univ j)

end Thm2Cert
```

Thm2Cert/Derivation.lean

```
/-
Certificate for the derivation of Eqs. (S:Q12) and (S:defects) of the Supplemental Material,
in the Schmidt-basis matrix model:

  * coefficient matrices: ψ ↦ C = diag(c_l); (a ψ)_lm = A_lm c_m, (b† ψ)_lm = c_l conj(B_lm),
    (a† ψ)_lm = conj(A_ml) c_m, (b ψ)_lm = c_l B_ml, with A_lr = ⟨u_l|a|u_r⟩, B_lr = ⟨v_l|b|v_r⟩;
  * test vectors O₁ψ = (a − q b†)ψ, O₂ψ = (q a† − b)ψ;
  * (S:defects): ‖O₁ψ‖² + ‖O₂ψ‖² = ∑ c_l c_r z† K z  (exact, any real c_l);
  * (S:Q12): Q(O_iψ) = Re⟨O_iψ, (T − μ − F) O_iψ⟩ equals the stated expressions, where
    (T − μ) O ψ is replaced by O 𝒢(C) + [T, O] ψ (Euler equation (S:euler)), with
    𝒢(C) = diag(c_l ln p_l), [T, a − q b†] = (qκ − α) a + (κ − qβ) b†,
    [T, q a† − b] = (qα − κ) a† + (β − qκ) b, and F the Daleckii–Krein divided difference
    F_lr = (p_l ln p_l − p_r ln p_r)/(p_l − p_r), F_ll = 1 + ln p_l  (c_l ≥ 0, kernel entries by continuity, with 0 ln 0 = 0).
  * `stationary_rigidity`: chaining (S:Q12), (S:mainid), Lemma S5 (M ≥ γK) and (S:defects):
    if the second variations along all test vectors sum to ≥ 0, every O_{i,j}ψ vanishes.

Inputs (not derived here):
  * the matrix model itself: ψ in Schmidt form on a finite index set (support plus the finitely
    many kernel directions reached by a, a†, b, b†), with A_lr, B_lr the matrix elements;
    `O1v`, `O2v` are DEFINED as the resulting coefficient matrices;
  * the sum rules (S:wccr) (consequences of the canonical commutation relations);
  * the Euler equation (S:euler) and the commutators [T, O] (CCR), which enter by writing
    (T − μ)Oψ as O𝒢 + [T,O]ψ directly in the statements;
  * the second-variation formula Q(Y) = Re⟨Y, (T − μ − F)Y⟩ (Daleckii–Krein) and the
    stability inequality (S:stability), which enters as the hypothesis `0 ≤ ∑ Q`.
-/
import Thm2Cert.Comparison

open Real Finset ComplexConjugate

set_option autoImplicit false

noncomputable section

namespace Thm2Cert

variable {ι : Type*} [Fintype ι]

/-! ## (S:defects) -/

/-- Coefficient matrix of `O₁ψ = (a − q b†)ψ`. -/
def O1v (q : ℝ) (c : ι → ℝ) (A B : ι → ι → ℂ) (l m : ι) : ℂ :=
  A l m * (c m : ℂ) - (q : ℂ) * (c l : ℂ) * conj (B l m)

/-- Coefficient matrix of `O₂ψ = (q a† − b)ψ`. -/
def O2v (q : ℝ) (c : ι → ℝ) (A B : ι → ι → ℂ) (l m : ι) : ℂ :=
  (q : ℂ) * (c m : ℂ) * conj (A m l) - (c l : ℂ) * B m l

lemma defect_term (q cl cr : ℝ) (A B : ℂ) :
    Complex.normSq (A * (cr : ℂ) - (q : ℂ) * (cl : ℂ) * conj B)
      + Complex.normSq ((q : ℂ) * (cl : ℂ) * conj A - (cr : ℂ) * B) = kterm q cl cr A B := by
  unfold kterm
  simp only [Complex.normSq_apply, Complex.sub_re, Complex.sub_im, Complex.mul_re,
    Complex.mul_im, Complex.ofReal_re, Complex.ofReal_im, Complex.conj_re, Complex.conj_im]
  ring

/-- **Eq. (S:defects)**: `‖O₁ψ‖² + ‖O₂ψ‖² = ∑_{l,r} c_l c_r z† K(x_lr) z` (in `p`-variables). -/
theorem defects (q : ℝ) (c : ι → ℝ) (A B : ι → ι → ℂ) :
    ∑ l, ∑ m, Complex.normSq (O1v q c A B l m) + ∑ l, ∑ m, Complex.normSq (O2v q c A B l m)
      = ∑ l, ∑ r, kterm q (c l) (c r) (A l r) (B l r) := by
  rw [Finset.sum_comm (f := fun l m => Complex.normSq (O2v q c A B l m)), ← Finset.sum_add_distrib]
  refine Finset.sum_congr rfl (fun l _ => ?_)
  rw [← Finset.sum_add_distrib]
  refine Finset.sum_congr rfl (fun r _ => ?_)
  unfold O1v O2v
  exact defect_term q (c l) (c r) (A l r) (B l r)
```

```
/-- If both test vectors have zero norm, all their coefficients vanish. -/
theorem defects_zero (q : ℝ) (c : ι → ℝ) (A B : ι → ι → ℂ)
    (h0 : ∑ l, ∑ r, kterm q (c l) (c r) (A l r) (B l r) = 0) :
    ∀ l m, O1v q c A B l m = 0 ∧ O2v q c A B l m = 0 := by
  rw [← defects] at h0
  have n1 : 0 ≤ ∑ l, ∑ m, Complex.normSq (O1v q c A B l m) :=
    Finset.sum_nonneg (fun _ _ => Finset.sum_nonneg (fun _ _ => Complex.normSq_nonneg _))
  have n2 : 0 ≤ ∑ l, ∑ m, Complex.normSq (O2v q c A B l m) :=
    Finset.sum_nonneg (fun _ _ => Finset.sum_nonneg (fun _ _ => Complex.normSq_nonneg _))
  have z1 : ∑ l, ∑ m, Complex.normSq (O1v q c A B l m) = 0 := by linarith
  have z2 : ∑ l, ∑ m, Complex.normSq (O2v q c A B l m) = 0 := by linarith
  intro l m
  have hz : ∀ (g : ι → ι → ℂ), ∑ l, ∑ m, Complex.normSq (g l m) = 0 → g l m = 0 := by
    intro g hg
    have hl := (Finset.sum_eq_zero_iff_of_nonneg (fun i _ => Finset.sum_nonneg
      (fun j _ => Complex.normSq_nonneg (g i j)))).mp hg l (Finset.mem_univ l)
    have hm := (Finset.sum_eq_zero_iff_of_nonneg (fun j _ =>
      Complex.normSq_nonneg (g l j))).mp hl m (Finset.mem_univ m)
    exact Complex.normSq_eq_zero.mp hm
  exact (hz _ z1, hz _ z2)

/-! ## Daleckii–Krein divided difference -/

/-- `F(p, p') = (p ln p − p' ln p')/(p − p')`, `= 1 + ln p` on the diagonal. -/
def Fdk (p p' : ℝ) : ℝ := if p = p' then 1 + log p else (p * log p - p' * log p') / (p - p')

lemma Lker_symm (p p' : ℝ) : Lker p p' = Lker p' p := by
  unfold Lker
  by_cases h : p = p'
  · subst h; rfl
  · have h' : p' ≠ p := fun e => h e.symm
    simp only [h, h', ↓reduceIte]
    rw [← neg_div_neg_eq]; ring_nf

lemma Fdk_symm (p p' : ℝ) : Fdk p p' = Fdk p' p := by
  unfold Fdk
  by_cases h : p = p'
  · subst h; rfl
  · have h' : p' ≠ p := fun e => h e.symm
    simp only [h, h', ↓reduceIte]
    rw [← neg_div_neg_eq]; ring_nf

/-- `ln p_r − F_lr = −p_l L_lr` and `ln p_l − F_lr = −p_r L_lr` (multiplied by `c`). -/
lemma F_rel {cl cm : ℝ} (hl : 0 < cl) (hm : 0 < cm) :
    cm * log (cm ^ 2) = cm * Fdk (cl ^ 2) (cm ^ 2) - cl ^ 2 * Lker (cl ^ 2) (cm ^ 2) * cm ∧
    cl * log (cl ^ 2) = cl * Fdk (cl ^ 2) (cm ^ 2) - cm ^ 2 * Lker (cl ^ 2) (cm ^ 2) * cl := by
  have hl2 : 0 < cl ^ 2 := by positivity
  have hm2 : 0 < cm ^ 2 := by positivity
  unfold Fdk Lker
  by_cases h : cl ^ 2 = cm ^ 2
  · simp only [h, ↓reduceIte]
    constructor <;> field_simp <;> ring
  · simp only [h, ↓reduceIte]
    have hd : cl ^ 2 - cm ^ 2 ≠ 0 := sub_ne_zero.mpr h
    constructor <;> field_simp <;> ring

/-- `F_rel` for `c ≥ 0`: with the conventions `0 ln 0 = 0` and the continuous values
`F_lr = ln p_l` if `p_r = 0 < p_l`, the relations persist on kernel entries. -/
lemma F_rel' {cl cm : ℝ} (hl : 0 ≤ cl) (hm : 0 ≤ cm) :
    cm * log (cm ^ 2) = cm * Fdk (cl ^ 2) (cm ^ 2) - cl ^ 2 * Lker (cl ^ 2) (cm ^ 2) * cm ∧
    cl * log (cl ^ 2) = cl * Fdk (cl ^ 2) (cm ^ 2) - cm ^ 2 * Lker (cl ^ 2) (cm ^ 2) * cl := by
  rcases hl.eq_or_lt with h0 | hlp
  · subst h0
    rcases hm.eq_or_lt with h1 | hmp
    · subst h1; simp
    · have hm2 : (0:ℝ) ≠ cm ^ 2 := (by positivity : (0:ℝ) < cm ^ 2).ne
      unfold Fdk
      simp only [ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, zero_pow, hm2, ↓reduceIte,
        log_zero, zero_mul, zero_sub, mul_zero, sub_zero, zero_mul]
      have : cm ^ 2 ≠ 0 := fun e => hm2 e.symm
      refine ⟨?_, ?_⟩ <;> first | trivial | field_simp
  · rcases hm.eq_or_lt with h1 | hmp
    · subst h1
      have hl2 : cl ^ 2 ≠ 0 := (by positivity : (0:ℝ) < cl ^ 2).ne'
      unfold Fdk
      simp only [ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, zero_pow, hl2, ↓reduceIte,
        log_zero, mul_zero, sub_zero, zero_mul]
      refine ⟨?_, ?_⟩ <;> first | trivial | field_simp
    · exact F_rel hlp hmp

/-! ## (S:Q12), first line -/

lemma Q1_term (q κ α β cl cm gl gm F L : ℝ) (A B : ℂ)
    (hgm : gm = cm * F - cl ^ 2 * L * cm) (hgl : gl = cl * F - cm ^ 2 * L * cl) :
    (conj (A * (cm : ℂ) - (q : ℂ) * (cl : ℂ) * conj B)
      * ((A * (gm : ℂ) - (q : ℂ) * (gl : ℂ) * conj B
          + ((q * κ - α : ℝ) : ℂ) * A * (cm : ℂ) + ((κ - q * β : ℝ) : ℂ) * (cl : ℂ) * conj B)
        - (F : ℂ) * (A * (cm : ℂ) - (q : ℂ) * (cl : ℂ) * conj B))).re
    = (q * κ - α) * (cm ^ 2 * Complex.normSq A)
      + (q ^ 2 * β - q * κ) * (cl ^ 2 * Complex.normSq B)
```

```
      + (κ * (1 - q ^ 2) + q * (α - β)) * (cl * cm * (A * B).re)
      - cl ^ 2 * cm ^ 2 * L * Complex.normSq A
      - q ^ 2 * (cl ^ 2 * cm ^ 2 * L * Complex.normSq B)
      + q * (cl * cm * (cl ^ 2 + cm ^ 2) * L * (A * B).re) := by
  subst hgm hgl
  simp only [Complex.normSq_apply, Complex.sub_re, Complex.sub_im, Complex.add_re,
    Complex.add_im, Complex.mul_re, Complex.mul_im, Complex.ofReal_re, Complex.ofReal_im,
    Complex.conj_re, Complex.conj_im]
  ring

/-- **Eq. (S:Q12), first line**, derived from the Euler equation and `[T, O₁]`. -/
theorem Q1_from_euler (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : ι → ι → ℂ)
    (α β κ q nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    (∑ l, ∑ m, conj (O1v q c A B l m)
      * ((A l m * ((c m * log (c m ^ 2) : ℝ) : ℂ)
          - (q : ℂ) * ((c l * log (c l ^ 2) : ℝ) : ℂ) * conj (B l m)
          + ((q * κ - α : ℝ) : ℂ) * A l m * (c m : ℂ)
          + ((κ - q * β : ℝ) : ℂ) * (c l : ℂ) * conj (B l m))
        - (Fdk (c l ^ 2) (c m ^ 2) : ℂ) * O1v q c A B l m)).re
    = Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B) := by
  unfold Q1
  rw [hA0, hB1]
  unfold kk sfA sfC
  simp only [Complex.re_sum, Finset.mul_sum, ← Finset.sum_add_distrib, ← Finset.sum_sub_distrib]
  refine Finset.sum_congr rfl (fun l _ => Finset.sum_congr rfl (fun m _ => ?_))
  obtain (h1, h2) := F_rel' (hc l) (hc m)
  have hs : √(c l ^ 2 * c m ^ 2) = c l * c m := by
    rw [show c l ^ 2 * c m ^ 2 = (c l * c m) * (c l * c m) by ring]
    exact sqrt_mul_self (mul_nonneg (hc l) (hc m))
  unfold O1v
  rw [Q1_term q κ α β (c l) (c m) _ _ (Fdk (c l ^ 2) (c m ^ 2)) (Lker (c l ^ 2) (c m ^ 2))
    (A l m) (B l m) h1 h2, hs]

/-! ## (S:Q12), second line -/

lemma Q2_term (q κ α β cx cy gx gy F L : ℝ) (A B : ℂ)
    (hgx : gx = cx * F - cy ^ 2 * L * cx) (hgy : gy = cy * F - cx ^ 2 * L * cy) :
    (conj ((q : ℂ) * (cx : ℂ) * conj A - (cy : ℂ) * B)
      * (((q : ℂ) * (gx : ℂ) * conj A - (gy : ℂ) * B
          + ((q * α - κ : ℝ) : ℂ) * (cx : ℂ) * conj A + ((β - q * κ : ℝ) : ℂ) * (cy : ℂ) * B)
        - (F : ℂ) * ((q : ℂ) * (cx : ℂ) * conj A - (cy : ℂ) * B))).re
    = (q ^ 2 * α - q * κ) * (cx ^ 2 * Complex.normSq A)
      + (q * κ - β) * (cy ^ 2 * Complex.normSq B)
      + (κ * (1 - q ^ 2) - q * (α - β)) * (cx * cy * (A * B).re)
      - q ^ 2 * (cx ^ 2 * cy ^ 2 * L * Complex.normSq A)
      - cx ^ 2 * cy ^ 2 * L * Complex.normSq B
      + q * (cx * cy * (cx ^ 2 + cy ^ 2) * L * (A * B).re) := by
  subst hgx hgy
  simp only [Complex.normSq_apply, Complex.sub_re, Complex.sub_im, Complex.add_re,
    Complex.add_im, Complex.mul_re, Complex.mul_im, Complex.ofReal_re, Complex.ofReal_im,
    Complex.conj_re, Complex.conj_im]
  ring

/-- **Eq. (S:Q12), second line**, derived from the Euler equation and `[T, O₂]`. -/
theorem Q2_from_euler (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : ι → ι → ℂ)
    (α β κ q nA nB : ℝ)
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r)) :
    (∑ l, ∑ m, conj (O2v q c A B l m)
      * (((q : ℂ) * ((c m * log (c m ^ 2) : ℝ) : ℂ) * conj (A m l)
          - ((c l * log (c l ^ 2) : ℝ) : ℂ) * B m l
          + ((q * α - κ : ℝ) : ℂ) * (c m : ℂ) * conj (A m l)
          + ((β - q * κ : ℝ) : ℂ) * (c l : ℂ) * B m l)
        - (Fdk (c l ^ 2) (c m ^ 2) : ℂ) * O2v q c A B l m)).re
    = Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B) := by
  rw [Finset.sum_comm]
  unfold Q2
  rw [hA1, hB0]
  unfold kk sfA sfC
  simp only [Complex.re_sum, Finset.mul_sum, ← Finset.sum_add_distrib, ← Finset.sum_sub_distrib]
  refine Finset.sum_congr rfl (fun x _ => Finset.sum_congr rfl (fun y _ => ?_))
  obtain (h1, h2) := F_rel' (hc y) (hc x)
  have hs : √(c x ^ 2 * c y ^ 2) = c x * c y := by
    rw [show c x ^ 2 * c y ^ 2 = (c x * c y) * (c x * c y) by ring]
    exact sqrt_mul_self (mul_nonneg (hc x) (hc y))
  rw [Lker_symm] at h1 h2
  unfold O2v
  rw [Q2_term q κ α β (c x) (c y) _ _ (Fdk (c y ^ 2) (c x ^ 2)) (Lker (c x ^ 2) (c y ^ 2))
    (A x y) (B x y) h1 h2, hs]

/-! ## Rigidity at a stable stationary point -/

/-- **Rigidity.** For `m` pairs with the hypotheses of Lemma S5, Schmidt coefficients `c ≥ 0`
and the sum rules, if the second variations (S:Q12) along all test vectors `O₁,ⱼψ, O₂,ⱼψ` —
computed from the Euler equation — sum to a nonnegative number, then every test vector
vanishes: `(a_j − q_j b_j†)ψ = 0` and `(q_j a_j† − b_j)ψ = 0` for all `j`. -/
theorem stationary_rigidity {m : ℕ} (κ α β w : Fin m → ℝ)
    (h : ∀ j, Hyp (κ j) (α j) (β j) (w j))
    (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : Fin m → ι → ι → ℂ) (nA nB : Fin m → ℝ)
```

```
    (hA0 : ∀ j, nA j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A j l r))
    (hA1 : ∀ j, nA j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A j l r))
    (hB0 : ∀ j, nB j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B j l r))
    (hB1 : ∀ j, nB j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B j l r))
    (hstab : 0 ≤ ∑ j,
      ((∑ l, ∑ m', conj (O1v (exp (-w j)) c (A j) (B j) l m')
        * ((A j l m' * ((c m' * log (c m' ^ 2) : ℝ) : ℂ)
            - ((exp (-w j) : ℝ) : ℂ) * ((c l * log (c l ^ 2) : ℝ) : ℂ) * conj (B j l m')
            + ((exp (-w j) * κ j - α j : ℝ) : ℂ) * A j l m' * (c m' : ℂ)
            + ((κ j - exp (-w j) * β j : ℝ) : ℂ) * (c l : ℂ) * conj (B j l m'))
          - (Fdk (c l ^ 2) (c m' ^ 2) : ℂ) * O1v (exp (-w j)) c (A j) (B j) l m')).re
      + (∑ l, ∑ m', conj (O2v (exp (-w j)) c (A j) (B j) l m')
        * ((((exp (-w j) : ℝ) : ℂ) * ((c m' * log (c m' ^ 2) : ℝ) : ℂ) * conj (A j m' l)
            - ((c l * log (c l ^ 2) : ℝ) : ℂ) * B j m' l
            + ((exp (-w j) * α j - κ j : ℝ) : ℂ) * (c m' : ℂ) * conj (A j m' l)
            + ((β j - exp (-w j) * κ j : ℝ) : ℂ) * (c l : ℂ) * B j m' l)
          - (Fdk (c l ^ 2) (c m' ^ 2) : ℂ) * O2v (exp (-w j)) c (A j) (B j) l m')).re)) :
    ∀ j l m', O1v (exp (-w j)) c (A j) (B j) l m' = 0 ∧ O2v (exp (-w j)) c (A j) (B j) l m' = 0 := by
  have hQ : ∀ j, ((∑ l, ∑ m', conj (O1v (exp (-w j)) c (A j) (B j) l m')
        * ((A j l m' * ((c m' * log (c m' ^ 2) : ℝ) : ℂ)
            - ((exp (-w j) : ℝ) : ℂ) * ((c l * log (c l ^ 2) : ℝ) : ℂ) * conj (B j l m')
            + ((exp (-w j) * κ j - α j : ℝ) : ℂ) * A j l m' * (c m' : ℂ)
            + ((κ j - exp (-w j) * β j : ℝ) : ℂ) * (c l : ℂ) * conj (B j l m'))
          - (Fdk (c l ^ 2) (c m' ^ 2) : ℂ) * O1v (exp (-w j)) c (A j) (B j) l m')).re
      + (∑ l, ∑ m', conj (O2v (exp (-w j)) c (A j) (B j) l m')
        * ((((exp (-w j) : ℝ) : ℂ) * ((c m' * log (c m' ^ 2) : ℝ) : ℂ) * conj (A j m' l)
            - ((c l * log (c l ^ 2) : ℝ) : ℂ) * B j m' l
            + ((exp (-w j) * α j - κ j : ℝ) : ℂ) * (c m' : ℂ) * conj (A j m' l)
            + ((β j - exp (-w j) * κ j : ℝ) : ℂ) * (c l : ℂ) * B j m' l)
          - (Fdk (c l ^ 2) (c m' ^ 2) : ℂ) * O2v (exp (-w j)) c (A j) (B j) l m')).re)
      = Q1 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
          (sfA c (B j)) (sfC c (A j) (B j))
        + Q2 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
          (sfA c (B j)) (sfC c (A j) (B j)) := by
    intro j
    rw [Q1_from_euler c hc (A j) (B j) (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (hA0 j) (hB1 j),
      Q2_from_euler c hc (A j) (B j) (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (hA1 j) (hB0 j)]
  simp only [hQ] at hstab
  -- γ₀ = min_j γ_j (or 1 if there are no pairs)
  obtain ⟨γ0, hγ0, hγ⟩ : ∃ γ0 : ℝ, 0 < γ0 ∧ ∀ j, γ0 ≤ gam (κ j) (α j) (β j) (w j) := by
    rcases Nat.eq_zero_or_pos m with hm | hm
    · subst hm; exact ⟨1, one_pos, fun j => j.elim0⟩
    · have : Nonempty (Fin m) := ⟨⟨0, hm⟩⟩
      obtain ⟨j0, -, hj0⟩ := Finset.exists_min_image Finset.univ
        (fun j => gam (κ j) (α j) (β j) (w j)) Finset.univ_nonempty
      exact ⟨_, gam_pos (h j0), fun j => hj0 j (Finset.mem_univ j)⟩
  have hz := (rigidity_multi κ α β w h γ0 hγ0 hγ c hc A B nA nB hA0 hA1 hB0 hB1).2 hstab
  intro j l m'
  exact defects_zero (exp (-w j)) c (A j) (B j) (hz j) l m'

end Thm2Cert
```